

Claude CCAF - Practice

Every practice question with its answer, full explanation and exam tip.

252 questions across 31 chapters

Generated 2026-08-24

1.1 Agentic Loops, stop_reason & Loop Termination

Claude CCAF: Agentic Architecture & Orchestration - 45 questions

1. Your agentic loop checks if `response.content[0].type` equals "text" to determine completion. The agent sometimes terminates prematurely mid-task; production logs show tasks incomplete when the agent could continue. What causes premature termination?

- (A) Checking for text content instead of `stop_reason` creates false termination signal.
- (B) Agent lacks sufficient tools to complete tasks requiring premature termination.
- (C) Context window exhaustion prevents agent from continuing with additional iterations.
- (D) Max iteration count reached before task completion causing early stop.

Answer: (A) Checking for text content instead of `stop_reason` creates false termination signal.

EXPLANATION

A single assistant turn can contain **mixed** content blocks - Claude routinely emits a `text` block ("I'll look up that order first...") followed by one or more `tool_use` blocks in the same message. Because `content[0]` is the **first** block, a leading narration block makes the check fire even though the turn's `stop_reason` is "tool_use" and work remains. A block's type describes **what a piece of the message is**, not **whether the turn is complete**; only `stop_reason` carries that meaning.

Why distractors fail:

- **B** - Missing tools produce a different signature: the model says it cannot proceed, or hallucinates a tool. That does not make a content-type check fire early.
- **C** - Context exhaustion surfaces as `stop_reason: "max_tokens"` or an API error, not as a silent early exit through this code path.
- **D** - The stem states the loop terminates on a content-type check; no iteration cap is described, and a cap would truncate at a fixed count rather than "sometimes, mid-task."

EXAM TIP

Exam Rule: - A single assistant turn can contain **mixed** content blocks - Claude routinely emits a `text` block ("I'll look up that order first...") follow

2. An agent repeatedly calls the same tool with identical parameters after receiving successful responses, creating an infinite loop. What is the most appropriate architectural safeguard?

- (A) Increase the maximum iteration limit so the agent has more opportunities to finish.
- (B) Add explicit loop detection by tracking repeated tool invocations and terminate or escalate when repetition is detected.
- (C) Improve the system prompt instructing the agent not to repeat tool calls.
- (D) Add more tools so the agent has additional choices.

Answer: (B) Add explicit loop detection by tracking repeated tool invocations and terminate or escalate when repetition is detected.

EXPLANATION

The failure is **spinning**: repeated action with no new information. The harness - not the model - is the reliable place to detect it, because the harness holds a deterministic record of every `(tool_name, input)` pair it has executed. Hashing invocations and tripping on repeats gives a hard guarantee, and it gives you an **escalation path** (surface to a human, inject a corrective observation, or terminate with a partial result) instead of a silent hang.

Why distractors fail:

- **A** - Raising the limit makes an unbounded loop **more** expensive, not bounded. The loop never converges; extra headroom just burns tokens.
- **C** - Prompt instructions are probabilistic guidance. They reduce frequency at best; they cannot guarantee termination, and this is a reliability requirement.
- **D** - More tools widen the action space and typically worsen selection confusion. It does nothing about repetition of one call.

EXAM TIP

Exam Rule: - The failure is **spinning**: repeated action with no new information. The harness - not the model - is the reliable place to detect it, because

3. The loop continues while `stop_reason == "tool_use"` and terminates on `"end_turn"`. The agent sometimes continues after completing the task; logs show `end_turn` received but the loop continues. What causes continued iteration after completion?
- (A) Context window size affects `stop_reason` reliability in long conversations.
 - (B) Tool results appended to conversation causing unexpected `tool_use stop_reason`.
 - (C) Multiple `stop_reason` values can occur requiring checking all values.
 - (D) Checking additional conditions beyond `stop_reason` creates continuation logic errors; `stop_reason="end_turn"` is definitive completion.

Answer: (D) Checking additional conditions beyond `stop_reason` creates continuation logic errors; `stop_reason="end_turn"` is definitive completion.

EXPLANATION

`stop_reason` is a **single scalar** on each response, and `"end_turn"` is authoritative: the model finished its turn of its own accord. If the loop still iterates after receiving it, the only remaining explanation is that some *additional* condition in the loop predicate (a text check, a "did we get a final answer?" heuristic, a leftover flag) is overriding the definitive signal. The fix is to make `stop_reason` the sole continuation predicate.

Why distractors fail:

- **A** - `stop_reason` is not degraded by conversation length; a near-full window produces `max_tokens` or an error, not an unreliable `end_turn`.
- **B** - Appending tool results is required behaviour and cannot retroactively change a `stop_reason` you have already received.
- **C** - A response carries exactly one `stop_reason`. There is no set of simultaneous values to check.

EXAM TIP

Exam Rule: - `stop_reason` is a single scalar on each response, and `"end_turn"` is authoritative: the model finished its turn of its own accord. If the

4. A production loop terminates when the response contains text content, assuming completion. The agent sometimes provides an explanation then continues with `tool_use`, so the loop terminates before tool execution. What is the correct loop termination signal?
- (A) Presence of text content in response indicates agent has completed reasoning and task.
 - (B) Maximum iteration count threshold as primary termination mechanism for safety.
 - (C) Check `stop_reason` value for `end_turn` as authoritative completion signal, not text content.
 - (D) Count `tool_use` occurrences in response; zero tool calls indicates completion.

Answer: (C) Check `stop_reason` value for `end_turn` as authoritative completion signal, not text content.

EXPLANATION

Same root cause as UD1-005, stated from the opposite direction. Text and tool-use blocks coexist inside one turn; the API's explicit completion contract is `stop_reason == "end_turn"`. Reading it is O(1), unambiguous, and stable across model versions.

Why distractors fail:

- **A** - This *is* the bug described in the stem.
- **B** - An iteration cap is a backstop, not a completion signal; used as the primary mechanism it truncates unfinished work and lets finished work spin to the ceiling.
- **D** - Counting `tool_use` blocks is a *derived proxy*. It coincides with `end_turn` in the common case but silently mishandles turns that stop for other reasons - `max_tokens`, `pause_turn`, `refusal` - which also contain zero tool calls yet are not successful completions. Read the field; don't infer it.

EXAM TIP

Exam Rule: - Same root cause as UD1-005, stated from the opposite direction. Text and tool-use blocks coexist inside one turn; the API's explicit complet

5. The loop continues while `stop_reason == tool_use` and stops on `end_turn`. It *also* checks whether the response contains "DONE" text, triggering termination. What causes incorrect early loop termination?
- (A) Text-based completion detection in addition to `stop_reason` creates false termination triggers.
 - (B) Text checking necessary as backup when `stop_reason` values are unreliable.
 - (C) Both `stop_reason` and text conditions required for robust production termination.
 - (D) Multiple termination conditions improve robustness catching all possible completion signals.

Answer: (A) Text-based completion detection in addition to `stop_reason` creates false termination triggers.

EXPLANATION

The added text check is a second, weaker trigger **OR-ed** into the termination predicate. "DONE" can legitimately appear inside reasoning ("the migration is not DONE until..."), inside quoted log output, or as a code identifier - so the loop exits on a coincidence. Adding conditions to an OR broadens termination; it does not harden it.

Why distractors fail:

- **B** - `stop_reason` is a documented, guaranteed field. "Unreliable" is a false premise.
- **C** - Requiring *both* would be an AND, which the stem does not describe - and it would then hang whenever the model finishes without saying "DONE".

- **D** - Inverts the reliability logic: more OR-ed triggers means more false positives, not more robustness.

EXAM TIP

Exam Rule: - The added text check is a second, weaker trigger OR-ed into the termination predicate. "DONE" can legitimately appear inside reasoning ("the

6. A developer's Agent SDK agent definition includes 3 tools and a system prompt. When they call `agent.run()`, what happens internally?

- (A) The SDK sends the tools to a separate tool execution service and polls for results.
- (B) The SDK makes a single API call with all tools and returns the complete result.
- (C) The SDK enters an agentic loop: sends messages to Claude, receives a response, checks for `tool_use` blocks, executes any tools, appends tool results to the conversation, and continues until Claude responds with `end_turn`.
- (D) The SDK makes parallel API calls.

Answer: (C) The SDK enters an agentic loop: sends messages to Claude, receives a response, checks for `tool_use` blocks, executes any tools, appends tool results to the conversation, and continues until Claude responds with `end_turn`.

EXPLANATION

This is the canonical agentic loop, and automating it is exactly what the SDK harness is for: request -> inspect `stop_reason` -> execute the requested tools -> append `tool_result` blocks as a user message -> request again -> repeat until `end_turn`. The SDK's value is owning that loop (plus context management, permissions, and hooks) so you only write tool bodies.

Why distractors fail:

- **A** - Tool execution is local to your process (client-side tools). There is no separate execution service to poll. The only server-executed tools are Anthropic's server tools (web search, web fetch, code execution), a different mechanism entirely.
- **B** - One call cannot complete a tool-using task: the first response *requests* tools, it does not contain their results.
- **D** - Parallelism exists *within* a turn (several `tool_use` blocks executed concurrently), not as parallel API calls replacing the loop.

EXAM TIP

Exam Rule: - This is the canonical agentic loop, and automating it is exactly what the SDK harness is for: request -> inspect `stop_reason` -> execute the

7. Streaming a response containing both text and thinking blocks; thinking must render in a separate UI panel. How do streaming events distinguish thinking from text content?

- (A) Thinking content uses a different streaming endpoint from text content.
- (B) Thinking content arrives first, followed by text content.
- (C) All content arrives in the same stream with inline markdown tags to distinguish thinking.
- (D) Streaming events include a `content_block_start` event with the block type (thinking or text), allowing the client to route content to the appropriate UI panel.

Answer: (D) Streaming events include a `content_block_start` event with the block type (thinking or text), allowing the client to route content to the appropriate UI panel.

EXPLANATION

The streaming protocol is block-structured. Each content block opens with a `content_block_start` event carrying an index and a typed `content_block` (`{"type": "thinking"}` or `{"type": "text"}`), then emits `content_block_delta` events (`thinking_delta` / `text_delta`) against that index, then `content_block_stop`. Routing is therefore a switch on a declared type - no parsing, no heuristics.

Why distractors fail:

- **A** - There is one streaming endpoint: `/v1/messages` with `stream: true`.
- **B** - Ordering is a convention, not a guarantee you can build UI routing on; with interleaved thinking, thinking and text can alternate across a turn.
- **C** - Thinking is never delivered as inline markdown tags inside text. (Leaked `<thinking>` tags are a known symptom of *disabling* thinking on current models - a bug, not the protocol.)

EXAM TIP

Exam Rule: - The streaming protocol is block-structured. Each content block opens with a `content_block_start` event carrying an index and a typed `con`

8. Three subagents (researcher, analyzer, reporter). The researcher frequently gets stuck in recursive web searches, making dozens of tool calls without converging. What architectural pattern prevents this?
- (A) Add a PostToolUse hook that counts tool calls and terminates after 10.
 - (B) Increase the researcher's context window to give it more room for exploration.
 - (C) Remove the researcher subagent and have the main agent do research directly.
 - (D) Set an iteration limit for the researcher subagent and require it to report findings at the limit even if incomplete.

Answer: (D) Set an iteration limit for the researcher subagent and require it to report findings at the limit even if incomplete.

EXPLANATION

Two things are needed and D supplies both: a **bound** on the subagent's exploration, and a **defined output contract at that bound** ("report what you have"). A subagent that returns partial findings keeps the orchestrator's pipeline alive and lets the coordinator decide whether to re-dispatch with a narrower brief. This is the standard multi-agent pattern: give each subagent an explicit effort budget and require a structured hand-back.

Why distractors fail:

- **A** - Half the answer. A hard terminate at 10 with no reporting requirement destroys the work done so far and hands the coordinator nothing, so the pipeline stalls instead of degrading gracefully.
- **B** - More context lets it spin *longer*. Context size is not the constraint; convergence criteria are.
- **C** - Collapsing the subagent moves the same unbounded search into the main agent and floods the orchestrator's context with raw search results - the exact problem subagents exist to prevent.

EXAM TIP

Exam Rule: - Two things are needed and D supplies both: a bound on the subagent's exploration, and a defined output contract at that bound ("report what

9. A loop processes `tool_use` blocks sequentially, one at a time, and is slow when Claude requests multiple tools in a single response. What optimization should they implement?
- (A) Execute independent tool calls in parallel.
 - (B) Queue tool calls and batch them for execution on a fixed schedule.
 - (C) Increase the API timeout to accommodate the sequential execution time.
 - (D) Limit Claude to one tool call per response using `tool_choice` to prevent multiple simultaneous requests.

Answer: (A) Execute independent tool calls in parallel.

EXPLANATION

Parallel tool use is enabled by default: one assistant message may carry several `tool_use` blocks, and the model emits them together *precisely because* it judged them independent. Execute them concurrently and return **all** the `tool_result` blocks in a **single** user message. (Splitting results across multiple user messages is itself an anti-pattern - it silently trains the model to stop batching.)

Why distractors fail:

- **B** - Fixed-schedule batching adds latency and machinery to solve a problem concurrency solves directly; the calls are already batched by the model.
- **C** - Raising the timeout hides the symptom and makes each turn slower still.
- **D** - Forcing serial tool use discards the model's own parallelism signal and multiplies round trips - the opposite of the fix.

EXAM TIP

Exam Rule: - Parallel tool use is enabled by default: one assistant message may carry several `tool_use` blocks, and the model emits them together *precisely because*

10. Your agentic loop continues executing even after Claude has signalled completion. What is the correct control-flow condition to terminate the loop?
- (A) Terminate when `stop_reason` is 'end_turn'; continue when `stop_reason` is 'tool_use'.
 - (B) Set a maximum iteration count of 10 and terminate when it is reached.
 - (C) Terminate when the assistant message contains no `tool_use` content blocks.
 - (D) Terminate when the response text contains phrases such as 'I have completed' or 'Done'.

Answer: (A) Terminate when `stop_reason` is 'end_turn'; continue when `stop_reason` is 'tool_use'.

EXPLANATION

It states the contract exactly and in both directions: `end_turn` -> stop, `tool_use` -> keep going. Everything else in the loop (caps, timeouts, budget checks) hangs off this as a secondary safety layer.

Why distractors fail:

- **B** - A cap is a backstop, not a completion condition; alone it both truncates long tasks and over-runs short ones.
- **C** - A derived proxy. It usually coincides with `end_turn`, but silently mishandles `max_tokens`, `pause_turn`, and `refusal` turns, which also contain no `tool_use` blocks yet are not successful completions.
- **D** - Text matching: the canonical anti-pattern (see UD1-049).

EXAM TIP

Exam Rule: - It states the contract exactly and in both directions: `end_turn` -> stop, `tool_use` -> keep going. Everything else in the loop (caps, timeou

11. *(1.1 | Agentic Loop, stop_reason, Loop Termination | Foundation)* The loop terminates by checking whether the assistant's latest message contains "Task complete". It occasionally fires when the phrase appears inside an explanation. Best fix?

(A) Replace the text-based check with checking `stop_reason == "end_turn"` to determine when to stop the loop.
(B) Add a stricter, less-common phrase (e.g. `===TASK COMPLETE===`) that's less likely to appear accidentally.
(C) Set a maximum iteration cap so the loop terminates safely regardless of text content.
(D) Ask the model to always confirm completion with a structured JSON object at the end of its response.

Answer: (A) Replace the text-based check with checking `stop_reason == "end_turn"` to determine when to stop the loop.

EXPLANATION

The defect is one of **category**, not **precision**: any signal generated inside the model's free text is probabilistic. `stop_reason` is emitted by the API as protocol metadata and cannot be produced accidentally by prose. Changing the class of signal eliminates the failure mode rather than shrinking it.

Why distractors fail:

- **B** - Reduces collision probability but keeps a text-based check; the model can still emit the sentinel while quoting instructions or reasoning about the protocol itself.
- **C** - Addresses runaway loops, not premature termination. The stem's bug is stopping **too early**, which a cap cannot prevent.
- **D** - Still model-generated content, plus a parsing step; it can be omitted or malformed. More machinery, same class of unreliability.

EXAM TIP

Exam Rule: - The defect is one of **category**, not **precision**: any signal generated inside the model's free text is probabilistic. `stop_reason` is emitted

12. *(1.1 | Agentic Loop, stop_reason, Tool Use | Foundation)* On a given turn the response's `stop_reason` is `"tool_use"`. What should the loop do?

(A) Truncate the response since the model ran out of output tokens.
(B) Execute the requested tool call(s), append the results to conversation history as tool results, and send the updated conversation back to the model for another turn.
(C) Treat this as the end of the task and return the response to the user.
(D) Treat this as an error and retry the turn.

Answer: (B) Execute the requested tool call(s), append the results to conversation history as tool results, and send the updated conversation back to the model for another turn.

EXPLANATION

`"tool_use"` means the model paused specifically to obtain information it cannot produce itself. The handling is fixed: run each `tool_use` block, build a user message containing one `tool_result` per call (matched by `tool_use_id`), append it, re-request.

Why distractors fail:

- **A** - Running out of output tokens is `stop_reason: "max_tokens"` - a different value with different handling (raise `max_tokens` and retry, or continue).
- **C** - Returning a `tool_use` turn to the user surfaces raw protocol JSON and abandons the task mid-flight.
- **D** - Nothing failed. `"tool_use"` is the normal, expected mid-task state.

EXAM TIP

Exam Rule: - `"tool_use"` means the model paused specifically to obtain information it cannot produce itself. The handling is fixed: run each `tool_use`

13. *(1.1 | Loop Control, Turn Budget, Spinning Detection | Exam)* 15 turns into a 20-turn budget on a complex refactor; each turn calls a different tool and history keeps growing. How do you decide whether to continue or escalate?

(A) Let it run until the full 20-turn budget is exhausted regardless of content, since turn budgets exist precisely to allow this.
(B) Escalate only if the agent explicitly says it's stuck.
(C) Check whether each turn's tool calls and results represent new information/progress toward the goal, not just whether the turn count is high.
(D) Escalate immediately once past turn 10, since that's over half the budget.

Answer: (C) Check whether each turn's tool calls and results represent new information/progress toward the goal, not just whether the turn count is high.

EXPLANATION

Turn count measures **spend**, not **health**. The diagnostic question is whether the loop is progressing: are new files or records being touched, are results changing the plan, is remaining work shrinking? A high turn count on a genuinely complex refactor is fine; a low turn count that repeats the same read is not. Progress-based evaluation is what separates legitimate multi-step work from spinning.

Why distractors fail:

- **A** - Burns the whole budget on a loop that may have stalled at turn 8, with no early warning.
- **B** - Self-reported stuckness is the least reliable detector; a spinning agent typically believes it is progressing.
- **D** - A fixed fraction of the budget is just another count-based heuristic: it escalates healthy long tasks and misses early stalls.

EXAM TIP

Exam Rule: - Turn count measures **spend**, not **health**. The diagnostic question is whether the loop is progressing: are new files or records being touche

14. **(1.1 | Conversation History, Context Window, Pruning | Exam)** A long-running session's history is approaching the model's context window limit after many tool calls. Most reliable fix?

- (A) Truncate the oldest turns from the conversation history and never look at them again.
- (B) Switch to a model with a larger context window as the permanent fix.
- (C) Stop making tool calls once history gets large enough.
- (D) Periodically summarize/compact older turns into a condensed representation, preserving only what's still needed for remaining work, rather than keeping every raw tool result.

Answer: (D) Periodically summarize/compact older turns into a condensed representation, preserving only what's still needed for remaining work, rather than keeping every raw tool result.

EXPLANATION

Compaction is **lossy on purpose but selective**: it discards bulky raw tool payloads while carrying forward decisions, constraints, file paths, and open work items. That bounds growth without severing the causal chain the agent depends on. It is what the API's server-side compaction and Claude Code's `/compact` both implement, and it is the recommended pattern for long-horizon agents.

Why distractors fail:

- **A** - Blind truncation deletes the earliest turns, which usually hold the **task definition and constraints** - the highest-value tokens in the window.
- **B** - Buys headroom without changing the growth rate; any unbounded loop eventually fills any window. It is also a cost and latency regression.
- **C** - Disabling the agent's only means of acting is a capability loss, not a context strategy.

EXAM TIP

Exam Rule: - Compaction is **lossy on purpose but selective**: it discards bulky raw tool payloads while carrying forward decisions, constraints, file path

15. **(1.1 | Tool Use, Client Tools, Server Tools | Foundation)** Your agent needs to call an internal, authenticated company API that only your backend can reach. Which tool implementation approach fits?

- (A) Define it as a client tool: the model requests the call, but your application code executes it and returns the result.
- (B) Define it as a server tool the model executes directly without your application in the loop.
- (C) Give the model direct network access so it can call the API itself.
- (D) Have the model generate the API call as text for a human to run manually.

Answer: (A) Define it as a client tool: the model requests the call, but your application code executes it and returns the result.

EXPLANATION

Client tools (user-defined tools) are exactly this shape: you publish a name, description, and `input_schema`; the model emits a `tool_use` request; **your** process executes it inside your network with your credentials; you hand back a `tool_result`. Secrets never enter the conversation, and the endpoint is reachable because the call originates from your backend.

Why distractors fail:

- **B** - Server tools are Anthropic-hosted (web search, web fetch, code execution). They run on Anthropic infrastructure, which by definition cannot reach a private internal endpoint, and you cannot register an arbitrary internal API as one.
- **C** - The model has no network stack; there is nothing to grant. Even conceptually this would place credentials in the model's hands.
- **D** - Removes automation entirely. A human step is an approval gate, not a tool implementation.

EXAM TIP

Exam Rule: - Client tools (user-defined tools) are exactly this shape: you publish a name, description, and `input_schema`; the model emits a `tool_use`

16. *(1.1 | Loop Control, Escalation, Tool Errors | Exam)* A tool call fails 3 times in a row with the same error, inside a loop with a 10-retry budget still remaining. What should happen?

- (A) Immediately terminate the entire session.
- (B) Classify the error type first.
- (C) Keep retrying until the retry budget is fully exhausted, since budget exists for exactly this.
- (D) Silently skip the failed tool call and continue the loop with the next step.

Answer: (B) Classify the error type first.

EXPLANATION

Retry policy must be a function of error **class**. A transient error (429, 5xx, timeout, connection reset) warrants continued retry with exponential backoff. A deterministic error (400 bad input, 401/403 auth, 404 missing resource, schema violation) will fail identically on every remaining attempt - retrying wastes the budget and delays escalation. Three identical failures is precisely the signal to stop and classify.

Why distractors fail:

- **A** - Over-reacts: a transient outage resolves on its own, and killing the session discards all completed work.
- **C** - Treats the budget as a quota to spend rather than a bound not to exceed; guarantees maximum latency and cost on permanently-failing calls.
- **D** - Silent skipping corrupts the result: the model keeps reasoning as though it received data it never got, producing confidently wrong output. Failed calls must be returned as a `tool_result` with `is_error: true` so the model **knows**.

EXAM TIP

Exam Rule: - Retry policy must be a function of error **class**. A transient error (429, 5xx, timeout, connection reset) warrants continued retry with expo

17. *(1.1 | Agentic Loop, Mental Models | Foundation)* How do "perception -> reasoning -> action -> observation" and "gather context -> take action -> verify work" relate?

- (A) The 4-phase framing applies only to single-agent systems and the 3-phase framing only to multi-agent systems.
- (B) The 3-phase framing replaced the 4-phase framing as an updated standard.
- (C) They describe the same underlying loop at different levels of abstraction.
- (D) They are mutually exclusive architectures and a system must choose one.

Answer: (C) They describe the same underlying loop at different levels of abstraction.

EXPLANATION

Both describe one cycle. **Perception** and **reasoning** collapse into **gather context**; **action** maps to **take action**; **observation** maps to **verify work**. The 3-phase framing is the practitioner-facing compression of the 4-phase cognitive-architecture framing - useful mainly because "verify" names the step teams most often omit.

Why distractors fail:

- **A** - Neither framing is scoped by agent count; both apply to a single agent and to each member of a multi-agent system.
- **B** - Not a versioned standard; both remain in current use, often in the same document.
- **D** - There is nothing to choose between: two vocabularies, one control flow.

EXAM TIP

Exam Rule: - Both describe one cycle. **Perception** and **reasoning** collapse into **gather context**; **action** maps to **take action**; **observation** maps to

18. *(1.1 | Loop Control, Spinning vs Progress | Expert)* An agent repeatedly calls the same search tool with slightly reworded queries across 6 turns without incorporating any new result into its plan. Best characterization and response?

- (A) This is legitimate exploration and should be allowed to continue since search tools are inherently iterative.
- (B) This is a tool error and the tool itself should be retried with backoff.
- (C) This is expected behavior for the perception-reasoning-action-observation loop and requires no intervention.
- (D) This is spinning, not legitimate multi-step progress.

Answer: (D) This is spinning, not legitimate multi-step progress.

EXPLANATION

The defining marker of spinning is **action without integration**: results return and the plan does not change. Reworded queries are the classic signature - the agent varies the surface of its action while its internal state stays fixed. The correct response is intervention: bound the search, inject a directive observation, or escalate.

Why distractors fail:

- **A** - Legitimate iterative search **narrows**: each result should prune hypotheses or retarget the next query. That is absent here.
- **B** - Nothing errored. The tool returned successfully every time; the failure is in how the agent used the results.
- **C** - The observation phase is precisely what is **not** happening. Calling it expected removes the loop's only self-correction step.

EXAM TIP

Exam Rule: - The defining marker of spinning is **action without integration**: results return and the plan does not change. Reworded queries are the class

19. *(Agentic Loop, stop_reason | Foundation)* A support agent's loop receives stop_reason: "end_turn" after calling get_customer and lookup_order in prior turns. What should the loop do?

- (A) Wait for an explicit "done" keyword in the response text before stopping.
- (B) Treat this as the model's final response and return it to the user.
- (C) Execute the next tool call automatically since the loop must always continue after a tool result.
- (D) Treat "end_turn" as an error state and retry the last tool call.

Answer: (B) Treat this as the model's final response and return it to the user.

EXPLANATION

"end_turn" means the model reached a natural stopping point with nothing further requested. The prior tool calls succeeded and their results are already in context; this turn is the synthesised answer. Return it.

Why distractors fail:

- A - Layers a text check over an authoritative signal, introducing the false-negative hang described in UD1-012.
- C - There is no rule that the loop continues after a tool result; the loop continues while stop_reason == "tool_use", which is not the case here.
- D - "end_turn" is the success state. Retrying re-executes a completed action, which on a mutating tool (refund, ticket update) produces real duplicate side effects.

EXAM TIP

Exam Rule: - "end_turn" means the model reached a natural stopping point with nothing further requested. The prior tool calls succeeded and their result

20. *(Agentic Loop, stop_reason, Tool Use | Foundation)* Mid-conversation, the response has stop_reason: "tool_use" requesting process_refund. What must the loop do before sending the next request to Claude?

- (A) Skip execution and ask the model to confirm it really wants to call the tool.
- (B) Wait for the customer to approve before appending anything to conversation history.
- (C) Execute process_refund, append its result to the conversation as a tool result, and send the updated conversation back to the model.
- (D) Return the tool_use response directly to the customer as the final answer.

Answer: (C) Execute process_refund, append its result to the conversation as a tool result, and send the updated conversation back to the model.

EXPLANATION

This is the mandatory protocol step. The next request is invalid unless the assistant message containing the tool_use block is followed by a user message containing a matching tool_result (keyed on tool_use_id). The question asks what the *loop* must do, and the loop's contract is execute -> append -> re-request.

Why distractors fail:

- A - Asking the model to re-confirm its own request wastes a turn and is not a control; the model simply reasserts.
- B - Human approval on a refund is a legitimate *policy* layer, but it is implemented as a gate around execution (a PreToolUse-style hook), not as "append nothing." The protocol still requires a tool_result; a denial is returned as an error result, not as silence.
- D - Surfaces internal protocol JSON to a customer and abandons the refund.

EXAM TIP

Exam Rule: - This is the mandatory protocol step. The next request is invalid unless the assistant message containing the tool_use block is followed by

21. *(Loop Termination, Anti-patterns | Exam)* The loop checks whether the response text contains "Resolved!" instead of checking stop_reason; it occasionally fires when the phrase appears inside an explanation. Best fix?

- (A) Add a stricter marker phrase less likely to appear by accident.
- (B) Cap the loop at 8 iterations so it can't run away regardless of the text signal.
- (C) Have the model repeat "Resolved!" three times in a row before stopping.
- (D) Replace the text check with reading stop_reason directly.

Answer: (D) Replace the text check with reading stop_reason directly.

EXPLANATION

Identical principle to QD1-02: replace a content-derived signal with the protocol-level signal. stop_reason cannot be emitted accidentally in prose.

Why distractors fail:

- A - Lowers collision probability, keeps the flawed mechanism.
- B - Bounds runaway; does nothing about premature exit, which is the actual bug.
- C - A more elaborate text convention: triples the tokens spent on the sentinel and remains model-generated.

EXAM TIP

Exam Rule: - Identical principle to QD1-02: replace a content-derived signal with the protocol-level signal. stop_reason cannot be emitted accidentally

22. *(Loop Control, Iteration Caps, Anti-patterns | Exam)* The loop terminates after exactly 6 tool calls regardless of whether the issue is resolved, occasionally cutting off mid-resolution on complex tickets. Most effective fix?

- (A) Replace the fixed iteration cap as the primary termination logic with `stop_reason`-based termination, keeping a higher cap only as a safety backstop.
- (B) Raise the fixed cap from 6 to 12 to reduce how often this happens.
- (C) Let the agent decide for itself when 6 calls is enough by reasoning about it in the prompt.
- (D) Remove the cap entirely so the loop runs until `stop_reason` appears, with no safety bound at all.

Answer: (A) Replace the fixed iteration cap as the primary termination logic with `stop_reason`-based termination, keeping a higher cap only as a safety backstop.

EXPLANATION

It corrects the *role* of each mechanism. `stop_reason` becomes the primary termination condition, so the loop ends when the work ends; the cap is demoted to a backstop set high enough not to bind on normal tasks, retaining runaway protection without truncating legitimate ones. Both properties are required, and only A delivers both.

Why distractors fail:

- **B** - The same broken design with a bigger number: still truncates 13-call tickets, still spins to 12 on tasks that finished at 3.
- **C** - Self-assessed budgets are unreliable and, worse, replace a hard bound with a soft one, removing the safety property.
- **D** - Fixes truncation but deletes the backstop, reintroducing unbounded cost on a stuck loop. Correct primary signal, unacceptable safety posture.

EXAM TIP

Exam Rule: - It corrects the *role* of each mechanism. `stop_reason` becomes the primary termination condition, so the loop ends when the work ends; the

23. *(Loop Control, Safety Backstops | Expert)* A teammate argues: "Since the loop already checks `stop_reason` correctly, the 6-iteration cap serves no purpose and should be removed." Is this reasoning sound?

- (A) Yes
- (B) No
- (C) Yes
- (D) No

Answer: (B) No

EXPLANATION

Correct `stop_reason` handling and an iteration cap defend against **different** failure modes. `stop_reason` handles the case where the model *finishes*; the cap handles the case where the model *never* finishes - a tool that always errors and keeps being retried, a search that never converges, an oscillation between two actions. In every one of those, `stop_reason` is legitimately `"tool_use"` forever. Defence in depth: keep both, with the cap set high enough that it never binds on healthy runs.

Why distractors fail:

It assumes `stop_reason` bounds the loop. It does not - it only reports the state of each individual turn, and nothing in the protocol guarantees the model will ever emit `end_turn`.

EXAM TIP

Exam Rule: - Correct `stop_reason` handling and an iteration cap defend against different failure modes. `stop_reason` handles the case where the model *

24. *(Loop Control, Anti-patterns | Expert - multi-response, select 2)* Which TWO are anti-patterns specifically called out for agentic loop termination in a customer support agent?

- (A) Parsing natural language signals in the response text to decide when to stop.
- (B) Checking `stop_reason == "end_turn"` as the primary termination condition.
- (C) Keeping a generous iteration cap active as a secondary safety bound alongside `stop_reason`.
- (D) Setting an arbitrary iteration cap as the primary stopping mechanism.

Answer: (A) Parsing natural language signals in the response text to decide when to stop.

EXPLANATION

- **A** - Text parsing is the canonical anti-pattern: the signal is probabilistic, collides with ordinary prose, and fails in both directions (early exit and never exiting).

- **D** - A cap as the *primary* mechanism truncates unfinished work and lets finished work spin to the ceiling. The damaging word is "primary", not "cap."

Why the others are correct practice.

- **B** - This is the recommended design, not an anti-pattern.

- **C** - A generous cap *alongside* `stop_reason` is exactly the prescribed defence-in-depth posture (see QD1S-04 and QD1S-05). Note the deliberate contrast between C and D: the same mechanism is good as a backstop and bad as the primary signal.

EXAM TIP

Exam Rule: - - A - Text parsing is the canonical anti-pattern: the signal is probabilistic, collides with ordinary prose, and fails in both directions (e

25. A Customer Support Agent action loop, during a billing-system outage, retried the same failing balance lookup hundreds of times until it exhausted the rate limit. Which design BEST addresses the root cause while preserving autonomy?

- (A) Lower the model temperature to zero so the loop behaves deterministically.
- (B) Replace the agent with a fixed workflow so retry behavior is hard-coded and the model can no longer choose to repeat the lookup.
- (C) Add explicit termination conditions plus exponential backoff that classifies the outage as a transient environment error and escalates after bounded retries.
- (D) Switch the balance lookup to a parallel fan-out of three concurrent calls so a healthy replica answers before the loop repeats.

Answer: (C) Add explicit termination conditions plus exponential backoff that classifies the outage as a transient environment error and escalates after bounded retries.

EXPLANATION

The root cause is an **unbounded retry against a failing dependency**. C addresses all three needed properties at once: a **bound** (explicit termination conditions), **pressure relief** (exponential backoff, which also stops the rate-limit exhaustion), and **classification plus escalation** (recognise "transient environment error", hand off after the bound). Crucially it keeps the agent - the constraint asked for a fix that "preserves autonomy."

Why distractors fail:

- **A** - Temperature affects token sampling, not control flow. A deterministic model retrying a dead endpoint retries it deterministically forever.
- **B** - Achieves reliability by deleting the capability. It also fails the stated constraint (preserve autonomy), and its premise - that removing autonomy is "the only sure way" - is false, as C demonstrates.
- **D** - Triples load against an already-failing system during an outage, accelerating rate-limit exhaustion. Fan-out addresses latency, not failure handling.

EXAM TIP

Exam Rule: - The root cause is an unbounded retry against a failing dependency. C addresses all three needed properties at once: a **bound** (explicit term

26. Which BEST captures the difference between tool use and an agentic loop in a Data Extraction context?

- (A) Tool use is strictly read-only, whereas an agentic loop introduces the ability to write changes back to external systems.
- (B) Tool use refers to a single tool invocation; an agentic loop is many tool invocations chained by the model based on observations.
- (C) Tool use requires JSON schemas; an agentic loop accepts free-form output.
- (D) Tool use is synchronous; an agentic loop is asynchronous.

Answer: (B) Tool use refers to a single tool invocation; an agentic loop is many tool invocations chained by the model based on observations.

EXPLANATION

Tool use is the **capability** - one request/response exchange in which the model asks for a call and receives a result. The agentic loop is the **control structure** built on that capability: the model observes each result and decides the next call, iterating until the goal is met. The distinguishing feature is **feedback-driven iteration**, not the nature of the tools.

Why distractors fail:

- **A** - Nothing about tool use is read-only; a single tool call can perform a write (process_refund).
- **C** - Both use the same tool definitions with `input_schema`. Looping does not relax schema requirements.
- **D** - Both are ordinary synchronous request/response over the Messages API. Async is an implementation choice on either.

EXAM TIP

Exam Rule: - Tool use is the **capability** - one request/response exchange in which the model asks for a call and receives a result. The agentic loop is t

27. Criteria for choosing between a workflow and an agentic system with workflow-like guardrails?

- (A) Workflow when sequence is predetermined and enforcement is paramount; agentic with workflow-like guardrails when adaptation within bounds is needed.
- (B) Workflow if the team likes structure, agentic if the team likes flexibility.
- (C) Workflow for new features, agentic for legacy ones.
- (D) Always use a workflow, on the grounds that workflows are simpler.

Answer: (A) Workflow when sequence is predetermined and enforcement is paramount; agentic with workflow-like guardrails when adaptation within bounds is needed.

EXPLANATION

The decision is driven by two properties of the *task*: (1) is the sequence knowable in advance, and (2) does the requirement demand enforcement or adaptation? Predetermined + enforcement-critical -> workflow, because determinism is the feature. Variable path but bounded action space -> agent with guardrails, keeping runtime adaptation while constraining what can happen.

Why distractors fail:

- **B** - Team preference is not a task property; this reduces an architectural decision to taste.
- **C** - Feature age is unrelated to whether the sequence is predetermined.
- **D** - Over-corrects a real principle ("start simple") into an absolute. Some tasks genuinely cannot be enumerated in advance, and forcing a workflow there produces an unmaintainable branch tree.

EXAM TIP

Exam Rule: - The decision is driven by two properties of the *task*: (1) is the sequence knowable in advance, and (2) does the requirement demand enforce

28. A team built an agent expecting runtime adaptation. Six months of telemetry shows the same five tool-call sequences 92% of the time. Most consistent interpretation?

- (A) Add more tools so it has even more flexibility.
- (B) It performs well on all the data, so no architectural change is needed.
- (C) The original choice was wrong for the common path; a workflow with the five sequences would better fit the 92%.
- (D) Replace the agent with a conversational system.

Answer: (C) The original choice was wrong for the common path; a workflow with the five sequences would better fit the 92%.

EXPLANATION

Telemetry is the empirical test of the workflow-vs-agent decision. A 92% concentration into five fixed sequences means the common path *is* predetermined - so it should be a workflow: cheaper (fewer model round trips), faster, deterministic, and testable. The remaining 8% is where the agent's adaptation actually earns its cost, typically as a fallback the router escalates to.

Why distractors fail:

- **A** - Adds cost and selection ambiguity to solve a problem of *too much* flexibility, not too little.
- **B** - Confuses output correctness with architectural fit. The system works; it is paying agent-tier cost, latency, and non-determinism for workflow-tier behaviour.
- **D** - A conversational system is a different class again (response-only, no actions). It cannot execute the tool sequences at all.

EXAM TIP

Exam Rule: - Telemetry is the empirical test of the workflow-vs-agent decision. A 92% concentration into five fixed sequences means the common path *is*

29. A research subagent loops over web search, scrape, and synthesize; it occasionally returns results with no actual sources cited. What property of agentic systems does this MOST directly highlight?

- (A) Agentic systems require structured tool output schemas to function correctly.
- (B) Agentic loops cannot reliably use web search tools because of latency.
- (C) Agentic autonomy can satisfy a stop condition without satisfying the actual goal.
- (D) Agentic systems require Claude's largest model to reliably follow every complex instruction.

Answer: (C) Agentic autonomy can satisfy a stop condition without satisfying the actual goal.

EXPLANATION

The loop terminated correctly by its own contract - the model emitted `end_turn` - while the *deliverable* failed a requirement (citations) that the stop condition never encoded. This is the central reliability gap in autonomous systems: **termination is not verification**. The remedy is an explicit verification step or output contract (schema requiring a non-empty `sources` array, a validator that rejects uncited claims and re-prompts).

Why distractors fail:

- **A** - A schema is one *remedy*, not the property being illustrated; and the question asks what the failure highlights.
- **B** - Latency is irrelevant; the searches ran and returned.
- **D** - Model size is a mitigation, not the underlying property. Any model can satisfy an under-specified stop condition.

EXAM TIP

Exam Rule: - The loop terminated correctly by its own contract - the model emitted `end_turn` - while the `*deliverable*` failed a requirement (citations)

30. A team wants their SDK app to fall back to a different model if Claude is unavailable. Which characterization is MOST accurate?

- (A) Fallback is the application's job.
- (B) The SDK automatically falls back to other vendors on failure, giving cross-provider resilience for free.
- (C) The SDK natively supports OpenAI as a fallback target configured directly in the SDK.
- (D) Cross-model fallback is not supported in any SDK at all, so the application must simply fail the request.

Answer: (A) Fallback is the application's job.

EXPLANATION

The Anthropic SDKs handle `*transport-level*` resilience - automatic retries on 408/409/429/5xx and connection errors, configurable via `max_retries` - but choosing and routing to a **different provider or model family** is application-layer policy. The app owns the health check, the circuit breaker, the request translation, and the decision of what degraded behaviour is acceptable. (Anthropic does offer a server-side `fallbacks` parameter, but that routes `*within*` Anthropic models on a `refusal` stop reason - it is not cross-vendor availability failover.)

Why distractors fail:

- **B** - No Anthropic SDK routes to another vendor. Nothing about cross-provider resilience is free.
- **C** - There is no OpenAI target configuration in the Anthropic SDKs; the SDKs speak the Messages API only.
- **D** - Too strong in the other direction: fallback is entirely implementable, just at the application layer. "Must simply fail" is false.

EXAM TIP

Exam Rule: - The Anthropic SDKs handle `*transport-level*` resilience - automatic retries on 408/409/429/5xx and connection errors, configurable via ``max_r``

31. A workflow (classify intent -> retrieve docs -> generate reply) is upgraded to handle multi-turn back-and-forth where Claude can ask clarifying questions and call account tools as needed. Which class change BEST describes the upgrade?

- (A) Workflow to agentic, because the new behavior requires runtime decisions about tool calls and clarifications.
- (B) Workflow to conversational, because clarifying questions are just user-turn interactions.
- (C) Workflow to multi-agent, because clarification is a separate concern from tool use.
- (D) No change, because the existing workflow can handle the new behavior with more branches.

Answer: (A) Workflow to agentic, because the new behavior requires runtime decisions about tool calls and clarifications.

EXPLANATION

The defining change is that the `*sequence is no longer knowable in advance*`: whether to ask a question, which account tool to call, and how many times all depend on what the user says and what the tools return. Runtime, model-driven control flow over tools is precisely the agentic class.

Why distractors fail:

- **B** - Conversational systems are response-only. This system takes actions against account systems, which conversational does not cover.
- **C** - Multi-agent means multiple independently-orchestrated agents with delegation. One agent asking clarifying questions is not delegation, and adding agents here is unjustified complexity.
- **D** - You could enumerate branches only if the paths were finite and known. Clarification loops are open-ended; the branch tree does not close.

EXAM TIP

Exam Rule: - The defining change is that the `*sequence is no longer knowable in advance*`: whether to ask a question, which account tool to call, and how

32. Task: rename a variable consistently across a codebase. Which architectural class fits BEST?

- (A) An agent that decides which occurrences to rename based on context.
- (B) A deterministic workflow with find-and-replace as a fixed step.
- (C) A conversational chatbot that asks the developer about each occurrence.
- (D) A multi-agent system with one agent per file.

Answer: (B) A deterministic workflow with find-and-replace as a fixed step.

EXPLANATION

The operation is fully specifiable, deterministic, verifiable, and has an exact tool already (scoped find/replace, or a language-server rename). Every criterion for "don't build an agent" is met: no ambiguity to resolve at runtime, no value added by adaptation, and an LLM in the path only introduces the risk of missed or spurious edits. Start at the simplest tier that solves the problem.

Why distractors fail:

- **A** - Introduces non-determinism into an operation whose correctness requirement is exhaustive consistency. Judgement is the failure mode, not the feature.
- **C** - Per-occurrence prompting turns a one-second operation into a manual review session.

- **D** - Maximum coordination overhead for a task with no coordination requirement; also risks inconsistent decisions across files.

EXAM TIP

Exam Rule: - The operation is fully specifiable, deterministic, verifiable, and has an exact tool already (scoped find/replace, or a language-server rena

33. A workflow summarizes news on five fixed topics every morning. Stakeholders want to occasionally add topics outside the predetermined set. Most consistent architectural change?

- (A) Replace the workflow with a conversational chatbot that takes topic requests.
- (B) Replace the workflow entirely with an agent so it can discover brand-new topics on its own at runtime.
- (C) Run the workflow five times in parallel for the original topics and add a new workflow for new topics.
- (D) Add a parameter to the workflow so the topic list can be configured per run, keeping the workflow class.

Answer: (D) Add a parameter to the workflow so the topic list can be configured per run, keeping the workflow class.

EXPLANATION

The *steps* have not changed at all - retrieve, then summarize, per topic. Only the *input data* varies. Varying data is what parameters are for; it is not evidence of a need for runtime reasoning. Parameterising preserves determinism, cost, and testability while fully satisfying the requirement.

Why distractors fail:

- **A** - A chatbot changes the interface, not the pipeline, and abandons the scheduled automated run.
- **B** - Classic over-engineering: the model is not being asked to *discover* topics, only to process ones humans supply. Adds cost, latency, and non-determinism for nothing.
- **C** - Duplicates a pipeline that already handles the case, creating two code paths to maintain and drift apart.

EXAM TIP

Exam Rule: - The *steps* have not changed at all - retrieve, then summarize, per topic. Only the *input data* varies. Varying data is what parameters are

34. Under which condition is a conversational system the right architectural fit?

- (A) When the team wants to standardize on Claude's largest available model.
- (B) When the feature is response-only.
- (C) When the team wants to log every user message.
- (D) When the team wants the feature to take action quickly.

Answer: (B) When the feature is response-only.

EXPLANATION

The conversational class is defined by the absence of actions: the system produces *responses* - answers, explanations, drafts - and never mutates external state or calls tools to act. Once a feature must take an action, it has left the conversational class for tool use, a workflow, or an agent.

Why distractors fail:

- **A** - Model choice is orthogonal to architectural class; any class can run any model.
- **C** - Logging is an observability concern implemented identically in every class.
- **D** - Taking action is the exact property that disqualifies the conversational class.

EXAM TIP

Exam Rule: - The conversational class is defined by the absence of actions: the system produces *responses* - answers, explanations, drafts - and never m

35. A CI job launches Claude Code to triage a failing test. Logs show it reading the failing test, then the test source, then the production module, then the recent git diff. Which characterization BEST describes what is happening?

- (A) Claude is running a hardcoded developer-authored script that fetches files in a fixed order.
- (B) Claude is fetching files via a router workflow that classifies the failure type first.
- (C) Claude is running parallel Reads as a fan-out and the logs are merely interleaved.
- (D) Claude is iterating its agentic loop.

Answer: (D) Claude is iterating its agentic loop.

EXPLANATION

Each read is *chosen based on what the previous read revealed*: the failing assertion points at a source file, that file points at a module, the module's suspect behaviour points at recent changes. That observation-driven selection of the next action is the agentic loop by definition - even though the resulting sequence looks orderly in hindsight.

Why distractors fail:

- **A** - The distractor's own reasoning is the trap: a sequence being *predictable in hindsight* does not make it *predetermined*. No script was written; the order emerged at runtime.
- **B** - No classification step appears in the logs, and Claude Code does not interpose a router workflow before file reads.
- **C** - Fan-out would show concurrent reads chosen up front, without dependence on prior results. Here each target is derivable only after the previous read.

EXAM TIP

Exam Rule: - Each read is *chosen based on what the previous read revealed*: the failing assertion points at a source file, that file points at a module,

36. MOST common reason teams over-engineer a Data Extraction feature as agentic when a workflow would suffice?

- (A) Treating tool use as the signal for agentic architecture even when the tool sequence is fixed.
- (B) Choosing the wrong Claude model for the task when first setting up the feature.
- (C) Not configuring any system prompt at all.
- (D) Using JSON mode instead of free-form output.

Answer: (A) Treating tool use as the signal for agentic architecture even when the tool sequence is fixed.

EXPLANATION

"It calls tools, therefore it's an agent" is the single most common category error. Tool use is a *capability* available to every tier; what makes a system agentic is the model *deciding the sequence at runtime*. Extraction pipelines usually have a fixed sequence (fetch -> parse -> extract -> validate -> store), so tool use there belongs in a workflow.

Why distractors fail:

- **B** - A model mis-pick is a cost/quality problem within whatever architecture you chose; it does not cause the wrong architecture.
- **C** - A missing system prompt degrades output quality at any tier.
- **D** - Output format is a serialization decision, orthogonal to control flow.

EXAM TIP

Exam Rule: - "It calls tools, therefore it's an agent" is the single most common category error. Tool use is a *capability* available to every tier; what

37. In a CI job, Claude Code detects mid-refactor that a precondition is unmet and stops without making changes. Which characterization of the execution model BEST fits?

- (A) The -p flag forces stop at the first uncertain step regardless of task state.
- (B) Claude Code crashed silently and the session should be restarted.
- (C) Only hooks can stop the loop; without a registered hook Claude must continue until it produces a final answer.
- (D) The agentic loop can terminate at any iteration when Claude judges progress impossible.

Answer: (D) The agentic loop can terminate at any iteration when Claude judges progress impossible.

EXPLANATION

Termination is a decision available on **every** iteration. When the model concludes the goal is unreachable, it ends its turn (end_turn) with an explanation instead of requesting more tools. Stopping cleanly without making changes is the *correct* and safe behaviour - far better than proceeding on a violated precondition.

Why distractors fail:

- **A** - -p selects non-interactive/print mode. It governs I/O, not uncertainty handling, and does not halt on uncertainty.
- **B** - A crash produces a non-zero exit and no explanatory output; here Claude reported the unmet precondition, which is a deliberate, graceful stop.
- **C** - Hooks are optional interception points. The loop's normal termination path is stop_reason, entirely independent of hooks; and nothing forces the model to produce an answer regardless of feasibility.

EXAM TIP

Exam Rule: - Termination is a decision available on every iteration. When the model concludes the goal is unreachable, it ends its turn (end_turn) with

38. A report-generation system must support both daily fixed reports and ad-hoc deep-dive analyses. Which pattern MOST cleanly satisfies both?

- (A) A single agent that handles both.
- (B) A workflow for the daily fixed reports and an agent for ad-hoc deep dives, with a router between them.
- (C) A single workflow that handles both.
- (D) Two separate products with no shared infrastructure.

Answer: (B) A workflow for the daily fixed reports and an agent for ad-hoc deep dives, with a router between them.

EXPLANATION

The two workloads have genuinely different properties: the daily report is predetermined, repeated, and cost-sensitive (workflow); the deep dive is open-ended and adaptive (agent). Matching each to its own tier and putting a router in front gives determinism and low cost where it is available, and adaptation only where it is paid for - while still sharing tools, prompts, and infrastructure.

Why distractors fail:

- **A** - Pays agent-tier cost, latency, and non-determinism for the daily path, which needs none of it.
- **C** - Cannot express open-ended deep dives; you would end up encoding an unbounded branch tree.
- **D** - Duplicates tools, credentials, prompts, and formatting logic with no compensating benefit.

EXAM TIP

Exam Rule: - The two workloads have genuinely different properties: the daily report is predetermined, repeated, and cost-sensitive (workflow); the deep

39. Claude Code reran a long-running Bash command after the user pressed Ctrl-C mid-execution. Which explanation BEST fits Claude Code's execution model?

- (A) Cancellation returns control to the agentic loop, which then plans a new action against the original goal.
- (B) Ctrl-C is forwarded to the parent agent only; the child shell keeps running, so the "reissue" is Claude reattaching to a live process.
- (C) Claude Code automatically retries any Bash command that exits with a non-zero status.
- (D) PreToolUse hooks reinject the command into the queue after any user interrupt event.

Answer: (A) Cancellation returns control to the agentic loop, which then plans a new action against the original goal.

EXPLANATION

Interrupting a tool call cancels *that action*, not the *goal*. Control returns to the loop with an observation ("command was interrupted"), and the model - still holding the original objective - plans the next action, which may reasonably be to run the command again. The correct way to stop the behaviour is to change the goal (tell Claude not to run it) rather than to cancel repeatedly.

Why distractors fail:

- **B** - Fabricates a process-tree behaviour; the interrupt terminates the child, and the second run is a genuinely new invocation, not a reattachment.
- **C** - There is no blanket auto-retry on non-zero exit. Re-running is a model decision, visible in the transcript, not a harness policy.
- **D** - PreToolUse hooks run *before* a tool call to allow, deny, or modify it. They have no queue-reinjection role and are not triggered by interrupts.

EXAM TIP

Exam Rule: - Interrupting a tool call cancels *that action*, not the *goal*. Control returns to the loop with an observation ("command was interrupted"),

40. Which architectural choice MOST reduces non-determinism in Claude Code's execution model inside a nightly job?

- (A) Set temperature to 0 in CLAUDE.md so the loop becomes a fully deterministic fixed sequence.
- (B) Disable all hooks so the loop has fewer side channels to vary across runs.
- (C) Narrow the tool surface via permissions, structure the prompt tightly, and use -p with explicit task scope.
- (D) Add many SKILL.md files so Claude has more triggers to match against the prompt.

Answer: (C) Narrow the tool surface via permissions, structure the prompt tightly, and use -p with explicit task scope.

EXPLANATION

Non-determinism in an agentic system comes from the *size of the action space* and the *looseness of the goal*. C shrinks both: permissions constrain which tools can ever be invoked, a tight prompt removes interpretive latitude, and -p with explicit scope gives a bounded non-interactive run with a defined completion criterion. This is the recommended posture for automated/CI use of Claude Code.

Why distractors fail:

- **A** - CLAUDE.md is context for the model, not a settings file; it cannot set sampling parameters. Even if temperature were 0, the loop would remain adaptive - determinism of token sampling is not determinism of control flow.
- **B** - Hooks are *deterministic* interception points; disabling them removes a mechanism for enforcing consistency rather than adding one.
- **D** - More skills means more trigger surface and more variation in which capability activates - the opposite of the goal.

EXAM TIP

Exam Rule: - Non-determinism in an agentic system comes from the *size of the action space* and the *looseness of the goal*. C shrinks both: permissions

41. In an evaluator-optimizer loop, later iterations make only tiny wording changes without improving quality. Which stopping criterion BEST prevents wasted iterations?

- (A) Continue until the maximum iteration count is reached regardless of improvement.
- (B) Stop once improvements fall below a defined convergence threshold or quality no longer meaningfully increases.
- (C) Lower temperature after every iteration so revisions naturally stop changing.
- (D) Always run exactly five optimization iterations to keep behavior predictable.

Answer: (B) Stop once improvements fall below a defined convergence threshold or quality no longer meaningfully increases.

EXPLANATION

The evaluator-optimizer pattern already produces a quality score every round, so the natural stopping rule is *marginal improvement*: halt when the delta falls under a threshold. That aligns spend with value - long on tasks that keep improving, short on tasks that converge quickly - and it directly targets the observed symptom (changes without improvement).

Why distractors fail:

- **A** - Guarantees the waste described in the stem.
- **C** - Reducing temperature makes revisions *look* stable without any evidence quality improved; it disguises non-convergence rather than detecting it.
- **D** - A fixed count is predictable but arbitrary: it wastes iterations on easy items and truncates hard ones.

EXAM TIP

Exam Rule: - The evaluator-optimizer pattern already produces a quality score every round, so the natural stopping rule is *marginal improvement*: halt w

42. An agent that edits files, runs builds, and commits changes sometimes runs 200+ iterations before stopping; it completes, but cost is unpredictable. Which architectural fix BEST addresses this?

- (A) Switch from an agent to a deterministic workflow whose fixed five-step plan bounds total cost.
- (B) Increase the model's context window so more history fits in a single call.
- (C) Add a maximum-iteration cap with an explicit escalation path when the cap is reached.
- (D) Remove the build tool so the agent cannot trigger long-running builds.

Answer: (C) Add a maximum-iteration cap with an explicit escalation path when the cap is reached.

EXPLANATION

The stated problem is *unpredictable cost*, not incorrect results - the agent completes successfully. A cap converts an unbounded worst case into a known ceiling, and the escalation path ensures hitting that ceiling produces a handled outcome (partial results + human review) instead of a silent failure. It preserves the agent's capability while making spend bounded.

Why distractors fail:

- **A** - Discards adaptation that the task evidently needs; edit-build-fix cycles cannot be reduced to five fixed steps because the number of fix rounds is data-dependent.
- **B** - Context size is unrelated to iteration count; a larger window makes each expensive iteration cost *more*.
- **D** - Removes the verification step. An agent that edits code without building is faster and much less reliable.

EXAM TIP

Exam Rule: - The stated problem is *unpredictable cost*, not incorrect results - the agent completes successfully. A cap converts an unbounded worst case

43. Which diagram element MOST clearly identifies a system as agentic rather than a workflow?

- (A) A user box at the top connected straight down to a response box at the bottom.
- (B) A box labeled 'Claude' connected by a directed arrow to a box labeled 'Tool'.
- (C) Multiple tool boxes connected in sequence.
- (D) An arrow from tool output back into the model's input as part of the next iteration.

Answer: (D) An arrow from tool output back into the model's input as part of the next iteration.

EXPLANATION

The **feedback edge** is the signature. An arrow returning tool output into the model's input means the model observes the result and chooses the next action from it - the loop. Without that edge you have a pipeline, however many tools it contains.

Why distractors fail:

- **A** - A straight user -> response path is the conversational class: no tools, no loop.
- **B** - A one-way Claude -> Tool arrow is plain tool use: a call with no observation edge.
- **C** - Tools in sequence is a workflow: the order is fixed in the diagram, therefore predetermined rather than chosen at runtime.

EXAM TIP

Exam Rule: - The feedback edge is the signature. An arrow returning tool output into the model's input means the model observes the result and chooses th

44. A research agent keeps spawning follow-up searches and the run is not converging. The most reliable way to prevent an endless loop is to:

- (A) Let it continue until it naturally stops.
- (B) Give the task an explicit budget and a coverage check, and stop once the questions are answered or the budget is spent.
- (C) Cut the run off at a random time.
- (D) Add more sub-agents so it finishes sooner.

Answer: (B) Give the task an explicit budget and a coverage check, and stop once the questions are answered or the budget is spent.

EXPLANATION

Two complementary stopping conditions: a **coverage check** (goal-relative - have the research questions been answered?) which stops the run as soon as it is genuinely done, and a **budget** (resource-relative) which bounds the worst case when coverage is never reached. Together they terminate on success *and* on failure, which is what "reliable" requires.

Why distractors fail:

- **A** - Restates the failing behaviour; the stem says it does not naturally stop.
- **C** - A random cutoff is a bound with no relationship to progress: it truncates converging runs and lets others run long.
- **D** - Parallelising an unconverged search multiplies cost and adds a synthesis burden; it does not supply a stopping condition.

EXAM TIP

Exam Rule: - Two complementary stopping conditions: a coverage check (goal-relative - have the research questions been answered?) which stops the run as

45. When the agent calls `lookup_order` and receives details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

- (A) The orchestration layer automatically routes to the next tool based on the order's status field.
- (B) The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.
- (C) The order details are added to the conversation and the model reasons about which action to take.
- (D) The agent executes the remaining steps in a tool sequence planned at the start of the request.

Answer: (C) The order details are added to the conversation and the model reasons about which action to take.

EXPLANATION

This is the mechanism of the loop. The tool result is appended as a `tool_result` block in the conversation; on the next request the model reads it *in context*, applies the policy in its system prompt (e.g. "refunds within 30 days, otherwise escalate"), and emits the corresponding `tool_use` block. The decision lives in the model's reasoning over conversation state, not in the harness.

Why distractors fail:

- **A** - The orchestration layer transports results and executes requested calls; it does not inspect payload fields to pick the next tool. If it did, the system would be a workflow.
- **B** - A pre-configured decision tree is again a workflow. Nothing in the agentic loop consults such a structure.
- **D** - Contradicts the loop: the plan is not fixed at request start. Had a full sequence been planned up front, the 45-day fact could not have influenced anything.

EXAM TIP

Exam Rule: - This is the mechanism of the loop. The tool result is appended as a `tool_result` block in the conversation; on the next request the model r

1.2 Coordinator & Subagent Orchestration

Claude CCAF: Agentic Architecture & Orchestration - 15 questions

46. A workflow requires multiple independent research tasks before generating a final report; each queries different sources with no dependencies. Which orchestration strategy minimizes latency?

- (A) Execute research tasks sequentially to simplify coordination.
- (B) Execute all research tasks in parallel and synthesize results after all complete.
- (C) Execute only one research task and infer the remaining information.
- (D) Ask the user to perform the research manually.

Answer: (B) Execute all research tasks in parallel and synthesize results after all complete.

EXPLANATION

With independent tasks and no inter-task dependencies, parallel fan-out reduces total latency from the sum of task durations to the maximum single task duration.

EXAM TIP

Independent subtasks = Parallel fan-out with coordinator fan-in synthesis.

47. Coordinator plus 3 specialized subagents. An error in subagent B propagates, causing cascade failure; the coordinator can't isolate it. How do you prevent error cascade in multi-agent systems?

- (A) Subagent errors should cause complete workflow restart for consistency.
- (B) Coordinator should ignore subagent errors, completing the workflow with partial results.
- (C) All subagents should retry operations indefinitely until success.
- (D) Structured error propagation with failure isolation enabling coordinator-level recovery.

Answer: (D) Structured error propagation with failure isolation enabling coordinator-level recovery.

EXPLANATION

Isolation confines subagent B's failure so branches A and C remain healthy. Structured error propagation returns typed errors (step, failure type, partial data) allowing the coordinator to execute selective recovery.

EXAM TIP

Prevent error cascade = Failure isolation + Structured typed error propagation to coordinator.

48. Hub-and-spoke with coordinator plus 4 subagents. Subagents occasionally communicate directly; the coordinator loses visibility and information flow becomes unpredictable. What violates the hub-and-spoke principle?

- (A) Coordinator bottleneck requires peer-to-peer subagent communication for performance.
- (B) Direct subagent communication bypasses coordinator, violating hub-and-spoke architecture pattern.
- (C) Using 4 subagents exceeds the recommended maximum of 3 subagents per coordinator.
- (D) Subagents should share a common context pool enabling direct communication efficiency.

Answer: (B) Direct subagent communication bypasses coordinator, violating hub-and-spoke architecture pattern.

EXPLANATION

Hub-and-spoke requires that every message crosses the hub. Direct peer-to-peer communication bypasses the coordinator, destroying centralized observability, uniform error handling, and curated context passing.

EXAM TIP

Hub-and-spoke invariant: All inter-subagent communication **MUST** route through the central coordinator.

49. A subagent fails and returns an empty result; the coordinator continues synthesis assuming success, and the silent failure produces incorrect output. How should subagent failures be propagated?

- (A) Subagent should retry operation indefinitely until success before returning.
- (B) Coordinator should assume success when subagent returns any result.
- (C) Empty results indicate unavailable data; coordinator handles gracefully continuing.
- (D) Structured error propagation to coordinator with failure details enabling recovery decisions.

Answer: (D) Structured error propagation to coordinator with failure details enabling recovery decisions.

EXPLANATION

Empty results are ambiguous (data not found vs tool failure). Subagents must return structured error payloads ({status: 'error', code, step, partial_data}) so the coordinator can distinguish failures from legitimate empty data.

EXAM TIP

Never return empty results on tool failure; propagate structured errors with `isError: true`.

50. A web-search subagent is assigned a task requiring document analysis and lacks the tools. How should it handle the task-capability mismatch?

- (A) Subagent requests additional document analysis tools from the coordinator at runtime.
- (B) Subagent attempts the task using available web search tools, doing best effort.
- (C) Coordinator should never assign tasks beyond subagent's advertised capabilities.
- (D) Subagent returns structured error indicating `missing_required_tool` with capability description.

Answer: (D) Subagent returns structured error indicating `missing_required_tool` with capability description.

EXPLANATION

Fail fast with a machine-readable typed error (`missing_required_tool`) and capability details so the coordinator can immediately re-route to the document analysis specialist.

EXAM TIP

Tool capability mismatch -> Fail fast with typed `missing_required_tool` error.

51. Synthesis over 6 subagents' results produces contradictory claims, reducing report quality. How do you improve synthesis quality when handling contradictions?

- (A) Sequential subagent invocation allowing synthesis to guide subsequent research.
- (B) Structured subagent outputs with explicit claim-evidence pairs enabling contradiction detection.
- (C) Multiple synthesis passes with a voting mechanism for claim validation.
- (D) Increase synthesis agent thinking time for more careful contradiction analysis.

Answer: (B) Structured subagent outputs with explicit claim-evidence pairs enabling contradiction detection.

EXPLANATION

Contradiction detection fails in free prose because claims must be aligned to common propositions. Requiring structured outputs ({claim, evidence, source, confidence}) allows mechanical comparison and preserved provenance.

EXAM TIP

Contradiction resolution in synthesis = Structured claim-evidence-source pairs.

52. An orchestrator must route tasks between an expensive/thorough model and a cheap/fast model in a customer-complaint system. Which routing strategy is recommended?
- (A) Randomly distribute requests between models to balance load.
 - (B) Always use the larger model for customer-facing interactions to ensure quality.
 - (C) Use the small model for classification and routing, the large model for generating the final customer response, and the small model for logging and follow-up actions.
 - (D) Use the cheaper model for everything and only escalate when it returns low confidence.

Answer: (C) Use the small model for classification and routing, the large model for generating the final customer response, and the small model for logging and follow-up actions.

EXPLANATION

Match model capability to the difficulty of each step. Low-ambiguity classification/routing uses fast cheap models; high-stakes customer-facing synthesis uses the capable model; mechanical logging uses the cheap model.

EXAM TIP

Step-difficulty routing: Small model (classify/log) + Large model (generate response).

53. The web-search subagent returns 3 of 5 source categories (news archives and social feeds timed out); document analysis fully succeeds. Synthesis must produce a summary from mixed-quality inputs. Most effective error-propagation strategy?

- (A) Continue synthesis using only successful sources and produce output without mentioning what was unavailable.
- (B) The synthesis subagent asks the coordinator to retry timed-out sources with a longer timeout before starting.
- (C) The synthesis subagent returns an error to the coordinator, triggering a full retry or task failure.
- (D) Structure the synthesis output with coverage annotations indicating which conclusions are well-supported and where gaps exist due to unavailable sources.

Answer: (D) Structure the synthesis output with coverage annotations indicating which conclusions are well-supported and where gaps exist due to unavailable sources.

EXPLANATION

Graceful degradation with disclosure: Partial results (60% sources + 100% docs) support real conclusions if coverage annotations clearly mark supported vs missing source categories.

EXAM TIP

Partial subagent failures = Graceful degradation with explicit coverage gap annotations.

54. "Analyze the uploaded quarterly report" routes to the web-search agent 45% of the time. The web-search agent's tool is `analyze_content` - "analyzes content and extracts key information"; the document agent's is `analyze_document` - "analyzes documents and extracts key information." How should you fix the misrouting?

- (A) Rename the web-search tool to `extract_web_results` and update its description to "processes and returns information retrieved from web search and URLs."
- (B) Add few-shot examples to the coordinator prompt showing correct routing.
- (C) Expand the document analysis tool description with usage examples, leaving the web-search tool unchanged.
- (D) Add a pre-routing classifier that detects whether the user refers to uploaded files or web content.

Answer: (A) Rename the web-search tool to `extract_web_results` and update its description to "processes and returns information retrieved from web search and URLs."

EXPLANATION

The root cause is near-identical names and descriptions (`analyze_content` vs `analyze_document`). Fixing the tool interface with distinct verbs and domain-specific descriptions eliminates semantic overlap at the source.

EXAM TIP

Fix tool selection ambiguity at the interface: Unique names + distinct domain descriptions.

55. A report on "impact of remote work on urban real estate" discusses only office vacancy, missing residential migration and housing prices. Every subagent succeeded. The coordinator decomposed into "office vacancy trends," "commercial lease rates," "downtown foot traffic." Root cause and fix?

- (A) The coordinator's task decomposition was too narrow; improve decomposition to cover all major sub-domains before delegating.
- (B) The synthesis subagent failed to flag coverage gaps; add gap-detection instructions to synthesis.
- (C) The web search subagent's queries were too narrow; broaden its search query templates.
- (D) The subagents lack shared context about the overall goal; pass the full original query to every subagent.

Answer: (A) The coordinator's task decomposition was too narrow; improve decomposition to cover all major sub-domains before delegating.

EXPLANATION

All subtopics assigned were commercial real estate. Subagents executed their briefs flawlessly, but residential and housing sub-domains were never assigned. Coverage failures originate in the coordinator's decomposition.

EXAM TIP

Missing broad topic areas despite subagent success = Coordinator task decomposition was too narrow.

56. Research -> writing -> editing, each stage strictly dependent on the completed output of the previous one, with no loop-back or extra coordination. Which topology, and why?

(A) Hub-and-Spoke
(B) Hub-and-Spoke, but with the coordinator running all three agents in parallel
(C) Pipeline
(D) Peer-to-peer

Answer: (C) Pipeline

EXPLANATION

Strict sequential dependency with a single flow direction and no coordination decisions beyond handoff is a Pipeline (prompt chaining). A hub adds unnecessary overhead.

EXAM TIP

Strict sequential dependency chain with no feedback loops = Pipeline Topology.

57. You need centralized error handling, auditability, and the ability to intervene between every subagent call in a compliance-sensitive workflow. Which topology, and why?

(A) Hub-and-spoke
(B) Peer-to-peer
(C) Pipeline
(D) Hub-and-spoke, but letting subagents communicate directly once invoked

Answer: (A) Hub-and-spoke

EXPLANATION

Hub-and-spoke routes all calls through the central coordinator, providing centralized error policy, unified audit logging, and programmatic intervention points between subagent calls.

EXAM TIP

Centralized error handling + Auditability + Intervention points = Hub-and-Spoke Topology.

58. A Multi-Agent Research orchestrator stores session state in a single shared database all subagents read and write; the team observes inconsistencies. Which design MOST directly addresses this?

(A) Have the subagents lock the database during every single write.
(B) Concentrate state ownership at the orchestrator.
(C) Accept the inconsistencies as inherent to multi-agent systems.
(D) Replicate the entire database separately for every subagent.

Answer: (B) Concentrate state ownership at the orchestrator.

EXPLANATION

Inconsistencies stem from shared mutable state with concurrent writers. Concentrating state ownership at the orchestrator establishes a single writer and single source of truth.

EXAM TIP

Multi-agent state inconsistency fix = Single writer / orchestrator-owned state.

59. An orchestrator prompt says "Combine subagent findings into a final report." Reports omit some findings without explanation. Which change BEST ensures every finding appears?

(A) Lower temperature to reduce omission variance.
(B) Increase max_tokens so all findings have room to fit.
(C) Require: enumerate every finding ID in an appendix, mark each as included/excluded with a one-sentence rationale.
(D) Concatenate all subagent outputs verbatim so nothing can be dropped.

Answer: (C) Require: enumerate every finding ID in an appendix, mark each as included/excluded with a one-sentence rationale.

EXPLANATION

Enforcing an explicit output contract (disposition per finding ID) makes omission structurally detectable and turns dropped items into deliberate, reviewable decisions.

EXAM TIP

Prevent synthesis omissions = Output contract requiring finding ID enumeration & disposition rationale.

60. An orchestrator dispatches three parallel subagents with a voting (majority-wins) fan-in. Which insight MOST directly applies?

- (A) Voting fan-in increases reliability through redundancy but pays 3x the cost of single execution.
- (B) Voting across branches is essentially always cheaper overall than a single execution.
- (C) Voting is inappropriate for any agentic task.
- (D) Voting requires sequential execution.

Answer: (A) Voting fan-in increases reliability through redundancy but pays 3x the cost of single execution.

EXPLANATION

Voting fan-in is a classic reliability-for-cost trade-off: 3 independent parallel attempts make single anomalous errors non-decisive at 3x token spend.

EXAM TIP

Voting fan-in = High reliability for error-sensitive judgements at 3x token cost multiplier.

1.3 Subagent Invocation, Task Tool & Context Isolation

Claude CCAF: Agentic Architecture & Orchestration - 10 questions

61. A coordinator spawns 4 subagents, passing the complete 60-message conversation history to each; subagents only need the task description. Token costs are excessive. How do you optimize context passing?

- (A) Prompt caching for conversation history, reducing token costs of repeated context.
- (B) Pass only task-relevant information to each subagent, isolating context per subagent role.
- (C) Sequential subagent invocation enabling context sharing and reuse.
- (D) Compress conversation history, reducing token count while preserving information.

Answer: (B) Pass only task-relevant information to each subagent, isolating context per subagent role.

EXPLANATION

Subagents run with isolated context by design. Passing full conversation history creates token bloat and dilutes attention. Provide only curated, task-specific instructions.

EXAM TIP

Subagent context passing = Curated task-relevant brief only; never dump full coordinator history.

62. A synthesis subagent invoked after web search and document analysis has no knowledge of their findings. Why, and what is the fix?

- (A) Subagents share a memory pool; findings weren't written correctly. Fix: call `memory_write()` in each subagent.
- (B) The coordinator needs to call a `sync_context` tool between agent invocations.
- (C) The synthesis agent's context window is too small. Fix: increase `max_tokens`.
- (D) Subagents operate with isolated context and do not inherit parent context automatically. Fix: include complete findings from prior agents directly in the synthesis subagent's prompt.

Answer: (D) Subagents operate with isolated context and do not inherit parent context automatically. Fix: include complete findings from prior agents directly in the synthesis subagent's prompt.

EXPLANATION

Subagents start with isolated context and do not automatically inherit parent conversation history. The coordinator must explicitly inject prior findings into the subagent's spawn prompt.

EXAM TIP

Subagents DO NOT inherit parent context. Coordinator must explicitly pass required facts in prompt.

63. A "database-migration-writer" subagent that only needs to read schema and write migration files currently has full parent tool access, including production deploys and database deletion. No incidents yet. Most important reason to fix this before launch?

- (A) Scoping the subagent's tools to only what its task requires limits the blast radius of a reasoning error or prompt injection, independent of whether one has happened yet.
- (B) Broad tool access makes responses slower because more tools must be considered per turn.
- (C) Broad tool access increases token cost since more tool definitions are included in context.
- (D) Other engineers reviewing the config will be confused about what it's supposed to do.

Answer: (A) Scoping the subagent's tools to only what its task requires limits the blast radius of a reasoning error or prompt injection, independent of whether one has happened yet.

EXPLANATION

Least privilege bounds the worst-case blast radius. If a subagent is manipulated via prompt injection or suffers a reasoning error, it structurally cannot execute deploy or delete actions.

EXAM TIP

Tool access scoping = Least privilege to bound blast radius against injection & reasoning errors.

64. Which fields best support a reliable research -> writing subagent handoff?

- (A) Just the raw text output of the research subagent's final response.
- (B) Only a success/failure flag plus a pointer to where full logs can be found.
- (C) The entire research subagent's tool-call history, so the writer can re-derive anything it needs.
- (D) Structured fields for the specific findings/data produced, the original task's relevant constraints, and an explicit statement of what's still unresolved/uncertain.

Answer: (D) Structured fields for the specific findings/data produced, the original task's relevant constraints, and an explicit statement of what's still unresolved/uncertain.

EXPLANATION

A reliable handoff schema contains: (1) Findings/data payload, (2) Task constraints (audience, tone, length), and (3) Unresolved/uncertain items to prevent presenting gaps as settled facts.

EXAM TIP

Reliable handoff schema = Findings payload + Task constraints + Unresolved uncertainties.

65. A web-search subagent's fetched content contains an embedded prompt injection. What limits the damage?

- (A) The coordinator's own prompt tells it to ignore any instructions found in fetched content.
- (B) The subagent's output is logged, so damage can be detected after the fact.
- (C) Prompt injections can't affect subagents because they only process search results, not user instructions.
- (D) The subagent's context is isolated and its tool access is scoped narrowly, so even if it's manipulated, it can't take actions or access data outside what it was already permitted to touch.

Answer: (D) The subagent's context is isolated and its tool access is scoped narrowly, so even if it's manipulated, it can't take actions or access data outside what it was already permitted to touch.

EXPLANATION

Prompt injection cannot be prevented by instructions alone. Isolated context and narrow tool permissions bound the blast radius - a compromised subagent holds no credentials or destructive tools.

EXAM TIP

Defense against indirect injection in subagents = Context isolation + Narrow tool permissions.

66. A coordinator's attempt to spawn a document-analysis subagent fails; its allowedTools is missing an entry. What's missing?

- (A) "Spawn"
- (B) Nothing is missing; subagent spawning doesn't require any tool permission.
- (C) The subagent's own AgentDefinition must list "Task" in its allowedTools.
- (D) "Task"

Answer: (D) "Task"

EXPLANATION

Delegation and subagent spawning occur via the Task tool. The coordinator must include "Task" in its allowedTools configuration to invoke subagents.

EXAM TIP

Coordinator allowedTools must include 'Task' to spawn subagents.

67. Coordinator prompts to subagents read like step-by-step procedural checklists ("First call search_web with X, then Y, then stop"), and subagents rarely adapt to unexpected findings. Most likely design issue?

- (A) The subagents are missing tool access needed to adapt their procedures.
- (B) The coordinator should switch to fork_session so each subagent can explore divergent procedures.
- (C) This is expected and correct behavior.
- (D) The coordinator prompts specify rigid procedures rather than research goals and quality criteria, which limits the subagent's ability to adapt as it discovers new information.

Answer: (D) The coordinator prompts specify rigid procedures rather than research goals and quality criteria, which limits the subagent's ability to adapt as it discovers new information.

EXPLANATION

Procedural checklists convert agents into rigid workflow executors. Delegating research goals and quality criteria (breadth, recency, contradiction checks) gives subagents latitude to adapt.

EXAM TIP

Delegate goals and quality criteria, not rigid procedural step checklists.

68. A firm wants to use compaction in subagents as well as the parent. Which design consideration BEST applies?
- (A) Subagents cannot be compacted at all.
 - (B) Subagents should compact more aggressively than parents, since tighter scope means heavier summarization does less damage.
 - (C) Subagents typically have shorter lifespans and tighter scope, so compaction is often unnecessary.
 - (D) Require compaction for every subagent as a universal context-hygiene rule.

Answer: (C) Subagents typically have shorter lifespans and tighter scope, so compaction is often unnecessary.

EXPLANATION

Subagents are scoped to bounded tasks and short lifespans, rarely generating context pressure. Compacting short-lived subagents wastes summarization passes and risks losing task-dense details.

EXAM TIP

Subagent compaction is usually unnecessary due to short lifespans and curated narrow scope.

69. An orchestrator dispatches refunds to a refund-subagent that holds the refund API tool - but the orchestrator also has the refund API tool registered for emergency direct refunds. What concrete cost does this duplication impose?
- (A) The orchestrator's tool registry consumes roughly twice the memory.
 - (B) Duplicate registrations cause the MCP protocol to reject the session at startup.
 - (C) The subagent's calls will be routed to the orchestrator's tool handler instead of its own.
 - (D) The orchestrator may invoke the refund tool directly for routine cases, bypassing the audit logging and review steps the refund-subagent layer adds.

Answer: (D) The orchestrator may invoke the refund tool directly for routine cases, bypassing the audit logging and review steps the refund-subagent layer adds.

EXPLANATION

The subagent is a control layer adding audit logging, validation, and policy checks. Registering the same tool on the orchestrator provides a bypass path that skips those controls.

EXAM TIP

Duplicate tool registration on orchestrator risks bypassing subagent-level policy & audit controls.

70. A Claude Code-driven literature review where the parent must keep its context clean for final synthesis. Which architecture BEST combines Claude Code's execution model with context isolation?
- (A) Parent reads every source itself sequentially so synthesis has full provenance.
 - (B) Parent spawns one subagent that reads all sources for all topics in one giant pass.
 - (C) Parent issues Agent tool calls per topic; subagents read sources in their own contexts and return only structured summaries, keeping parent context lean for synthesis.
 - (D) Parent uses Bash grep across the corpus and pipes raw matches into its own reasoning.

Answer: (C) Parent issues Agent tool calls per topic; subagents read sources in their own contexts and return only structured summaries, keeping parent context lean for synthesis.

EXPLANATION

Heavy reading occurs in isolated subagent contexts. Only distilled structured summaries cross back to the parent, preserving the parent's context window for high-level synthesis.

EXAM TIP

High-volume literature review = Subagents per topic read in isolation -> return structured summaries.

1.4 Workflow Enforcement & Human Handoff Protocols

Claude CCAF: Agentic Architecture & Orchestration - 10 questions

71. A support agent processes refunds without verifying account ownership in 8% of cases despite a system prompt saying "always verify customer before refunds." Which architectural fix provides a deterministic guarantee for financial operations?
- (A) Improve the system prompt with stronger instructions and more examples.
 - (B) Add few-shot examples showing get_customer before every refund.
 - (C) Implement a programmatic prerequisite hook blocking process_refund until get_customer completes successfully.
 - (D) Deploy a separate classifier routing refund requests through identity verification.

Answer: (C) Implement a programmatic prerequisite hook blocking process_refund until get_customer completes successfully.

EXPLANATION

Prompts are probabilistic. Financial operations requiring deterministic guarantees must use code-level PreToolUse hooks that block execution unless prerequisite conditions (e.g. verified get_customer) are satisfied.

EXAM TIP

Deterministic financial/policy guarantee = PreToolUse programmatic prerequisite hook.

72. A support agent must compile a self-contained handoff summary when escalating; the human has no transcript access. What must be included?

- (A) Escalation reason only, as the human will investigate independently.
- (B) References to conversation turn numbers for the human to look up.
- (C) A brief summary assuming the human has full conversation context.
- (D) Customer ID, conversation summary, root cause, refund amount, recommended action.

Answer: (D) Customer ID, conversation summary, root cause, refund amount, recommended action.

EXPLANATION

A self-contained handoff must provide all necessary facts directly (Customer ID, root cause, amounts, attempted actions, recommended next steps) without requiring transcript access.

EXAM TIP

Self-contained handoff: Customer ID + Root cause + Amounts/Details + Attempted steps + Recommended action.

73. A customer asks to speak with a human manager after one unsuccessful troubleshooting attempt. The agent has more steps it could try. What should the agent do?

- (A) Ask the customer if they'd like to try one more troubleshooting step before escalating.
- (B) Escalate but first run diagnostic checks to provide the human with context.
- (C) Honor the customer's explicit request for a human representative immediately.
- (D) Attempt the remaining troubleshooting steps since the agent may resolve the issue.

Answer: (C) Honor the customer's explicit request for a human representative immediately.

EXPLANATION

An explicit customer request for a human is a hard escalation trigger. Overriding it damages user trust. Escalate immediately with the context already gathered.

EXAM TIP

Explicit customer request for human = Immediate escalation without delaying for extra diagnostic calls.

74. Policy requires human approval for refunds above \$500; a customer requests \$847. An emphatic system prompt still yields a 3% failure rate. What fix provides zero tolerance?

- (A) A separate classifier model evaluating each refund request first.
- (B) A self-check step where the agent validates the amount before calling process_refund.
- (C) An application-layer hook intercepting process_refund, blocking it server-side when the amount exceeds \$500, and triggering human escalation.
- (D) Strengthen the system prompt with more emphatic language and examples.

Answer: (C) An application-layer hook intercepting process_refund, blocking it server-side when the amount exceeds \$500, and triggering human escalation.

EXPLANATION

Threshold comparisons are deterministic numbers. Server-side PreToolUse hooks enforce numerical thresholds at code level (amount > 500 -> block & escalate), providing 0% bypass.

EXAM TIP

Zero-tolerance policy threshold = Server-side hook intercepting tool call and enforcing threshold.

75. A payment call times out, the agent retries, and a duplicate charge occurs. What pattern prevents duplicate transactions?

- (A) Increase the timeout to prevent timeouts from occurring.
- (B) Add a 30-second delay between retries so the first request completes or fails.
- (C) Use idempotency keys with each payment request so retries produce the same result as the original call.
- (D) Check the account balance before retrying to see if the first charge went through.

Answer: (C) Use idempotency keys with each payment request so retries produce the same result as the original call.

EXPLANATION

Timeouts are ambiguous (network drop vs processing success). Idempotency keys ensure payment providers deduplicate retries and return existing transaction records without double charging.

EXAM TIP

Prevent duplicate charges on timeout retry = Client-generated Idempotency Keys.

76. In Claude Code's permission model, what is the correct evaluation order?

- (A) Allow -> Ask -> Deny
- (B) Ask -> Allow -> Deny
- (C) Deny -> Ask -> Allow
- (D) Allow -> Deny -> Ask

Answer: (C) Deny -> Ask -> Allow

EXPLANATION

Deny is evaluated first so prohibitions cannot be overridden by broad allows. Ask is evaluated second. Allow is evaluated last.

EXAM TIP

Claude Code permission precedence order: Deny -> Ask -> Allow.

77. The refund flow works end to end, but 3% of sessions refund without a fresh identity check - traced to a session-cache race condition under load, not an agent reasoning error. Most effective fix?

- (A) Add a hook-based prerequisite that blocks process_refund unless a fresh, cache-validated get_customer result exists for this turn.
- (B) Add few-shot examples reinforcing "always verify identity first."
- (C) Reduce the session cache TTL to shrink the race window.
- (D) Add a retry loop that re-runs get_customer whenever process_refund is called.

Answer: (A) Add a hook-based prerequisite that blocks process_refund unless a fresh, cache-validated get_customer result exists for this turn.

EXPLANATION

Because the defect is a session-cache race rather than agent reasoning, prompt changes are irrelevant. A PreToolUse hook validating fresh, verified identity check for the current turn closes the race condition.

EXAM TIP

Cache race condition / state staleness = PreToolUse hook validating freshness on current turn.

78. A tool result passes schema validation but refund_amount is negative, which is nonsensical here. What does this reveal?

- (A) The schema itself is broken and needs a type change from number to string.
- (B) This is purely a reasoning error, fixable via prompt instructions rather than validation.
- (C) Schema validation is redundant once semantic validation exists, so it can be dropped.
- (D) Schema validation alone isn't sufficient.

Answer: (D) Schema validation alone isn't sufficient.

EXPLANATION

Schema validation verifies structure and types (e.g. number), but cannot verify domain semantics (e.g. amount >= 0, date not in future). Semantic validation gates are complementary and required.

EXAM TIP

Schema validation enforces structure/type; Semantic validation enforces domain business logic.

79. An agent occasionally uses an overly casual tone in customer-facing summaries. No safety, financial, or irreversible consequence - a style preference. Best fix?

- (A) Add a validation gate rejecting any output not matching a strict tone template.
- (B) This requires the same programmatic enforcement as a payment threshold, since any behavior can theoretically drift.
- (C) A prompt instruction/style guide is sufficient here.
- (D) Add a hook blocking any response containing casual phrasing.

Answer: (C) A prompt instruction/style guide is sufficient here.

EXPLANATION

Controls should be proportional to risk. Stylistic and tone preferences are low-stakes and subjective, best handled via prompt instructions and style guides without heavyweight hooks.

EXAM TIP

Low-stakes style/tone preference = Prompt instruction/style guide (do not over-engineer with hooks).

80. A Bash tool is exposed via MCP; security wants destructive shell commands refused. An engineer adds "You are a careful DevOps expert; never run destructive commands." A red-team request still elicits one. MOST accurate architectural assessment?
- (A) The persona grants authority to block destructive commands; the failure must stem from a malformed prompt section.
 - (B) Setting temperature to 0 will make refusals deterministic.
 - (C) More forceful language and capitalized warnings will make the constraint reliable enough to serve as the control.
 - (D) Prompt-based guardrails are probabilistic and can fail under adversarial input; the destructive-command block belongs in the MCP tool's deterministic permission layer.

Answer: (D) Prompt-based guardrails are probabilistic and can fail under adversarial input; the destructive-command block belongs in the MCP tool's deterministic permission layer.

EXPLANATION

Personas and prompt instructions are probabilistic and susceptible to adversarial prompt injection. Destructive command restrictions must be enforced in the tool's deterministic permission/command-allowlist layer.

EXAM TIP

Adversarial security / destructive actions = Enforce in code/MCP permission layer, never prompts.

1.5 Agent SDK Hooks, Lifecycle Events & Policy Enforcement

Claude CCAF: Agentic Architecture & Orchestration - 7 questions

81. A compliance workflow must verify identity before financial operations; a system prompt saying "verify first" is insufficient. Which Agent SDK hook enforces verification?
- (A) PostToolUse hook validating tool results after execution for compliance.
 - (B) Hooks cannot enforce compliance; programmatic checks required separately.
 - (C) Agent reasoning hook guiding decision making toward compliant behavior.
 - (D) PreToolUse hook checking conditions before tool execution, blocking violations.

Answer: (D) PreToolUse hook checking conditions before tool execution, blocking violations.

EXPLANATION

PreToolUse intercepts the call before execution and can return exit code 2 / deny to prevent the unauthorized financial action.

EXAM TIP

PreToolUse = Gating before execution. PostToolUse = Inspecting/transforming after execution.

82. Multiple MCP tools return timestamps as Unix epochs, ISO 8601, and locale-specific strings; the agent struggles with cross-format comparison. Where should timestamp normalization occur?
- (A) Fix the timestamp format at source in all MCP tools, returning a consistent format.
 - (B) The agent should flexibly handle all common formats without normalization.
 - (C) PostToolUse hook normalizing all timestamps to ISO 8601 before processing.
 - (D) Agent system prompt with instructions for handling multiple formats.

Answer: (C) PostToolUse hook normalizing all timestamps to ISO 8601 before processing.

EXPLANATION

When consuming third-party or heterogeneous MCP tools you cannot modify at source, a PostToolUse hook normalizes tool outputs into canonical ISO 8601 before the model reasons over them.

EXAM TIP

Normalizing heterogeneous third-party tool outputs = PostToolUse transformation hook.

83. Extracted services must use com.company.service.<service-name>; subagents sometimes write com.company.app.<name> or com.company.<name>. A teammate proposes stronger system-prompt instructions. Correct approach?
- (A) Add the naming convention to the system prompt with three concrete examples.
 - (B) Implement a PostToolUse hook that inspects written files and normalises package declarations to the correct format.
 - (C) Create a separate validation subagent reviewing written files after each batch.
 - (D) Use tool_choice to restrict the agent to a custom write_java_file tool that enforces the convention.

Answer: (B) Implement a PostToolUse hook that inspects written files and normalises package declarations to the correct format.

EXPLANATION

A mechanically checkable regex/string pattern is best enforced mechanically. A PostToolUse hook on Edit/Write checks the written file and automatically normalizes package headers.

EXAM TIP

Mechanical syntax/package normalization on write = PostToolUse hook.

84. A hard rule: never push to main. Where should it live so Claude cannot skip it?

- (A) In the project CLAUDE.md as an "IMPORTANT" rule.
- (B) In the local CLAUDE.md so it is scoped to you.
- (C) In a pre-tool use hook that stops the push.
- (D) In a skill's reference.md file.

Answer: (C) In a pre-tool use hook that stops the push.

EXPLANATION

A PreToolUse hook inspecting Bash commands and blocking git push origin main executes in deterministic code outside the LLM context, making it impossible to bypass via prompt phrasing.

EXAM TIP

Non-negotiable hard constraint (e.g. block push to main) = PreToolUse hook.

85. A PreToolUse hook meant to validate a precondition throws an unexpected exception due to a bug in the hook's own code. Safest default?

- (A) Disable the hook automatically after its first error
- (B) Fail closed
- (C) Fail open
- (D) Retry the hook indefinitely until it stops erroring

Answer: (B) Fail closed

EXPLANATION

Failing closed (blocking the tool call when the validator errors) preserves safety invariants. Failing open would allow unchecked execution whenever a hook crashes.

EXAM TIP

Safety-critical hook exceptions **MUST** fail closed (deny/block execution).

86. A compliance rule prevents refund processing above \$500 without manager approval. Which hook configuration correctly implements this?

- (A) A PreToolUse hook on process_refund with exit code 2 and JSON output containing a deny decision when the amount exceeds \$500.
- (B) A SessionStart hook setting a global \$500 limit for all financial operations.
- (C) A PostToolUse hook on process_refund that reverses transactions over \$500.
- (D) A system prompt instruction specifying the \$500 threshold with examples.

Answer: (A) A PreToolUse hook on process_refund with exit code 2 and JSON output containing a deny decision when the amount exceeds \$500.

EXPLANATION

PreToolUse scoped to process_refund uses exit code 2 with JSON decision output {"decision": "deny", "reason": "..."} to block the call and return a clear rationale to the model.

EXAM TIP

PreToolUse blocking: Exit code 2 + JSON deny decision to block tool execution.

87. Every Bash command issued by the agent must appear in a central audit log with the user identity. Which mechanism BEST supports this?

- (A) A CLAUDE.md instruction asking the agent to log each command itself.
- (B) Manual review of session transcripts after each session.
- (C) A PostToolUse hook (or PreToolUse for pre-execution capture) firing on every Bash invocation, writing command + arguments + user identity to the central audit log.
- (D) Enable verbose logging on the underlying system shell.

Answer: (C) A PostToolUse hook (or PreToolUse for pre-execution capture) firing on every Bash invocation, writing command + arguments + user identity to the central audit log.

EXPLANATION

Hooks fire deterministically at the harness level on every matching tool invocation, capturing structured command parameters, timestamp, and user context without relying on model compliance.

EXAM TIP

Central audit logging of tool commands = PostToolUse/PreToolUse hook firing on tool matchers.

1.6 Task Decomposition Strategies & Planning

Claude CCAF: Agentic Architecture & Orchestration - 10 questions

88. Decomposition creates a fixed 4 subtopics for all topics: simple topics are over-decomposed, complex ones under-decomposed. How do you improve the approach?

- (A) Increase to a fixed 8 subtopics.
- (B) Dynamic decomposition assessing topic complexity to determine the appropriate subtopic count.
- (C) Reduce to a single comprehensive subtopic.
- (D) Predefined topic taxonomy with mappings to fixed subtopic structures.

Answer: (B) Dynamic decomposition assessing topic complexity to determine the appropriate subtopic count.

EXPLANATION

Fixed subtopic counts are inherently flawed. Sizing must be dynamic and complexity-proportional: assess breadth first, then spawn the right number of subtasks.

EXAM TIP

Dynamic adaptive decomposition sizes subtasks to query complexity; avoid fixed subtopic counts.

89. A team writes tests first, then uses Claude to implement code passing those tests, iterating by sharing test failures. Which workflow pattern is this?

- (A) The test-driven iteration pattern: write tests first, then iterate by sharing failures.
- (B) Plan mode followed by direct execution.
- (C) Prompt chaining with separate steps for testing and implementation.
- (D) The interview pattern: have Claude ask questions before implementing.

Answer: (A) The test-driven iteration pattern: write tests first, then iterate by sharing failures.

EXPLANATION

Test-driven iteration uses tests as executable specifications, feeding failing assertions back into the loop as an objective correction signal.

EXAM TIP

Write tests first -> share failure output in loop = Test-Driven Iteration Pattern.

90. Adding comprehensive test coverage to a legacy codebase with unclear module boundaries and undocumented dependencies. Most appropriate decomposition strategy?

- (A) Adaptive decomposition: first map the codebase structure, identify high-impact areas, then create a prioritized plan that adapts as dependencies are discovered.
- (B) Fixed sequential prompt chaining: tests for each file in alphabetical order, one per turn.
- (C) Decompose by test type: all unit tests, then integration, then end-to-end.
- (D) Give Claude the entire codebase and a single instruction to "add comprehensive tests."

Answer: (A) Adaptive decomposition: first map the codebase structure, identify high-impact areas, then create a prioritized plan that adapts as dependencies are discovered.

EXPLANATION

You cannot plan what you have not mapped. With unknown boundaries, discovery is the mandatory first phase, followed by a prioritized adaptive plan that adjusts as dependencies surface.

EXAM TIP

High architectural uncertainty / legacy code = Adaptive decomposition (Explore -> Plan -> Code).

91. Auth extraction and billing extraction are independent; "wire up service mesh config" depends on both. One engineer, limited time. Most reliable decomposition structure?

- (A) Fan-out the two independent extractions in parallel, then fan-in to a single sequential subtask that only starts once both extractions are verified complete.
- (B) Run all three subtasks in parallel to save the most time.
- (C) Run all three strictly sequentially in the order mentioned.
- (D) Decompose into a single subtask doing the entire migration.

Answer: (A) Fan-out the two independent extractions in parallel, then fan-in to a single sequential subtask that only starts once both extractions are verified complete.

EXPLANATION

Express the DAG accurately: parallel fan-out for independent nodes, with an explicit synchronization barrier ensuring dependent nodes execute only after upstream completion is verified.

EXAM TIP

DAG schedule: Parallel fan-out for independent tasks -> Verification barrier -> Sequential fan-in.

92. "Clean up the auth module" - it contains dead code, inconsistent naming, and a non-constant-time token comparison. Time-boxed to one turn. What FIRST?

- (A) Ask the user to clarify exactly what "clean up" means before doing anything.
- (B) Fix everything found, including the security issue.
- (C) Only fix the security issue since it's highest severity, leaving the rest for a separate task.
- (D) Proceed with the unambiguous, low-risk cleanup (dead code, naming) immediately, but pause and explicitly flag the security issue for confirmation before touching it, since fixing it changes security-relevant behavior.

Answer: (D) Proceed with the unambiguous, low-risk cleanup (dead code, naming) immediately, but pause and explicitly flag the security issue for confirmation before touching it, since fixing it changes security-relevant behavior.

EXPLANATION

Split work by risk profile: execute unambiguous, reversible cleanup immediately while explicitly pausing and gating the security-relevant behaviour change for user confirmation.

EXAM TIP

Ambiguous task with security implications: Execute safe cleanup + Flag security changes for confirmation.

93. Translate 200 nearly-identical product templates into 5 languages each, with a fixed, well-understood structure. Best decomposition?

- (A) Fixed prompt chaining: a predictable sequence applied uniformly across all templates.
- (B) Adaptive decomposition: explore the templates first, then build a plan that adapts.
- (C) A coordinator that decides template by template whether translation is needed.
- (D) Spawn one subagent per language with full isolated context.

Answer: (A) Fixed prompt chaining: a predictable sequence applied uniformly across all templates.

EXPLANATION

Zero structural uncertainty and uniform work fits fixed prompt chaining. It is cheap, predictable, easily parallelized, and avoids adaptive overhead.

EXAM TIP

Uniform, repetitive, known structure = Fixed Prompt Chaining.

94. Three replans into an open-ended investigation, the current plan no longer clearly serves the original goal. What should happen?

- (A) Escalate to a human on every replan, regardless of how minor.
- (B) Explicitly re-check the current plan against the original goal before continuing to replan, rather than letting each incremental replan drift the objective without ever validating against the original ask.
- (C) Continue replanning since adaptive decomposition is designed to evolve freely.
- (D) Revert to the very first plan, since the original plan was presumably correct.

Answer: (B) Explicitly re-check the current plan against the original goal before continuing to replan, rather than letting each incremental replan drift the objective without ever validating against the original ask.

EXPLANATION

Goal drift occurs when incremental replanning evaluates only against the previous drifted plan. Periodically re-validating the plan against the original objective reference prevents drift.

EXAM TIP

Prevent goal drift = Re-validate replanned actions against original root goal & constraints.

95. "Rename this variable across the codebase" split into 6 subagents (one per file type), each with coordinator hand-off overhead, for one mechanical low-risk operation. Best assessment?

- (A) Correctly decomposed, since parallelizing by file type is always more efficient.
- (B) Six subagents is appropriate as long as the token budget allows.
- (C) The decomposition is fine, but it should use peer-to-peer topology.
- (D) This is over-decomposition.

Answer: (D) This is over-decomposition.

EXPLANATION

Decomposition is only justified by parallelism gains, context isolation, or specialization. Spawning 6 subagents for a mechanical variable rename introduces coordination overhead that exceeds the work.

EXAM TIP

Over-decomposition: Adding subagent overhead to mechanical single-pass tasks adds latency and divergence.

96. Designing a general-purpose orchestration layer handling both sequential and parallel execution uniformly. MOST consistent design strategy?

- (A) Force everything strictly sequential to keep the orchestrator simple.
- (B) Express subtasks with explicit dependencies in a DAG; the orchestrator runs independent nodes in parallel and dependent nodes sequentially.
- (C) Force everything parallel to maximize speed.
- (D) Let each subagent decide its own execution pattern.

Answer: (B) Express subtasks with explicit dependencies in a DAG; the orchestrator runs independent nodes in parallel and dependent nodes sequentially.

EXPLANATION

A DAG (Directed Acyclic Graph) is the universal representation: nodes with no incoming edges execute concurrently in parallel; dependent nodes execute sequentially upon predecessor completion.

EXAM TIP

Universal orchestration = DAG dependency graph with automatic parallel/sequential scheduling.

97. An extraction agent must store audited records. A source API returns a permanent schema-change error; the agent replans to a looser scraping path, succeeds, but the records now skip mandatory audit validation. MOST accurate diagnosis of the replanning defect?

- (A) The agent should never replan after a tool error; it should halt and escalate every failure.
- (B) The replan satisfied the immediate obstacle but was not validated against the original goal constraint requiring audited records.
- (C) The agent misclassified the error; a permanent schema-change error should have been retried with exponential backoff.
- (D) The agent used in-context memory for the audit records when it should have used semantic memory.

Answer: (B) The replan satisfied the immediate obstacle but was not validated against the original goal constraint requiring audited records.

EXPLANATION

The replan solved the local obstacle ('get the data') while silently discarding a non-negotiable constraint ('records must be audited'). Replans must be validated against all original constraints.

EXAM TIP

Replanning failure: Overcoming local obstacle while dropping global security/audit constraints.

1.7 Session State, Resumption & Forking

Claude CCAF: Agentic Architecture & Orchestration - 5 questions

98. Explore two distinct refactoring approaches from the same codebase analysis without context contamination. Which session technique?

- (A) Resume the same session and clearly label each approach in the prompts.
- (B) Use `fork_session` to create two independent branches from the shared analysis baseline.
- (C) Use a single session but instruct the agent to keep the approaches in separate mental namespaces.
- (D) Start two completely fresh sessions, repeating the codebase analysis in each.

Answer: (B) Use `fork_session` to create two independent branches from the shared analysis baseline.

EXPLANATION

`fork_session` branches from a completed baseline analysis, giving both explorations identical starting context with zero cross-contamination.

EXAM TIP

Divergent exploration from common baseline = `fork_session`.

99. What does the `--resume` flag do in Claude Code?

- (A) It continues a specific named prior conversation session.
- (B) It restores the last auto-saved scratchpad file.
- (C) It reloads all `CLAUDE.md` configuration files.
- (D) It re-runs the last command with fresh context.

Answer: (A) It continues a specific named prior conversation session.

EXPLANATION

`--resume <session-name>` loads and continues a specific named session. In contrast, `--continue` picks up the most recent session.

EXAM TIP

`--resume <name>` = Target specific session. `--continue` = Pick up latest session.

100After 45 minutes, several early file reads are stale because a colleague pushed changes, and the agent is making recommendations from outdated contents. Best recovery strategy?

- (A) Continue in the current session and ask the agent to re-read the changed files.
- (B) Start a completely new session with no context from the previous one.
- (C) Start a fresh session with a summary of the key findings and decisions, then read the changed files for current state.
- (D) Use `fork_session` to create a new branch that excludes the stale tool results.

Answer: (C) Start a fresh session with a summary of the key findings and decisions, then read the changed files for current state.

EXPLANATION

When context contains broad stale reads and contaminated reasoning, starting fresh with an injected summary of durable decisions sheds stale baggage and re-establishes clean state.

EXAM TIP

Broad context staleness / contaminated reasoning = Fresh session + injected summary + re-read.

101A multi-agent pipeline crashes at subagent 7 of 12. How should the system recover without reprocessing the first 6?

- (A) Re-run the full pipeline with a faster model.
- (B) Store the full conversation history in memory and replay it on restart.
- (C) Log all intermediate results to a database and query it on restart.
- (D) Have each agent export structured state to a known file location; the coordinator loads a manifest on resume and injects agent states into prompts.

Answer: (D) Have each agent export structured state to a known file location; the coordinator loads a manifest on resume and injects agent states into prompts.

EXPLANATION

State durability requires: (1) each agent persisting structured outputs, (2) coordinator reading a completion manifest, and (3) injecting completed state into downstream prompts.

EXAM TIP

Crash recovery = Persisted structured state files + Coordinator manifest loading + Prompt injection.

102Resuming a session where only one specific file changed and the rest of the prior findings are still valid. Most efficient reliable approach?

- (A) Resume without mentioning the change at all.
- (B) Use `fork_session` to create a branch specifically to handle the changed file.
- (C) Inform the resumed session about the specific file change for targeted re-analysis, rather than requiring full re-exploration.
- (D) Discard the entire session and start fresh.

Answer: (C) Inform the resumed session about the specific file change for targeted re-analysis, rather than requiring full re-exploration.

EXPLANATION

When staleness is narrow and known (one file), resuming the session and explicitly informing Claude of the changed file scopes the re-read while preserving all still-valid architectural context.

EXAM TIP

Narrow, isolated file change = Resume session + explicitly prompt about the changed file.

3.1 CLAUDE.md Hierarchy & Memory Management

Claude CCAF: Claude Code Configuration & Workflows - 12 questions

103A pattern is defined in `.claude/rules/error_handling.yaml` with `glob: '**/*.ts'`. Some developers aren't following it; investigation shows a user-level `~/.claude.json` overriding project rules. How do you ensure project rules take precedence?

- (A) Add a flag in `.mcp.json` that enforces project rules and disables personal configuration overrides.
- (B) Remove `~/.claude.json`, forcing all developers to use only project rules.
- (C) User-level `~/.claude.json` takes precedence over project rules by design.
- (D) Team conventions are the only enforcement mechanism, since user-level overrides of project rules are technically possible and cannot be blocked by any configuration flag.

Answer: (D) Team conventions are the only enforcement mechanism, since user-level overrides of project rules are technically possible and cannot be blocked by any configuration flag.

EXPLANATION

Of the four options it is the only one that is both truthful about the mechanism and responsive to the question: no per-project flag exists to disable personal configuration, so at the level described the remedy is social - documented team convention, code review, and CI checks - rather than configurational.

> **Important caveat for the exam and for practice.** Two things are worth knowing beyond this item. First, for `settings.json` values the documented precedence actually places **project settings above user settings** (see GEN3-006), so a user file does not generally override project configuration. Second, an organisation *can* enforce policy that users cannot override - that is exactly what enterprise `managed-settings.json` is for (UD3-086). Neither appears among the options here, so D remains the best available answer, but do not carry away the belief that overrides are unpreventable in general.

Why distractors fail:

- **A** - `.mcp.json` configures MCP servers. It has no bearing on rule precedence, and no such flag exists.
- **B** - Deletes a file holding legitimate personal configuration, and nothing stops a developer recreating it.
- **C** - States a (contested) fact rather than answering "how do we ensure precedence"; it offers no action.

EXAM TIP

Exam Rule: - Of the four options it is the only one that is both truthful about the mechanism and responsive to the question: no per-project flag exists

104 Global architecture rules live in the project root `CLAUDE.md`; directory-specific linting conventions live in subdirectory `CLAUDE.md` files. How does Claude Code apply the hierarchy?

- (A) Subdirectory `CLAUDE.md` overrides project root for files in that directory.
- (B) All `CLAUDE.md` files are merged, applying all rules to all files.
- (C) The closest `CLAUDE.md` to the working file determines rules; no inheritance.
- (D) Only the project root `CLAUDE.md` is applied; subdirectory files are ignored.

Answer: (A) Subdirectory `CLAUDE.md` overrides project root for files in that directory.

EXPLANATION

Both files are in effect, and **specificity resolves conflicts**: the root's architecture rules continue to apply inside the subdirectory, while the subdirectory's linting conventions win wherever the two disagree. That is the inheritance-with-override model, and it is why teams can state a rule once at the root and refine it locally.

Why distractors fail:

- **B** - "All rules to all files" would apply a subdirectory's local conventions to unrelated parts of the repository, defeating the point of scoping them.
- **C** - Correct that the closest file wins on conflicts, wrong that there is **no inheritance**: root rules that the subdirectory does not contradict remain in force.
- **D** - Would make subdirectory files pointless.

EXAM TIP

Exam Rule: - Both files are in effect, and specificity resolves conflicts: the root's architecture rules continue to apply inside the subdirectory, while

105 Architectural rules apply globally; `src/` and `tests/` have directory-specific conventions. Where should each go?

- (A) Global rules in `.claude/rules/` with broad globs; directory rules in subdirectory `CLAUDE.md`.
- (B) Global rules in the project `CLAUDE.md`; directory rules in `src/CLAUDE.md` and `tests/CLAUDE.md`.
- (C) All rules in the project root `CLAUDE.md` with conditional logic per directory.
- (D) A single `CLAUDE.md` with path-checking logic embedded throughout.

Answer: (B) Global rules in the project `CLAUDE.md`; directory rules in `src/CLAUDE.md` and `tests/CLAUDE.md`.

EXPLANATION

Placement mirrors scope. Global rules go in the root file where they apply to everything; directory conventions go in the `CLAUDE.md` **inside the directory they govern**, so they load exactly when work happens there and are obviously owned by that area.

Why distractors fail:

- **A** - Inverted: `.claude/rules/` with globs is the mechanism for *path-specific* rules, while the root `CLAUDE.md` is the natural home for global ones.
- **C** and **D** - Both encode location logic as prose the model must interpret on every file ("if you are editing under `tests/`, then..."). The hierarchy already does this deterministically, and prose conditionals grow unreadable and get missed.

EXAM TIP

Exam Rule: - Placement mirrors scope. Global rules go in the root file where they apply to everything; directory conventions go in the `CLAUDE.md` inside t

106 Shared architectural rules must be followed by all developers and version-controlled with the codebase. Where should they be stored?

- (A) Each developer's `~/ .claude.json` with manual synchronization.
- (B) Team wiki documentation with manual Claude Code configuration per developer.
- (C) A central configuration service distributing rules to all developer instances.
- (D) Project `CLAUDE.md` and `.mcp.json` files committed to repository version control.

Answer: (D) Project `CLAUDE.md` and `.mcp.json` files committed to repository version control.

EXPLANATION

Committing the configuration to the repository satisfies every stated requirement at once: everyone who clones gets it automatically, it is versioned alongside the code it governs, changes go through review, and rules evolve in the same commit as the architecture they describe.

Why distractors fail:

- **A** - Manual synchronisation across developers guarantees drift, and user files are not version-controlled.
- **B** - A wiki documents rules for humans; nothing loads them into Claude's context.
- **C** - Custom distribution infrastructure to replicate what `git clone` already does.

EXAM TIP

Exam Rule: - Committing the configuration to the repository satisfies every stated requirement at once: everyone who clones gets it automatically, it is

107 A legacy codebase has inconsistent patterns; the team wants Claude to learn existing patterns before suggesting improvements. How should existing patterns be provided as context?

- (A) Upload the entire codebase to provide complete pattern examples.
- (B) `CLAUDE.md` with few-shot examples showing key patterns and team conventions.
- (C) Reference the codebase directory location for Claude to explore and learn patterns.
- (D) Iterative pattern learning: Claude identifies patterns through codebase exploration.

Answer: (B) `CLAUDE.md` with few-shot examples showing key patterns and team conventions.

EXPLANATION

The codebase is **inconsistent**, so exploration cannot tell Claude which patterns are the ones to follow - it would find several and no basis for choosing. Curated examples in `CLAUDE.md` make the **intended** conventions explicit, load automatically in every session, and cost a fraction of the tokens of exploration.

Why distractors fail:

- **A** - Not possible within a context window, and it reproduces the inconsistency rather than resolving it.
- **C** and **D** - Both delegate the judgement to exploration. Faced with three competing patterns, Claude may adopt the most common one, which in a legacy codebase is often the one the team wants to retire.

EXAM TIP

Exam Rule: - The codebase is inconsistent, so exploration cannot tell Claude which patterns are the ones to follow - it would find several and no basis f

108 Architectural rules are in the root `CLAUDE.md`; test conventions in `tests/CLAUDE.md`. Claude works on `tests/integration_test.py`. Which rules apply?

- (A) All `CLAUDE.md` files merged, applying all rules simultaneously.
- (B) Only `tests/CLAUDE.md` applies, as the closest configuration.
- (C) Only the root `CLAUDE.md` applies; subdirectory files are ignored.
- (D) Both root and `tests/CLAUDE.md` apply, with subdirectory rules overriding conflicts.

Answer: (D) Both root and `tests/CLAUDE.md` apply, with subdirectory rules overriding conflicts.

EXPLANATION

The precise statement of the model: architectural rules from the root still bind test code, and where the two files disagree the nearer file wins. Both halves matter - inheritance **and** override.

Why distractors fail:

- **A** - "All rules simultaneously" ignores conflict resolution and would also apply **other** subdirectories' rules here.
- **B** - Discards the root's architectural rules, which were never limited to non-test code.
- **C** - Discards the test conventions entirely.

EXAM TIP

Exam Rule: - The precise statement of the model: architectural rules from the root still bind test code, and where the two files disagree the nearer file

109The main branch's CLAUDE.md has production rules; a feature branch's has experimental conventions. How does Claude Code handle branch-specific CLAUDE.md files?

- (A) Uses the CLAUDE.md from the current working branch, respecting branch-specific conventions.
- (B) Always uses the main branch CLAUDE.md regardless of the current branch.
- (C) Reads CLAUDE.md from the repository default branch only.
- (D) Merges main and feature branch CLAUDE.md together.

Answer: (A) Uses the CLAUDE.md from the current working branch, respecting branch-specific conventions.

EXPLANATION

Claude Code reads files **from the working tree**, and the working tree contains whatever the checked-out branch provides. CLAUDE.md is not special-cased - it is versioned like any other file, so branch-specific conventions work naturally and merge into main through the normal review process.

Why distractors fail:

- **B** and **C** - Both assume special git awareness that reads from a branch other than the one checked out. Claude Code has no such mechanism; it sees files on disk.
- **D** - Cross-branch merging of a file is not something the tool does; git merges branches, not Claude Code.

EXAM TIP

Exam Rule: - Claude Code reads files from the working tree, and the working tree contains whatever the checked-out branch provides. CLAUDE.md is not spec

110A new team member's Claude Code is not applying the team's Python standards; the standards live in `~/ .claude/CLAUDE.md` on the senior engineer's machine. Root cause and fix?

- (A) The new member needs to run `/memory reload`.
- (B) User-level `~/ .claude/CLAUDE.md` applies only to that user and is not shared via version control. The standards should be moved to the project-level CLAUDE.md committed to the repository.
- (C) The standards need to be duplicated in both user-level and project-level CLAUDE.md files.
- (D) The new member should manually copy the standards to their own `~/ .claude/CLAUDE.md`.

Answer: (B) User-level `~/ .claude/CLAUDE.md` applies only to that user and is not shared via version control. The standards should be moved to the project-level CLAUDE.md committed to the repository.

EXPLANATION

It identifies both the cause (the rules are in a personal, machine-local file that nobody else has) and the correct fix (team standards belong in the **committed project** file). Moving them means every current and future developer gets them automatically, with no per-person action.

Why distractors fail:

- **A** - Reloading cannot load a file that does not exist on this machine.
- **C** - Duplication creates two sources of truth that will drift, and the user-level copy still needs manual distribution.
- **D** - Fixes one developer today and repeats the problem for every new hire; also unversioned.

EXAM TIP

Exam Rule: - It identifies both the cause (the rules are in a personal, machine-local file that nobody else has) and the correct fix (team standards belong)

111API conventions apply in some sessions but not others on the same project; the developer suspects the wrong memory files are loading. Fastest way to confirm what a session has actually loaded?

- (A) Run `/memory` in the session to list the loaded memory files.
- (B) Run `/compact` to reload the configuration hierarchy from disk.
- (C) Delete `~/ .claude/CLAUDE.md` so only project-level configuration can load.
- (D) Ask Claude in the session to repeat the team's API conventions back.

Answer: (A) Run `/memory` in the session to list the loaded memory files.

EXPLANATION

`/memory` reports exactly this: which memory files the current session has loaded and from where. It is direct evidence rather than inference, and it takes one command.

Why distractors fail:

- **B** - `/compact` summarises conversation history to reclaim context; it neither reloads nor reports configuration.
- **C** - Destructive guesswork: it removes a file that may be innocent, and it does not tell you what *was* loaded in the failing sessions.
- **D** - Asking the model to recite the conventions tests whether it can produce them, not which files are loaded - and a plausible recitation from general knowledge would be actively misleading.

EXAM TIP

Exam Rule: - `/memory` reports exactly this: which memory files the current session has loaded and from where. It is direct evidence rather than inference

112 Every Java file written during refactoring must be formatted with the project's Checkstyle rules before being saved; developers forget to run it manually. Correct hook configuration?

- (A) A PreToolUse hook on Write that runs Checkstyle on the content before the file is written.
- (B) A PostToolUse hook on file write operations that runs the Checkstyle formatter on the written file, automatically correcting any style violations.
- (C) Add "Always run Checkstyle before saving files" to CLAUDE.md.
- (D) A PreToolUse hook on Read that verifies Java files are Checkstyle-compliant before being read into context.

Answer: (B) A PostToolUse hook on file write operations that runs the Checkstyle formatter on the written file, automatically correcting any style violations.

EXPLANATION

Formatters operate on **files**, so the natural point is immediately after the write: the file exists on disk, the formatter runs against it, and violations are corrected automatically. This is the standard lint/format hook pattern, and it fires on every write without depending on anyone remembering.

Why distractors fail:

- **A** - The closest alternative, and workable in principle, but it requires intercepting and rewriting in-flight content rather than running the formatter against a real file - awkward for a tool designed to reformat files in place.
- **C** - CLAUDE.md is guidance; the stem asks for a guarantee, and "developers occasionally forget" is precisely the failure guidance does not fix.
- **D** - Hooks the wrong operation entirely: reading a file has nothing to do with formatting what Claude writes.

EXAM TIP

Exam Rule: - Formatters operate on files, so the natural point is immediately after the write: the file exists on disk, the formatter runs against it, and

113 Project CLAUDE.md says 4-space indentation; an architect's user-level CLAUDE.md says 2-space, and their code has been coming back in 2 spaces and breaking the build. The team needs a **guarantee** of 4-space indentation on every save. What should they do?

- (A) Add a PostToolUse hook that runs the team's formatter after every Write/Edit, so 4-space indentation is enforced regardless of what Claude generates.
- (B) Leave the rule in project-level .claude/CLAUDE.md.
- (C) Ask the architect to delete their user-level ~/ .claude/CLAUDE.md.
- (D) Move the 4-space rule into a CLAUDE.local.md at the project root so it is appended last.

Answer: (A) Add a PostToolUse hook that runs the team's formatter after every Write/Edit, so 4-space indentation is enforced regardless of what Claude generates.

EXPLANATION

"Guarantee" rules out every context-layer answer. A formatter hook makes indentation **independent of what the model generates or which CLAUDE.md files are loaded** - whatever is written gets reformatted deterministically, so the build cannot break on indentation again. It also generalises: the same hook enforces the rest of the team's formatting.

Why distractors fail:

- **B** - Restates the current state, which is producing the failure.
- **C** - Relies on one person's cooperation, removes their legitimate personal settings, and the next contributor reintroduces the problem.
- **D** - Load-order tinkering to win a precedence race; still guidance, still probabilistic, and CLAUDE.local.md is deprecated (GEN3-002).

EXAM TIP

Exam Rule: - "Guarantee" rules out every context-layer answer. A formatter hook makes indentation independent of what the model generates or which CLAUDE

114 A consultancy works across 12 client projects. There are personal developer preferences, firm-wide coding standards, per-client conventions, and subsystem-specific rules. Correct configuration architecture?

- (A) User ~/ .claude/CLAUDE.md for personal preferences; project .claude/CLAUDE.md for firm-wide standards **and** client conventions; directory-level CLAUDE.md / .claude/rules/ for subsystem rules.
- (B) User ~/ .claude/CLAUDE.md for personal preferences **and firm-wide standards**; project .claude/CLAUDE.md for client-specific conventions; directory-level CLAUDE.md / .claude/rules/ for subsystem-specific rules.
- (C) A single CLAUDE.md at each project root containing all four levels with clear section headings.
- (D) User ~/ .claude/CLAUDE.md for everything, using @ path imports to pull in project-specific files.

Answer: (B) User ~/ .claude/CLAUDE.md for personal preferences **and firm-wide standards**; project .claude/CLAUDE.md for client-specific conventions; directory-level CLAUDE.md / .claude/rules/ for subsystem-specific rules.

EXPLANATION

Each rule sits at the layer matching its **scope**. Firm-wide standards apply to all 12 projects for a given consultant, so the user level holds them once instead of duplicating them into twelve repositories that would then drift. Client conventions are per-repository, so they belong in that project's committed file. Subsystem rules are path-scoped, so they go directory-

level. Four scopes, four layers, no duplication.

Why distractors fail:

- **A** - Differs from B only in placing firm-wide standards in each project file - which means maintaining twelve copies and updating all of them whenever a firm standard changes. That single difference is the item's discriminator.
- **C** - Collapses four scopes into one file per project: personal preferences end up committed to client repositories, and firm standards are duplicated twelve times.
- **D** - Puts client-specific configuration in a personal file, so nothing is shared with the client team or version-controlled with their code.

EXAM TIP

Exam Rule: - Each rule sits at the layer matching its scope. Firm-wide standards apply to all 12 projects for a given consultant, so the user level holds

3.2 Custom Slash Commands, Skills & Plugins

Claude CCAF: Claude Code Configuration & Workflows - 4 questions

115A custom Agent Skill with context: fork generates verbose output while the main session context stays clean. When should context: fork be used?

- (A) Forking creates overhead and should be avoided unless absolutely necessary.
- (B) context: fork is required for all custom skills to maintain context cleanliness.
- (C) Use context: fork when a skill generates verbose output that would pollute the main context.
- (D) context: fork only for skills invoking external tools or MCP servers.

Answer: (C) Use context: fork when a skill generates verbose output that would pollute the main context.

EXPLANATION

context: fork runs the skill in an isolated sub-agent context so thousands of lines of intermediate logs/search results do not pollute the parent session window.

EXAM TIP

Verbose skill output = context: fork in SKILL.md frontmatter.

116Invoking /migrate users_table - which substitution variables inside SKILL.md can access the argument?

- (A) process.argv[2] contains it, passed through to the shell.
- (B) \$PARAMS[migration_name] based on the argument-hint definition.
- (C) \${input} contains it and arguments must be parsed manually.
- (D) \$ARGUMENTS contains users_table and \$1 contains it as the first positional argument.

Answer: (D) \$ARGUMENTS contains users_table and \$1 contains it as the first positional argument.

EXPLANATION

Slash commands and skills expose \$ARGUMENTS (the complete argument string) and positional variables (\$1, \$2, etc.).

EXAM TIP

Slash command argument variables: \$ARGUMENTS (full string) and \$1, \$2 (positional).

117Which element of a Skill definition determines whether Claude invokes it for a given request?

- (A) The Skill's filename in .claude/skills.
- (B) The list of allowed_tools in the manifest.
- (C) The description field.
- (D) The Skill's load order in the skills directory.

Answer: (C) The description field.

EXPLANATION

The skill's description field in YAML frontmatter is the semantic matching trigger Claude evaluates against the user prompt.

EXAM TIP

Skill invocation matching is determined solely by the description frontmatter field.

118Does invoking a slash command count as a "turn" in the session?

- (A) No - they bypass per-turn limits as pre-approved built-in shortcuts.
- (B) No - they run on a separate channel and do not consume context.
- (C) They count as half a turn for context budgeting.
- (D) Yes - the resolved prompt is sent to Claude as a normal user turn and counts toward context and any per-turn limits.

Answer: (D) Yes - the resolved prompt is sent to Claude as a normal user turn and counts toward context and any per-turn limits.

EXPLANATION

Slash commands are expanded client-side into prompt text and sent as standard user turns, consuming tokens and turn counts identically to manual prompts.

EXAM TIP

Slash commands expand into standard user turns and count toward context tokens and turn limits.

3.3 Path-Specific Rules & Scoping

Claude CCAF: Claude Code Configuration & Workflows - 2 questions

119 Terraform conventions should load only when editing .tf files, not for all files. How do you configure this?

- (A) Create a /terraform slash command that loads the conventions on demand.
- (B) Create a .claude/rules/ file with YAML frontmatter paths: ['terraform/**/*.tf'] so the rules only activate when editing matching files.
- (C) Add Terraform conventions to the root CLAUDE.md under a # Terraform section header.
- (D) Create a CLAUDE.md file in the terraform/ directory.

Answer: (B) Create a .claude/rules/ file with YAML frontmatter paths: ['terraform/**/*.tf'] so the rules only activate when editing matching files.

EXPLANATION

.claude/rules/*.md files with YAML frontmatter paths: ['glob_pattern'] activate conditionally when files matching the pattern are edited.

EXAM TIP

Conditionally loaded rules for file types/paths = .claude/rules/ with paths frontmatter globs.

120 A project uses JavaScript (Prettier) and Rust (rustfmt). How should CLAUDE.md handle this dual-language setup?

- (A) Rely on the language-specific formatters and don't include formatting rules in CLAUDE.md.
- (B) Use .claude/rules/ with glob-based rules: one file with **/*.js, **/*.ts for Prettier rules, another with **/*.rs for rustfmt rules.
- (C) Create separate CLAUDE.md files in js/ and rust/ directories.
- (D) Include both conventions in the root CLAUDE.md with clear section headers.

Answer: (B) Use .claude/rules/ with glob-based rules: one file with **/*.js, **/*.ts for Prettier rules, another with **/*.rs for rustfmt rules.

EXPLANATION

Glob-based rules in .claude/rules/ decouple conventions from directory structure, loading Prettier rules for JS/TS and rustfmt for Rust automatically.

EXAM TIP

Dual-language repos = .claude/rules/ with language-specific path globs.

3.4 Plan Mode vs Direct Execution & Model Tiering

Claude CCAF: Claude Code Configuration & Workflows - 2 questions

121 A developer wants Claude Code to choose plan mode vs. direct execution automatically based on task complexity. Can it?

- (A) Claude Code defaults to plan mode for all tasks unless overridden.
- (B) No, plan mode versus direct execution must be explicitly specified by the developer.
- (C) Yes, Claude Code analyzes task complexity and selects the mode automatically.
- (D) Mode selection based on file count: plan for multiple files, direct for single.

Answer: (B) No, plan mode versus direct execution must be explicitly specified by the developer.

EXPLANATION

Plan mode is an explicit developer choice (via Shift+Tab / plan toggle / CLI). Claude Code does not automatically toggle plan mode based on complexity.

EXAM TIP

Plan Mode is an explicit developer toggle; it is never auto-switched by Claude.

122 A team does high-volume routine work (boilerplate, formatting) and lower-volume complex architectural refactoring. Correct model configuration?

- (A) Haiku for all tasks.
- (B) Opus for all tasks.
- (C) Sonnet as default, with Opus available for complex architectural tasks.
- (D) Haiku for routine tasks, Sonnet for complex tasks, with Opus disabled.

Answer: (C) Sonnet as default, with Opus available for complex architectural tasks.

EXPLANATION

Sonnet provides the ideal speed/cost/intelligence balance as default, with Opus reserved for deep architectural reasoning.

EXAM TIP

Model tiering: Sonnet default for balanced development; Opus for deep architecture.

3.5 Iterative Refinement & Multi-Pass Review

Claude CCAF: Claude Code Configuration & Workflows - 2 questions

123A 14-file PR reviewed in a single pass yields contradictory findings - file A's refactor breaks assumptions made while reviewing file B. How do you improve the review architecture?

- (A) Sequential file-by-file review building cumulative understanding.
- (B) Single comprehensive review with extended thinking time.
- (C) Increase context window size for complete simultaneous review.
- (D) Multi-pass review: per-file local analysis passes plus a cross-file integration pass.

Answer: (D) Multi-pass review: per-file local analysis passes plus a cross-file integration pass.

EXPLANATION

Multi-pass review separates concerns: per-file local passes ensure deep file-level analysis without dilution, while a final integration pass reconciles cross-file data flows.

EXAM TIP

Large PR review = Local per-file passes + Cross-file integration pass.

124 Code review in CI/CD; the review instance must not have access to the generation reasoning. How do you ensure independence?

- (A) Same instance with extended thinking time for critical review mode.
- (B) Review-specific system prompt overriding generation context with critical evaluation.
- (C) Clear conversation history before review, removing generation context.
- (D) A separate independent Claude instance for review, without generation reasoning context.

Answer: (D) A separate independent Claude instance for review, without generation reasoning context.

EXPLANATION

Evaluator independence requires structural separation: a fresh Claude instance evaluating code without the generator's confirmation-biased reasoning trace.

EXAM TIP

Independent code review = Separate Claude instance without generation reasoning context.

3.6 Claude Code in CI/CD & Enterprise Deployment

Claude CCAF: Claude Code Configuration & Workflows - 2 questions

125A CI script runs `claude 'Analyze this PR for security issues'` and hangs indefinitely waiting for interactive input. Correct fix?

- (A) Add the `-p` flag: `claude -p 'Analyze this PR for security issues'`
- (B) Add a `--batch` flag.
- (C) Redirect stdin: `... < /dev/null`.
- (D) Set `CLAUDE_HEADLESS=true` before running.

Answer: (A) Add the `-p` flag: `claude -p 'Analyze this PR for security issues'`

EXPLANATION

`-p` (print / non-interactive mode) runs Claude Code headlessly, outputting results directly to stdout and exiting without prompting for interactive input.

EXAM TIP

Unattended CI execution = `claude -p 'prompt'`.

126 CI runs behave inconsistently versus local runs because developers have different `CLAUDE.md` files, skills, and MCP servers. What ensures consistent CI execution?

- (A) Using the `--bare` flag, which skips auto-discovery of all local configuration (hooks, skills, plugins, MCP, `CLAUDE.md`).
- (B) Setting a `CLAUDE_CI=true` environment variable.
- (C) Storing all developer configurations in version control and loading them in CI.
- (D) Running CI jobs on standardized Docker containers with identical configurations.

Answer: (A) Using the `--bare` flag, which skips auto-discovery of all local configuration (hooks, skills, plugins, MCP, `CLAUDE.md`).

EXPLANATION

`--bare` skips auto-discovery of local hooks, skills, and memory files, providing an isolated, deterministic execution baseline in CI.

EXAM TIP

Deterministic CI baseline = --bare flag to disable local auto-discovery.

3.7 Prompt Caching, Message Batches API & High Availability

Claude CCAF: Claude Code Configuration & Workflows - 4 questions

127What is the structural requirement for an exact cache hit when using Anthropic's Prompt Caching mechanism?

- (A) The system prompt must change completely on every single API request turn.
- (B) The prompt prefix (from the beginning of the prompt up to the cache breakpoint marker) must be structurally and textually identical to a previously cached sequence.
- (C) The user must be connecting from the exact same IP address.
- (D) The max_tokens parameter must be set to an odd number.

Answer: (B) The prompt prefix (from the beginning of the prompt up to the cache breakpoint marker) must be structurally and textually identical to a previously cached sequence.

EXPLANATION

Prompt caching requires an exact character-for-character prefix match starting from token 0 up to the cache breakpoint marker. Any dynamic variable placed early invalidates downstream caching.

EXAM TIP

Cache Hit Requirement = 100% identical prompt prefix from index 0 to cache control breakpoint.

128You place the current_timestamp and user_session_id at the very beginning of the system prompt, followed by a 40,000-token corporate policy document. What is the impact on cache performance?

- (A) Cache hit rate will be 100% because the massive document is included.
- (B) Cache hit rate will drop to 0% because the dynamic variables at the beginning change every time, invalidating the exact prefix match required for the subsequent static text.
- (C) The API will automatically move the dynamic variables to the end of the prompt.
- (D) The cost will be reduced by 90% regardless of the order.

Answer: (B) Cache hit rate will drop to 0% because the dynamic variables at the beginning change every time, invalidating the exact prefix match required for the subsequent static text.

EXPLANATION

Placing dynamic, frequently changing variables (timestamps, session IDs) at the start of a prompt breaks the static prefix match, dropping the cache hit rate to zero.

EXAM TIP

Dynamic variables at start of prompt = 0% cache hit rate. Keep static text at the prefix, dynamic data at the tail.

129What is the minimum token threshold required to trigger or construct a cache milestone for Claude 3.5 Sonnet prompts?

- (A) 10 tokens
- (B) 1,024 tokens (Standard minimum for caching sequences)
- (C) 100,000 tokens
- (D) 50,000 tokens

Answer: (B) 1,024 tokens (Standard minimum for caching sequences)

EXPLANATION

The minimum prompt length required to establish a cache breakpoint on Claude 3.5 Sonnet is 1,024 tokens.

EXAM TIP

Claude 3.5 Sonnet Prompt Caching minimum threshold = 1,024 tokens.

130Your application experiences a sudden surge in traffic, resulting in HTTP 429 (Too Many Requests) rate limit errors from the Anthropic API. What is the correct architectural pattern to manage this gracefully?

- (A) Immediately retry the request in an unconstrained loop as fast as possible.
- (B) Implement an exponential backoff retry strategy with jitter in the client gateway orchestrator.
- (C) Crash the application server and force a hard reboot.
- (D) Lower the API key security permission level.

Answer: (B) Implement an exponential backoff retry strategy with jitter in the client gateway orchestrator.

EXPLANATION

HTTP 429 rate limit exceptions require exponential backoff with randomized jitter to prevent thundering herd retries on API gateway pathways.

EXAM TIP

HTTP 429 Rate Limits = Exponential backoff with randomized jitter.

5.1 Context Dynamics, Compaction & Prompt Caching

Claude CCAF: Context Management & Reliability - 22 questions

131 An agent loop appends tool results to history; after 20 iterations the context window approaches its limit. How do you manage conversation history in long-running loops?

- (A) Increase context window size to accommodate all iterations without truncation.
- (B) Clear conversation history periodically, restarting the agent with fresh context.
- (C) Implement a sliding window maintaining context relevance, summarizing older content.
- (D) Truncate the oldest tool results, keeping only recent results.

Answer: (C) Implement a sliding window maintaining context relevance, summarizing older content.

EXPLANATION

A sliding window **with summarisation** bounds growth while preserving the causal chain: recent turns stay verbatim (where detail matters most), older material is condensed into what still bears on the remaining work, and nothing is silently dropped. This is the pattern behind both server-side compaction and /compact.

Why distractors fail:

- **A** - Buys headroom without changing the growth rate; any unbounded loop fills any window.
- **B** - Clearing discards the task definition and everything learned, so the agent restarts from nothing.
- **D** - Truncating the oldest results removes the earliest turns, which typically hold the goal and constraints - the highest-value tokens present.

EXAM TIP

Exam Rule: - A sliding window with summarisation bounds growth while preserving the causal chain: recent turns stay verbatim (where detail matters most),

132 Long-document extraction shows the "lost in the middle" effect - middle content is missed despite being within the context window. How do you address attention dilution?

- (A) Use extended thinking, giving the model more time for complete analysis.
- (B) Explicit instructions directing attention to middle sections.
- (C) Increase context window size, reducing the "middle" proportion of the document.
- (D) Split the document into focused chunks, processing each independently with overlap.

Answer: (D) Split the document into focused chunks, processing each independently with overlap.

EXPLANATION

Chunking **eliminates the middle**: in a small enough chunk, every position is effectively a high-attention position. The overlap matters too - it ensures information spanning a boundary appears intact in at least one chunk, so nothing is lost at the seams.

Why distractors fail:

- **C** - The stem already says the content is *within* the window; a larger window leaves the same proportion in the middle, just of a bigger document.
- **A** - Thinking time does not change where attention falls across a long input.
- **B** - "Pay attention to the middle" is an instruction against an architectural effect; it cannot reliably override it.

EXAM TIP

Exam Rule: - Chunking eliminates the middle: in a small enough chunk, every position is effectively a high-attention position. The overlap matters too -

133 A 200-page document pipeline frequently misses middle sections. Address attention dilution?

- (A) Extended thinking time allowing complete document analysis.
- (B) Explicit instructions directing attention to middle sections.
- (C) Increase context window to maximum, reducing the "middle" proportion.
- (D) Split the document into overlapping chunks, processing each independently.

Answer: (D) Split the document into overlapping chunks, processing each independently.

EXPLANATION

Same remedy as UD5-004, and at 200 pages the case is stronger still: the "middle" is enormous, so only chunking gives every section a position where it will actually be attended to.

Why distractors fail:

- **C** - Capacity was never the constraint.
- **A** and **B** - Neither redistributes attention across a long input.

EXAM TIP

Exam Rule: - Same remedy as UD5-004, and at 200 pages the case is stronger still: the "middle" is enormous, so only chunking gives every section a position

134A 47-turn support conversation approaches the context limit; earlier turns contain critical account details needed for the current issue. How do you preserve critical information?

- (A) Truncate oldest messages, keeping only the most recent.
- (B) Increase context window to maximum, accommodating the complete history.
- (C) Sliding window approach with structured summarization preserving critical details.
- (D) Restart the conversation with fresh context when approaching the limit.

Answer: (C) Sliding window approach with structured summarization preserving critical details.

EXPLANATION

The requirement is explicitly to **preserve the critical details**, and structured summarisation is what distinguishes this from ordinary compaction: account numbers, balances, and decisions are lifted into a durable structured form, while the conversational padding around them is condensed.

Why distractors fail:

- **A** - The oldest messages are precisely where the critical account details live.
- **D** - Restarting discards them entirely, and the customer must repeat everything.
- **B** - Delays the limit without preserving anything, and the conversation will reach it again.

EXAM TIP

Exam Rule: - The requirement is explicitly to preserve the critical details, and structured summarisation is what distinguishes this from ordinary compac

135After 45 iterations the conversation is large and latency has increased. How do you manage conversation growth in long-running loops?

- (A) Increase context window to maximum size.
- (B) A fixed iteration maximum preventing excessive conversation length.
- (C) Clear conversation history every 15 iterations.
- (D) Sliding window approach maintaining relevant context with summarization of older content.

Answer: (D) Sliding window approach maintaining relevant context with summarization of older content.

EXPLANATION

It addresses both stated symptoms: bounded context keeps **size** in check, and fewer input tokens per request directly reduces the **latency** the stem reports. Relevance-based retention means the reduction costs little in capability.

Why distractors fail:

- **A** - A larger window makes each request slower, worsening the latency complaint.
- **B** - Caps the work rather than managing the context; a legitimate 60-iteration task is truncated.
- **C** - Periodic clearing loses accumulated findings every 15 iterations, so the agent repeatedly re-derives what it knew.

EXAM TIP

Exam Rule: - It addresses both stated symptoms: bounded context keeps size in check, and fewer input tokens per request directly reduces the latency the

136A 180-page extraction pipeline frequently misses middle sections. Which architectural approach addresses attention dilution?

- (A) Extended thinking for thorough processing.
- (B) Explicit instructions to pay close attention to middle sections.
- (C) Increase the context window to maximum.
- (D) Split the document into overlapping chunks, processing each independently before aggregating results.

Answer: (D) Split the document into overlapping chunks, processing each independently before aggregating results.

EXPLANATION

Chunk with overlap, then aggregate - the standard architecture for long-document extraction, and the only option that changes where content sits relative to attention.

Why distractors fail:

- **A, B, C** - None redistributes attention; C in particular treats a capacity symptom that is not the cause.

EXAM TIP

Exam Rule: - Chunk with overlap, then aggregate - the standard architecture for long-document extraction, and the only option that changes where content

137 Web-search output (85K tokens **including page content**) plus document-analysis output (70K **including chains of thought**) totals 155K, but the synthesis agent performs best under 50K. Most effective solution?

- (A) Have the synthesis agent process findings in sequential batches, maintaining state between calls.
- (B) Modify upstream agents to return structured data (key facts, quotes, relevance scores) instead of verbose content and reasoning.
- (C) Store findings in a vector database and give synthesis search tools to query during its work.
- (D) Add an intermediate summarization agent that condenses findings before synthesis.

Answer: (B) Modify upstream agents to return structured data (key facts, quotes, relevance scores) instead of verbose content and reasoning.

EXPLANATION

The stem tells you what the bulk ***is***: raw page content and reasoning traces - material the synthesis agent does not need. **Fixing the contract at the source** eliminates the tokens rather than compressing them later, and structured output (facts, quotes, scores) is more useful to synthesis than prose ever was. It also costs nothing extra: no additional agent, no additional call.

Why distractors fail:

- **D** - The closest alternative, and it works - but it adds a whole agent and a pass over 155K tokens to discard material that should never have been produced.
- **A** - Batching means synthesis never sees all findings together, which is exactly what synthesis requires.
- **C** - Retrieval is lossy and non-deterministic for a task that needs ***all*** findings; a missed query silently drops a source.

EXAM TIP

Exam Rule: - The stem tells you what the bulk ***is***: raw page content and reasoning traces - material the synthesis agent does not need. Fixing the contra

138 An MCP database tool returns 500 rows as 200K tokens. Following MCP design best practices, what should the tool return?

- (A) The full 200K token result so Claude has complete information.
- (B) Only the first row as a sample with total row count.
- (C) A compressed binary representation.
- (D) A summary with row count, column names, first 10 rows as examples, and a pagination token for fetching more rows on demand.

Answer: (D) A summary with row count, column names, first 10 rows as examples, and a pagination token for fetching more rows on demand.

EXPLANATION

It gives the model everything needed to **decide what to do next** - how many rows matched, what the shape of the data is, what the values look like - at a tiny fraction of the tokens, plus a pagination token so the full data remains reachable on demand. Nothing is lost; it is deferred.

Why distractors fail:

- **A** - 200K tokens for one tool call would consume most of the window and crowd out the task.
- **B** - One row and a count is too little: no column names, no sense of the data's variety, and no way to fetch more.
- **C** - Compression does not reduce token count once decoded, and the model cannot read a binary blob.

EXAM TIP

Exam Rule: - It gives the model everything needed to decide what to do next - how many rows matched, what the shape of the data is, what the values look

139 80,000 documents each needing one classification call; results are needed for a report generated **every Friday**. Most cost-effective approach?

- (A) Real-time API calls with prompt caching.
- (B) Message Batches API submitted Thursday with results available within 24 hours for Friday's report.
- (C) Real-time API calls processed sequentially throughout the week.
- (D) Multiple small batch jobs submitted daily throughout the week.

Answer: (B) Message Batches API submitted Thursday with results available within 24 hours for Friday's report.

EXPLANATION

The deadline is **weekly**, so a 24-hour processing window is comfortably within tolerance - which makes this the textbook Batch API case and captures the ~50% discount on all 80,000 calls. Submitting Thursday for Friday leaves the full window available.

Why distractors fail:

- **A** - Caching helps with repeated prefixes but does not approach the batch discount, and real-time pricing applies to every call.
- **C** - Full price, plus a week of sequential processing.
- **D** - Also uses the Batch API, so it captures the discount, but daily submission adds orchestration and tracking for no benefit when nothing is needed before Friday.

EXAM TIP

Exam Rule: - The deadline is weekly, so a 24-hour processing window is comfortably within tolerance - which makes this the textbook Batch API case and ca

140 60% of cost is input tokens (mostly repeated context), 15% output, 25% thinking. Which optimization provides the biggest reduction?

- (A) Reduce output length with `max_tokens` to save on the 15%.
- (B) Switch to a smaller model with lower per-token costs.
- (C) Reduce thinking budget to save on the 25%.
- (D) Implement prompt caching to save on the 60%.

Answer: (D) Implement prompt caching to save on the 60%.

EXPLANATION

Two facts point the same way: input is the **largest** cost category, and the content is **repeated** - which is precisely the condition under which caching pays, since cache reads are billed at a large discount. Attacking the biggest line item with the mechanism designed for its exact shape gives the largest saving with no capability loss.

Why distractors fail:

- **A** - Optimises the smallest category, and truncating output degrades results.
- **C** - Cuts reasoning quality on a quarter of spend when the larger, cost-free saving is available.
- **B** - Reduces every rate but lowers capability across the board; a capability decision should not be driven by a cost problem that caching solves.

EXAM TIP

Exam Rule: - Two facts point the same way: input is the largest cost category, and the content is repeated - which is precisely the condition under which

141 A 48-turn session covering a refund inquiry, subscription question, and payment update is approaching context limits. Optimal **compression strategy**?

- (A) Switch to a model with a larger context window.
- (B) Summarise the entire session into a single paragraph and continue.
- (C) Summarise earlier, **resolved** issues into a narrative description while preserving full verbatim history only for the **active, unresolved** issue.
- (D) Delete the oldest 20 turns from context.

Answer: (C) Summarise earlier, **resolved** issues into a narrative description while preserving full verbatim history only for the **active, unresolved** issue.

EXPLANATION

It compresses **by resolution status**, which is the right axis: resolved issues need only their outcome recorded, while the active issue needs full detail because that is where the next turn's reasoning happens. Maximum compression where information has served its purpose, zero compression where it is still in use.

Why distractors fail:

- **B** - A single paragraph for three issues loses the active issue's detail, which is the part that must be preserved.
- **D** - Deletes by position rather than relevance; the oldest 20 turns may contain the active issue's origin.
- **A** - Not a compression strategy, and it postpones rather than solves.

EXAM TIP

Exam Rule: - It compresses **by resolution status**, which is the right axis: resolved issues need only their outcome recorded, while the active issue needs

142 A "turn" in the Agent SDK makes multiple Claude API calls with tool use. Which metric tracks how many iterations the agent performed?

- (A) The number of `tool_use` blocks generated across all responses.
- (B) Total output tokens divided by average tokens per turn.
- (C) Number of API calls multiplied by input tokens.
- (D) The `usage.iterations` field, counting agent turns.

Answer: (D) The `usage.iterations` field, counting agent turns.

EXPLANATION

A reported field is authoritative; a derived estimate is not. Reading the count directly avoids every assumption the other options require.

Why distractors fail:

- **A** - Counts `*tool calls*`, not iterations: one iteration may emit several parallel `tool_use` blocks, and a final iteration emits none.
- **B** - Division by an "average" produces an estimate that is wrong whenever turn lengths vary, which they always do.
- **C** - Multiplying calls by input tokens yields a quantity with no meaningful units and no relation to iteration count.

EXAM TIP

Exam Rule: - A reported field is authoritative; a derived estimate is not. Reading the count directly avoids every assumption the other options require.

143 A long-running agent has a stable system prompt and first few turns, with frequently-changing recent messages. Where should explicit cache breakpoints go?

- (A) On every message to cache as much as possible.
- (B) On the most recent messages to cache the newest content.
- (C) On the last stable content block before the frequently changing messages, with up to 4 explicit breakpoints on stable blocks.
- (D) Only on the system prompt, since tool definitions shouldn't be cached.

Answer: (C) On the last stable content block before the frequently changing messages, with up to 4 explicit breakpoints on stable blocks.

EXPLANATION

Caching is **prefix-based**, so a breakpoint is only useful where everything before it is byte-identical across requests. Placing it at the boundary between the stable region and the volatile region maximises the cached prefix; the 4-breakpoint limit is the documented maximum.

Why distractors fail:

- **B** - Caches content that changes next request, so it is never read back.
- **A** - Exceeds the breakpoint limit, and breakpoints inside the volatile region are wasted.
- **D** - Tool definitions render **first** in the prefix and are usually the most stable, largest block - excluding them forfeits the biggest saving.

EXAM TIP

Exam Rule: - Caching is prefix-based, so a breakpoint is only useful where everything before it is byte-identical across requests. Placing it at the boundary between the stable region and the volatile region maximises the cached prefix; the 4-breakpoint limit is the documented maximum.

144 The synthesis agent receives 12 research summaries; findings from summaries 4-8 are systematically omitted. Likely cause and fix?

- (A) The synthesis agent needs more detailed instructions about which summaries to include.
- (B) The "lost in the middle" effect. Place key findings at the beginning of aggregated inputs and use explicit section headers throughout.
- (C) The context window is too small; increase `max_tokens`.
- (D) Summaries 4-8 contain contradictory information the model excludes.

Answer: (B) The "lost in the middle" effect. Place key findings at the beginning of aggregated inputs and use explicit section headers throughout.

EXPLANATION

The pattern is diagnostic: the omissions are **positional** (a contiguous middle block), not content-based. The fix is structural - a key-findings digest at the top puts the important material in a high-salience position, and section headers give the model navigational anchors into the body.

Why distractors fail:

- **C** - `max_tokens` limits **output**, not input capacity, and nothing suggests truncation.
- **D** - Would produce scattered omissions correlated with content, not a clean 4-8 block.
- **A** - Instructions cannot redirect attention across a long input (UD5-004).

EXAM TIP

Exam Rule: - The pattern is diagnostic: the omissions are positional (a contiguous middle block), not content-based. The fix is structural - a key-findings digest at the top puts the important material in a high-salience position, and section headers give the model navigational anchors into the body.

145 Combining prompt caching with compaction: after compaction creates a summary block, subsequent requests show **no cache hits** even though the system prompt is unchanged. Explanation?

- (A) Cache TTL expired during compaction processing.
- (B) The compaction block itself needs a `cache_control` breakpoint to establish a new cache entry subsequent requests can match against.
- (C) Compaction is incompatible with prompt caching.
- (D) The compaction block is too large for the minimum cacheable threshold.

Answer: (B) The compaction block itself needs a `cache_control` breakpoint to establish a new cache entry subsequent requests can match against.

EXPLANATION

Compaction **rewrites the message history**, so the prefix that follows the system prompt is entirely new - and because caching matches on a contiguous prefix, the old cache entry no longer applies from that point on. The fix is to mark a new breakpoint after the compaction block so a fresh cache entry is established for the new prefix.

Why distractors fail:

- **C** - They compose fine; the interaction just has to be handled.
- **D** - Inverts the threshold: it is a **minimum** size, and being large helps rather than hinders.
- **A** - TTL expiry would be time-dependent and intermittent, not consistently correlated with compaction.

EXAM TIP

Exam Rule: - Compaction rewrites the message history, so the prefix that follows the system prompt is entirely new - and because caching matches on a contiguous prefix, the old cache entry no longer applies from that point on. The fix is to mark a new breakpoint after the compaction block so a fresh cache entry is established for the new prefix.

146With server-side compaction, an architect wants to inject additional context (e.g. updated tool results) **after** compaction but **before** the conversation continues. Which configuration enables this?

- (A) Set instructions in the compaction config to include the additional context in the summary.
- (B) Append additional context as a system message after receiving the compaction block.
- (C) Use a PreCompact hook to inject context before compaction occurs.
- (D) Configure `pause_after_compaction: true`, which returns `stop_reason: 'compaction'`, allowing context injection before continuing.

Answer: (D) Configure `pause_after_compaction: true`, which returns `stop_reason: 'compaction'`, allowing context injection before continuing.

EXPLANATION

It creates the **control point** the requirement needs: compaction completes, the request returns with a distinguishable stop reason, and your code injects whatever it wants before issuing the next request. Timing is exactly "after compaction, before continuation".

Why distractors fail:

- **C** - PreCompact runs **before** compaction, so injected content would itself be summarised away - the wrong side of the boundary.
- **A** - Compaction instructions shape how existing history is summarised; they cannot introduce new tool results.
- **B** - Appending after the fact without pausing means the model has already continued; the injection misses the turn it was meant to inform.

EXAM TIP

Exam Rule: - It creates the control point the requirement needs: compaction completes, the request returns with a distinguishable stop reason, and your c

147Adding images alongside thinking mode fills the context window more quickly. What explains the interaction?

- (A) Each image consumes fixed tokens regardless of thinking mode, limiting remaining capacity.
- (B) Images are not compatible with extended thinking and should be processed separately.
- (C) All tokens including thinking tokens count toward the context window limit, so images plus thinking traces consume context faster than either alone.
- (D) Images disable thinking mode automatically.

Answer: (C) All tokens including thinking tokens count toward the context window limit, so images plus thinking traces consume context faster than either alone.

EXPLANATION

The window is a **single shared budget**: image tokens, input text, output, and thinking tokens all draw on it. The two additions are therefore cumulative, which is why the combination fills the window faster than either alone - exactly the "interaction" the question asks about.

Why distractors fail:

- **A** - True as far as it goes (images do have a token cost) but it says nothing about thinking, so it does not explain the interaction.
- **B** - False: images and extended thinking work together.
- **D** - Invented behaviour; images do not disable thinking.

EXAM TIP

Exam Rule: - The window is a single shared budget: image tokens, input text, output, and thinking tokens all draw on it. The two additions are therefore

148An agentic loop encounters `stop_reason: 'max_tokens'` after requesting a complex analysis. What does it indicate and what should be done?

- (A) Claude's response was truncated due to the token limit.
- (B) The context window is full.
- (C) Claude wants to call a tool but the response was cut off.
- (D) Claude has finished its analysis.

Answer: (A) Claude's response was truncated due to the token limit.

EXPLANATION

`max_tokens` means generation stopped at the **output limit**, so the response is incomplete. The remedy is to raise `max_tokens` (using streaming for large values), or to break the analysis into smaller requests - and critically, **not** to treat the partial output as a finished answer.

Why distractors fail:

- **B** - A full context window produces a different error at request time, not this stop reason.
- **D** - `end_turn` signals completion; `max_tokens` is the opposite.
- **C** - Speculates about intent the stop reason does not convey; a truncated turn may or may not have been heading toward a tool call.

EXAM TIP

Exam Rule: - `max_tokens` means generation stopped at the output limit, so the response is incomplete. The remedy is to raise `max_tokens` (using streami

149 8K system prompt, 5 tool definitions at 2K each, user messages averaging 500 tokens, responses up to 4K, on a 200K model. Approximately how many turns before hitting the limit?

(A) About 180 turns - **B.** About 20 turns - **C.** About 40 turns - **D.** Unlimited, because old messages are truncated automatically

Answer: (A) About 180 turns - **B.** About 20 turns - **C.** About 40 turns - **D.** Unlimited, because old messages are truncated automatically

EXPLANATION

The arithmetic:

| Component | Tokens |

|---|---|

| System prompt | 8,000 |

| 5 tools x 2,000 | 10,000 |

| **Fixed overhead** | **18,000** |

| Remaining budget | 200,000 - 18,000 = **182,000** |

| Per turn (500 user + 4,000 response) | **4,500** |

| Turns | 182,000 / 4,500 = **40** |

The key insight is that the fixed overhead is paid **once**, while each turn's tokens **accumulate** in history and are re-sent on every subsequent request.

Why distractors fail:

- **A** - ~180 would follow from counting only the 500-token user messages, ignoring the 4K responses that dominate.
- **B** - ~20 would follow from charging the 18K overhead per turn rather than once.
- **D** - Nothing truncates automatically; exceeding the window is an error unless you implement compaction.

EXAM TIP

Exam Rule: - The arithmetic: | Component | Tokens | |---|---| | System prompt | 8,000 | | 5 tools x 2,000 | 10,000 | | **Fixed overhead** | **18,000** | | **Remain**

150 After compaction, specific numerical values from early in the conversation (balances, dates, transaction IDs) are lost in the summary. What design pattern prevents this?

- (A) Increase the compaction trigger threshold to delay summarization.
(B) Disable compaction and rely on larger context windows.
(C) Custom compaction instructions specifically requesting preservation of numerical values.
(D) Extract critical transactional facts into a **persistent structured block** that survives compaction.

Answer: (D) Extract critical transactional facts into a **persistent structured block** that survives compaction.

EXPLANATION

Summarisation is inherently lossy on specifics, so the durable fix is to move those facts **out of the summarisable region entirely**. A persistent structured block is carried forward verbatim through every compaction cycle, so a transaction ID recorded at turn 3 is still exact at turn 300.

Why distractors fail:

- **C** - The plausible runner-up, and worth doing - but instructions to a summariser are probabilistic, so values will still be lost sometimes. "Prevents" requires the structural guarantee.
- **A** - Delays the loss to a later compaction rather than preventing it.
- **B** - Removes the mechanism that keeps long sessions viable, and hits the same wall later.

EXAM TIP

Exam Rule: - Summarisation is inherently lossy on specifics, so the durable fix is to move those facts out of the summarisable region entirely. A persist

151 Documenting a 500,000-line codebase by summarising each module and discarding its source. After 20 modules, docs for module 21 reference class names from module 3 that were **renamed in module 12**. What caused this?

- (A) Progressive summarisation lost the rename detail: when module 12's summary replaced its source, the rename was condensed to "various refactoring changes", so the agent retained module 3's original class names from its earlier summary.
(B) The context window overflowed, causing hallucinated class names.
(C) Claude Code cached module 3's source and served the stale version instead of the updated file.
(D) The model confused similar class names because too many modules were processed in one session.

Answer: (A) Progressive summarisation lost the rename detail: when module 12's summary replaced its source, the rename was condensed to "various refactoring changes", so the agent retained module 3's original class names from its earlier summary.

EXPLANATION

It traces the exact failure path. Module 3's summary - recorded before the rename - carried the old names. Module 12's summary was *supposed* to record the rename, but summarisation compressed away the specific old->new mapping. The agent is now working from two summaries, one stale and one that omitted the correction, and has no source to fall back on because it was discarded. This is UD5-032's lesson applied to a real workflow: **cross-cutting facts like renames must be persisted structurally**, not left to per-module summaries.

Why distractors fail:

- **B** - Overflow would raise an error, and the names are not hallucinated - they were real, at module 3.
- **C** - No such stale-source caching, and the summaries, not the files, are what the agent is reading.
- **D** - Not confusion between similar names; the agent is confidently using a name that was correct earlier.

EXAM TIP

Exam Rule: - It traces the exact failure path. Module 3's summary - recorded before the rename - carried the old names. Module 12's summary was *supposed

152 What happens when a project knowledge base approaches context limits?

- (A) Claude stops accepting new file uploads.
- (B) Older files are automatically deleted to make room.
- (C) You must create a new project to continue.
- (D) Claude searches your project knowledge and retrieves only what's relevant, expanding capacity by up to 10x.

Answer: (D) Claude searches your project knowledge and retrieves only what's relevant, expanding capacity by up to 10x.

EXPLANATION

Beyond a threshold the project switches from loading everything into context to **retrieving what is relevant per request**. Because only the pertinent material is loaded each time, the effective capacity of the knowledge base grows substantially without any of it being discarded.

Why distractors fail:

- **A** and **C** - Both describe hard limits; the system degrades gracefully instead.
- **B** - Nothing is deleted; retrieval changes what is *loaded*, not what is *stored*.

EXAM TIP

Exam Rule: - Beyond a threshold the project switches from loading everything into context to retrieving what is relevant per request. Because only the pe

5.2 Escalation Triggers & Ambiguity Resolution

Claude CCAF: Context Management & Reliability - 4 questions

153 Policy covers refunds within 30 days. A customer asks for a price adjustment matching a competitor - something the policy does not address. What should the agent do?

- (A) Ask the customer to submit a formal exception request.
- (B) Deny the request since it is not in the policy.
- (C) Apply the closest analogous policy (30-day price guarantee) by inference.
- (D) Escalate to a human, since the request involves a policy gap not addressed by existing guidelines.

Answer: (D) Escalate to a human, since the request involves a policy gap not addressed by existing guidelines.

EXPLANATION

Policy gaps (uncovered scenarios) must be escalated to human operators. Agents must never fabricate policy or infer non-existent rules.

EXAM TIP

Uncovered policy gaps = Escalate to human operator; never fabricate analogous rules.

154 "This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW." The agent has not yet called any tools. What should it do?

- (A) Acknowledge the frustration and ask one targeted question before escalating.
- (B) Briefly explain what the agent can help with and offer to resolve it quickly, escalating only if the customer repeats the request.
- (C) Immediately call `escalate_to_human` with the conversation history.
- (D) First call `get_customer` and `lookup_order` to gather account context, then escalate.

Answer: (C) Immediately call `escalate_to_human` with the conversation history.

EXPLANATION

Explicit requests for human representatives must be honoured immediately without delaying the user for extra tool calls or negotiations.

EXAM TIP

Explicit customer human request = Immediate `escalate_to_human` with existing context.

155"Cancel my plan and refund me" - the account has two active plans and two recent charges, so the target is ambiguous, and both actions are irreversible. Which approach BEST balances autonomy and safety?

- (A) Treat the unresolved ambiguity over irreversible actions as an escalation trigger to human-in-the-loop confirmation, then proceed.
- (B) Pick the most recent plan and charge by default, since dynamic planning means committing to a best guess.
- (C) Replan repeatedly with different assumptions until one path completes without an error, then report that path.
- (D) Lower the temperature so the model deterministically resolves the ambiguity.

Answer: (A) Treat the unresolved ambiguity over irreversible actions as an escalation trigger to human-in-the-loop confirmation, then proceed.

EXPLANATION

Ambiguity over irreversible/destructive actions is a primary trigger for human-in-the-loop confirmation before executing state mutations.

EXAM TIP

Unresolved ambiguity + Irreversible action = Human confirmation gate before execution.

156Which escalation-trigger approach will most reliably identify cases that genuinely require human intervention?

- (A) Escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.
- (B) Escalate after three consecutive tool calls that fail to resolve the issue.
- (C) Sentiment analysis monitoring frustration indicators, escalating above a threshold.
- (D) A rules engine mapping issue types, customer segments, and product categories to escalation decisions.

Answer: (A) Escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.

EXPLANATION

Structural escalation triggers (explicit human request, policy exception, inability to progress) correlate directly with human necessity, unlike noisy sentiment proxies.

EXAM TIP

Reliable escalation triggers: (1) Explicit human request, (2) Policy exception required, (3) Stuck/no progress.

5.3 Multi-Agent Error Propagation & Resilience

Claude CCAF: Context Management & Reliability - 2 questions

157A third-party API is limited to 100 requests/minute and the agent's rapid tool calls exceed it. Where should rate limiting be implemented?

- (A) In the system prompt: "Wait at least 600ms between API calls".
- (B) In Claude's API parameters via a `request_delay` field.
- (C) In the MCP server implementation, using a token bucket algorithm that queues requests exceeding the rate limit.
- (D) In a `PreToolUse` hook that adds a delay between tool invocations.

Answer: (C) In the MCP server implementation, using a token bucket algorithm that queues requests exceeding the rate limit.

EXPLANATION

Rate limiting is an infrastructure concern that belongs in the MCP server adapter using token bucket / leaky bucket queuing, preventing 429 errors.

EXAM TIP

Rate limiting on external APIs belongs in MCP server implementation (token bucket queue).

158A docs pipeline reads source comments, wiki pages, and API schemas. Wiki retrieval times out; the other two succeed. The pipeline currently halts entirely on any source failure. Best error handling strategy?

- (A) Retry the wiki three times with exponential backoff; if all retries fail, halt to prevent incomplete documentation.
- (B) Return structured error context for the wiki failure, proceed with available sources, and mark the gaps that lack wiki content.
- (C) Silently skip the wiki source and generate from the remaining two without noting the omission.
- (D) Use the source comments to infer what the wiki pages would have contained.

Answer: (B) Return structured error context for the wiki failure, proceed with available sources, and mark the gaps that lack wiki content.

EXPLANATION

Graceful degradation with disclosure: continue pipeline execution with the successful sources (comments + schemas) while explicitly annotating sections missing wiki data.

EXAM TIP

Partial pipeline source outage = Graceful degradation with explicit gap annotations.

5.4 Large Codebase Context Management & Chunking

Claude CCAF: Context Management & Reliability - 2 questions

159 Adding error-handling wrappers across 120 files in three phases: (1) discover call sites, (2) collaboratively design the approach, (3) implement consistently. Phase 1's output fills the context window before discovery finishes. Most effective approach?

- (A) Define the pattern in CLAUDE.md, then process files in batches across multiple sessions relying on the shared memory file for consistency.
- (B) Use an Explore subagent for Phase 1 to isolate verbose discovery output and return a summary, then continue Phases 2-3 in the main conversation.
- (C) Switch to headless mode with --continue, passing explicit context summaries between batch calls.
- (D) Do all phases in the main conversation, periodically using /compact.

Answer: (B) Use an Explore subagent for Phase 1 to isolate verbose discovery output and return a summary, then continue Phases 2-3 in the main conversation.

EXPLANATION

Phase 1 discovery is verbose and exploratory. Delegating it to an Explore subagent confines raw search outputs to a disposable context, returning a clean summary for collaborative design.

EXAM TIP

Large discovery phase = Explore subagent -> returns summary -> design/edit in main session.

160 Repeated lookup_order calls return 40+ fields each; tool outputs now dominate the conversation, and the customer mentions two more orders. Most effective approach before making additional lookups?

- (A) Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details.
- (B) Have the model generate a natural language summary of each order, replacing structured responses with prose.
- (C) Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues.
- (D) Proceed with additional lookups without modifying the existing context.

Answer: (A) Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details.

EXPLANATION

Payload projection: prune 40+ fields down to the 4 return-relevant fields (items, date, window, status), reclaiming context while retaining structured precision.

EXAM TIP

Context bloat from repeated tool results = Project payloads to task-relevant fields only.

5.5 Human Review & Confidence Calibration

Claude CCAF: Context Management & Reliability - 3 questions

161 Confidence thresholds calibrated over three months; >90% auto-approves. A new client submits contracts in an unusual two-column format the system has never seen, and the system reports 92% confidence. Appropriate safeguard?

- (A) Maintain stratified random sampling of high-confidence extractions across document types to catch the new format's error rate.
- (B) Trust the calibrated threshold, since 92% exceeds 90%.
- (C) Raise the threshold from 90% to 98% for all document types.
- (D) Add a document format classifier that flags unknown formats for mandatory human review before any auto-approval.

Answer: (D) Add a document format classifier that flags unknown formats for mandatory human review before any auto-approval.

EXPLANATION

Confidence calibration is only valid within the distribution on which it was trained. Out-of-distribution formats must be flagged by a format classifier for mandatory human review.

EXAM TIP

Novel / unseen format = Flag for mandatory human review (confidence calibration is invalid out-of-distribution).

162 An agent emits confidence scores 0-1; confidence 0.9 and 0.5 produce wrong code at similar rates. MOST important step before relying on these scores?

- (A) Calibrate the confidence scores against a labeled validation set so 0.9 maps to actual 90% accuracy.
- (B) Trust the raw scores as-is.
- (C) Drop confidence scoring and gate auto-merge purely on whether the code compiles.
- (D) Train the model further on edge cases until confidence becomes reliable.

Answer: (A) Calibrate the confidence scores against a labeled validation set so 0.9 maps to actual 90% accuracy.

EXPLANATION

Raw self-reported confidence is uncalibrated. Calibration against labeled golden datasets maps raw model scores to actual empirical accuracy probabilities.

EXAM TIP

Raw confidence scores **MUST** be calibrated against labeled validation datasets before gating automation.

163 Three months at 100% human review; extractions with confidence >90% show 97% accuracy overall. Before automating high-confidence extractions, what validation step is most critical?

- (A) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- (B) Compare accuracy at 85%, 90%, and 95% thresholds to find the optimal cutoff.
- (C) Run a two-week pilot routing 25% of high-confidence extractions to downstream systems and monitor error reports.
- (D) Verify that 97% accuracy meets requirements for all downstream systems.

Answer: (A) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

EXPLANATION

Aggregate accuracy masks segment-level failures (e.g. 99% on standard invoices but 70% on tables). Validate by segment and field before removing human review.

EXAM TIP

Before automating high-confidence approvals: Segment accuracy by document type and field (Simpson's paradox).

5.6 Provenance, Attribution & Multi-Source Uncertainty

Claude CCAF: Context Management & Reliability - 4 questions

164 Batch API processing 50 documents; results must be matched to original documents and the batch returns them unordered. How do you correlate?

- (A) Use the custom_id field in batch requests to correlate results with document IDs.
- (B) Sequential processing with the real-time API for guaranteed ordering.
- (C) Results return in submission order, enabling positional array correlation.
- (D) Store batch request IDs in a database for post-processing correlation.

Answer: (A) Use the custom_id field in batch requests to correlate results with document IDs.

EXPLANATION

custom_id round-trips through the Batch API, enabling order-independent, deterministic correlation between submitted documents and returned results.

EXAM TIP

Batch API result correlation = Match via custom_id parameter.

165 A synthesis report attributes some claims to the wrong sources; provenance tracking failed. How do you maintain provenance in multi-agent synthesis?

- (A) The coordinator tracks which subagent produced which information.
- (B) Natural language source attribution in subagent responses is sufficient.
- (C) A separate provenance tracking subagent logging all information sources.
- (D) Subagents emit structured claim-source mappings in results passed to the coordinator.

Answer: (D) Subagents emit structured claim-source mappings in results passed to the coordinator.

EXPLANATION

Provenance must be bound at extraction time. Structured claim-source mappings ({claim, source_url, location}) survive synthesis merging and rewriting without misattribution.

EXAM TIP

Preserve multi-agent provenance = Subagents emit structured claim-source mappings.

166 Web search returns "2024: 35% adoption"; document analysis returns "2022: 18% adoption". The synthesis agent flags these as contradictory rather than recognising growth over time. What change best enables correct interpretation?

- (A) Require subagents to include publication or data collection dates in their structured outputs.
- (B) Add a conflict resolution agent that automatically discards older data when newer data exists.
- (C) Configure the web search agent to only return results from the past 6 months.
- (D) Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a historical appendix.

Answer: (A) Require subagents to include publication or data collection dates in their structured outputs.

EXPLANATION

Attaching publication/collection dates to structured findings transforms apparent contradictions into recognized time-series trends (growth from 18% in 2022 to 35% in 2024).

EXAM TIP

Reconcile metric discrepancies over time = Require dates in structured findings.

167 Contracts frequently include amendments (original "30-day payment terms", Amendment 1 changes it to "45 days"); the model inconsistently extracts one value or the other with no indication of which applies. Most effective approach?

- (A) Redesign the schema so amended fields capture multiple values, each with source location and effective date.
- (B) Prompt instructions to always extract the most recent amendment and ignore superseded original terms.
- (C) Preprocess documents with a classifier that identifies and removes superseded sections before extraction.
- (D) Post-extraction validation using pattern matching to detect amendments and flag them for manual review.

Answer: (A) Redesign the schema so amended fields capture multiple values, each with source location and effective date.

EXPLANATION

Contracts with amendments contain multiple valid terms over different effective periods. Structuring the schema as an array of entries with source location and effective date captures full legal provenance.

EXAM TIP

Contract amendments = Schema with array of {value, source_clause, effective_date}.

2.1 Tool Interface & JSON Schema Design

Claude CCAF: Tool Design & MCP Integration - 26 questions

168 An MCP tool description says "Retrieves customer information." The agent misroutes requests because the description lacks specificity; the tool accepts customer_id, email, or phone number. How should the description be improved?

- (A) Keep the description concise, as brevity improves agent processing speed.
- (B) Tool descriptions should remain minimal, delegating selection to agent reasoning.
- (C) Expand the description with specific use cases, accepted parameters, return format, and when to use.
- (D) Add a schema definition describing all possible parameters and return fields.

Answer: (C) Expand the description with specific use cases, accepted parameters, return format, and when to use.

EXPLANATION

Tool selection is made *from the description*, so specificity there is what fixes routing. Naming the accepted identifier types, the return shape, and the situations the tool is for gives the model everything it needs to decide before calling - and to recognise when a different tool fits better.

Why distractors fail:

- **A** - Trades selection accuracy for a "processing speed" benefit that a few hundred tokens of description do not actually confer.
- **B** - Delegating to reasoning over an under-specified description is exactly what produced the misrouting.
- **D** - A schema constrains *how to call* the tool once chosen; it does not explain *when* to choose it. Necessary but not sufficient, and the stem's failure is selection.

EXAM TIP

Exam Rule: - Tool selection is made *from the description*, so specificity there is what fixes routing. Naming the accepted identifier types, the return

169A tool description says "Processes orders." It actually updates order status, cancels orders, and initiates refunds; the agent misuses it for status checks. Improve the description?

- (A) Schema definition alone is sufficient without natural language description.
- (B) Tool descriptions should be minimal, allowing agent reasoning to determine usage.
- (C) Keep descriptions concise for faster agent processing.
- (D) Detailed description with specific capabilities, use cases, and when NOT to use.

Answer: (D) Detailed description with specific capabilities, use cases, and when NOT to use.

EXPLANATION

"Processes orders" understates a tool that performs three distinct **mutating** actions, and the observed failure is the model reaching for it on a read-only status check. The exclusion clause is what fixes that: stating explicitly that this tool mutates order state and is **not** for status lookups steers the model to the read tool instead.

Why distractors fail:

- **A** - A schema lists parameters; it does not communicate that this tool cancels orders and issues refunds.
- **B** and **C** - Both prescribe minimalism, which is the cause of the misuse.

EXAM TIP

Exam Rule: - "Processes orders" understates a tool that performs three distinct mutating actions, and the observed failure is the model reaching for it o

170An MCP server exposes several tools with overlapping capabilities; agents frequently choose the wrong one even though all are technically correct. Best improvement?

- (A) Shorten all tool descriptions so agents reason independently.
- (B) Remove overlapping tools entirely regardless of business requirements.
- (C) Expand tool descriptions with capabilities, limitations, expected inputs, outputs, and appropriate use cases.
- (D) Rely on parameter schemas alone because descriptions are unnecessary.

Answer: (C) Expand tool descriptions with capabilities, limitations, expected inputs, outputs, and appropriate use cases.

EXPLANATION

When several tools are all **technically** applicable, selection depends on the finer distinctions - limits, expected inputs, output shapes, and intended situations. Making those explicit converts an ambiguous set into a decidable one without removing capability.

Why distractors fail:

- **A** - Less information makes an already-ambiguous choice harder.
- **B** - Overrides business requirements to simplify routing; capability loss is not a description fix.
- **D** - Schemas describe call mechanics, not selection criteria.

EXAM TIP

Exam Rule: - When several tools are all **technically** applicable, selection depends on the finer distinctions - limits, expected inputs, output shapes, a

171The agent calls `get_customer` for order questions instead of `lookup_order`. Both have minimal descriptions ("Get customer info", "Get order details") and accept similar identifier formats. Most effective **first step**?

- (A) Consolidate into a single unified `customer_and_order` tool.
- (B) Expand tool descriptions with specific use cases, parameters, and when to use each.
- (C) Add few-shot examples showing correct tool usage for order and customer queries.
- (D) Implement a routing classifier analyzing requests to enable an appropriate tool subset.

Answer: (B) Expand tool descriptions with specific use cases, parameters, and when to use each.

EXPLANATION

Note "**first step**": fix the cheapest, most direct cause before adding machinery. Two three-word descriptions over tools with overlapping identifier formats leave the model almost nothing to discriminate on. Expanding them is a change to the tool definitions themselves, costs nothing at runtime, and typically resolves the misrouting outright.

Why distractors fail:

- **A** - Merging two distinct domains produces one tool with a mode parameter, relocating the ambiguity into an argument.
- **C** - A reasonable **second** step, but it compensates for weak descriptions with prompt tokens on every request rather than fixing the source.
- **D** - A whole additional component (latency, cost, its own errors) to route around a documentation gap.

EXAM TIP

Exam Rule: - Note "**first step**": fix the cheapest, most direct cause before adding machinery. Two three-word descriptions over tools with overlapping iden

172 Multiple MCP tools return data in different encodings (UTF-8, Latin-1, ASCII) and the agent struggles with the inconsistency. Where should encoding normalization occur?

- (A) Agent system prompt instructions handling multiple encoding types.
- (B) Fix encoding at the MCP tool source, returning consistent UTF-8.
- (C) The agent should flexibly handle all common encodings without normalization.
- (D) PostToolUse hook normalizing all tool outputs to UTF-8.

Answer: (B) Fix encoding at the MCP tool source, returning consistent UTF-8.

EXPLANATION

Encoding is part of the **tool's output contract**. Fixing it at the source means every consumer - this agent, other agents, future integrations - receives correct data, with no translation layer to maintain. Nothing in the stem says the tools are outside your control, so the root-cause fix is available.

Why distractors fail:

- **D** - The strongest runner-up, and the right answer *when the tools are third-party and unmodifiable* (see **UD2-036**, where the stem says exactly that, and **UD1-032** in 1.5). Here it is a workaround that leaves the defect in place and adds a component to maintain. The discriminator across this pool is **ownership**: own the tool -> fix the contract; don't own it -> normalise in a hook.
- **A** and **C** - Both push byte-level decoding into the model, which is neither its job nor something it can do reliably.

EXAM TIP

Exam Rule: - Encoding is part of the tool's output contract. Fixing it at the source means every consumer - this agent, other agents, future integrations

173 Two tools overlap in functionality and the agent frequently selects the wrong one. Consolidation was considered but the tools serve different systems. How do you improve selection without consolidation?

- (A) Expand tool descriptions with specific use cases and when to use each vs. the other.
- (B) Add a routing layer selecting the appropriate tool based on request analysis.
- (C) Few-shot examples showing correct tool selection in ambiguous cases.
- (D) Implement `tool_choice` forcing correct tool selection for each request type.

Answer: (A) Expand tool descriptions with specific use cases and when to use each vs. the other.

EXPLANATION

With consolidation ruled out, the remaining lever is **mutual disambiguation**: each description states not only what its tool does but how it differs from the other ("use this for System A records; for System B use . . ."). Cross-referencing is what turns two overlapping descriptions into a decidable pair.

Why distractors fail:

- **B** - Adds a component to compensate for descriptions you could simply improve.
- **C** - Helps, but costs prompt tokens on every request and generalises poorly to phrases not exemplified. A supplement, not the fix.
- **D** - `tool_choice` forces a tool for a *whole request*; to apply it "per request type" you would first need a classifier deciding which - i.e. you have not removed the routing problem, you have moved it outside the model and lost the model's judgement.

EXAM TIP

Exam Rule: - With consolidation ruled out, the remaining lever is mutual disambiguation: each description states not only what its tool does but how it d

174 An MCP tool's operation takes 45 seconds; the agent times out and never receives the (successful) result. How should long-running MCP operations be handled?

- (A) Implement an async pattern: the tool returns immediately with a `job_id`, plus a separate status-check tool.
- (B) Split the long operation into smaller incremental steps with partial results.
- (C) Increase agent timeout values to accommodate long-running operations.
- (D) Long-running operations should be handled outside agentic workflows entirely.

Answer: (A) Implement an async pattern: the tool returns immediately with a `job_id`, plus a separate status-check tool.

EXPLANATION

The async job pattern decouples *starting* the work from *waiting* for it. The initiating call returns in milliseconds with a handle, and the agent polls a status tool - so no single call risks a timeout, work is never orphaned after a timeout, and the agent can do other things while it runs.

Why distractors fail:

- **B** - Only viable if the operation is genuinely decomposable, which the stem does not establish; and it multiplies round trips.
- **C** - Raises the ceiling without changing the shape: a 90-second operation breaks it again, and the agent blocks throughout.
- **D** - Removes the capability rather than designing for it.

EXAM TIP

Exam Rule: - The async job pattern decouples *starting* the work from *waiting* for it. The initiating call returns in milliseconds with a handle, and th

175An MCP database_query tool executes SQL constructed by the agent from natural language. SQL injection risk exists. How should the tool protect against it?

- (A) Include SQL injection prevention instructions in the tool description.
- (B) The agent should validate SQL queries before execution.
- (C) The tool implementation uses parameterized queries preventing injection.
- (D) SQL injection is handled by the database layer, not the MCP tool.

Answer: (C) The tool implementation uses parameterized queries preventing injection.

EXPLANATION

Parameterised queries defeat injection **structurally**: values are bound as data and can never be reinterpreted as SQL, regardless of what the model produced. This is server-side enforcement in the component that owns the database connection - the only place a guarantee can be made.

Why distractors fail:

- **A** - A description is model-facing guidance; an injection payload does not care what it says.
- **B** - Makes the model its own security control. Model-side validation of adversarial input is exactly what must not be relied on.
- **D** - The database executes what it is given; it cannot distinguish an injected fragment from an intended one. Security belongs at the boundary that constructs the query.

EXAM TIP

Exam Rule: - Parameterised queries defeat injection structurally: values are bound as data and can never be reinterpreted as SQL, regardless of what the

176An async-operations tool returns a job_id for a long-running process and the agent needs to check completion. Tool design for async operations?

- (A) Streaming tool providing incremental progress updates.
- (B) Tool implements a callback notifying the agent when complete.
- (C) Initial tool returns job_id immediately; a separate check_status tool polls for completion.
- (D) Single tool blocking until the operation completes, returning the final result.

Answer: (C) Initial tool returns job_id immediately; a separate check_status tool polls for completion.

EXPLANATION

Two tools, cleanly separated: one starts work and returns a handle, one queries state by that handle. It fits the request/response tool protocol exactly, needs no persistent connection, and survives session interruption because the job_id remains valid.

Why distractors fail:

- **A** - Tool results are returned as a completed block; there is no incremental progress channel back into an in-flight tool result.
- **B** - There is no callback mechanism into an agent loop; the model only receives what a tool_result carries.
- **D** - Blocking is the design that caused the timeout in UD2-010.

EXAM TIP

Exam Rule: - Two tools, cleanly separated: one starts work and returns a handle, one queries state by that handle. It fits the request/response tool prot

177:(restatement of UD2-013)* A database_query tool accepts SQL constructed by the agent. Protection against injection?

- (A) Tool description with SQL injection prevention instructions.
- (B) Tool implementation uses parameterized queries preventing injection.
- (C) Database layer handles injection, not the tool.
- (D) Agent validates SQL before execution.

Answer: (B) Tool implementation uses parameterized queries preventing injection.

EXPLANATION

Same principle as UD2-013: bind values as parameters server-side so injected text can never be parsed as SQL.

Why distractors fail:

- **A** - Guidance, not enforcement.
- **C** - The database cannot tell intended SQL from injected SQL.
- **D** - Model-side validation of adversarial input is not a security control.

EXAM TIP

Exam Rule: - Same principle as UD2-013: bind values as parameters server-side so injected text can never be parsed as SQL.

178An MCP tool returns different timestamp formats from different backends (Unix, ISO 8601, human-readable) and the agent misinterprets dates. Correct fix?

- (A) Create separate tool wrappers for each backend that enforce their own format.
- (B) Use few-shot examples showing correct interpretation of each format.
- (C) Implement a PostToolUse hook to normalise all timestamps to ISO 8601 before the agent processes the result.
- (D) Add prompt instructions telling the agent how to parse each format.

Answer: (C) Implement a PostToolUse hook to normalise all timestamps to ISO 8601 before the agent processes the result.

EXPLANATION

One deterministic normalisation point covering every backend, applied before the model ever sees a timestamp. Format conversion is exact computation and belongs in code; the model then reasons over a single canonical representation.

Why distractors fail:

- **A** - Per-backend wrappers "enforcing their own format" preserve the heterogeneity that is the problem, and multiply the tool surface.
- **B** and **D** - Both leave parsing to the model, which is the failing step.

EXAM TIP

Exam Rule: - One deterministic normalisation point covering every backend, applied before the model ever sees a timestamp. Format conversion is exact com

179The agent misinterprets outputs: Unix timestamps from get_customer, ISO 8601 from lookup_order, numeric status codes (1=pending, 2=shipped). **Some tools are third-party MCP servers you cannot modify.** Most maintainable normalization approach?

- (A) A normalize_data tool the agent calls after every data retrieval.
- (B) Modify tools you control to return human-readable formats and create wrappers for third-party tools.
- (C) Use a PostToolUse hook to intercept tool outputs and apply formatting transformations before the agent processes them.
- (D) Add detailed format documentation to the system prompt.

Answer: (C) Use a PostToolUse hook to intercept tool outputs and apply formatting transformations before the agent processes them.

EXPLANATION

The unmodifiable third-party servers are the deciding constraint: a hook is the **one place** that can normalise *every* tool's output regardless of who owns it. One component, one set of rules, applied automatically - the maintainability criterion the question names.

Why distractors fail:

- **B** - Two mechanisms (direct edits plus wrappers) and a wrapper per third-party tool, all drifting independently. This is the answer when you own everything (UD2-008); here it is the higher-maintenance option.
- **A** - Relies on the model remembering to call a normaliser after every retrieval, doubling round trips and failing silently when it forgets.
- **D** - Documents the inconsistency instead of removing it, and still leaves conversion to the model.

EXAM TIP

Exam Rule: - The unmodifiable third-party servers are the deciding constraint: a hook is the one place that can normalise *every* tool's output regardless

180Two tools: analyze_content ("Analyses content") and extract_web_results ("Extracts data from web pages"). After a system prompt update added **"Always analyse content before responding"**, the agent began routing web-extraction tasks to analyze_content, though descriptions were unchanged. Most likely cause and fix?

- (A) The system prompt's "analyse content" phrasing keyword-matches the analyze_content tool; rephrase it to remove the overlap.
- (B) Add few-shot examples to the system prompt showing when to use each tool.
- (C) Rename analyze_content to something less generic, such as summarize_document.
- (D) The descriptions have degraded over time and need rewriting with clearer boundaries.

Answer: (A) The system prompt's "analyse content" phrasing keyword-matches the analyze_content tool; rephrase it to remove the overlap.

EXPLANATION

The stem hands you the causal evidence: behaviour changed **when and only when** the system prompt changed, with tool definitions untouched. The new instruction contains the tool's own name as a phrase, so it reads as a standing directive to invoke that tool. Rephrasing the instruction ("review the retrieved information before responding") removes the collision at its source.

Why distractors fail:

- **B** - Adds prompt material to counteract prompt material, leaving the conflicting instruction in place.
- **C** - A genuinely good hygiene change and the right long-term move for a name that generic - but it does not address *this* regression, whose cause is the instruction wording. Renaming would also break nothing else that changed.

- **D** - Contradicted by the stem: the descriptions did not change, and the behaviour did.

EXAM TIP

Exam Rule: - The stem hands you the causal evidence: behaviour changed when and only when the system prompt changed, with tool definitions untouched. The

181A tool `analyze_content` ("Analyses content from various sources") is used indiscriminately for web scraping, document parsing, and code analysis, producing poor results for each. Most effective fix?

- (A) Add few-shot examples showing which content types the tool handles well.
- (B) Improve the description to list all supported content types and formats.
- (C) Split `analyze_content` into purpose-specific tools: `extract_web_results`, `parse_document`, `analyze_code`, each with a narrow remit.
- (D) A routing layer inspecting content type and dispatching to the appropriate internal function within `analyze_content`.

Answer: (C) Split `analyze_content` into purpose-specific tools: `extract_web_results`, `parse_document`, `analyze_code`, each with a narrow remit.

EXPLANATION

The tool is **genuinely three tools** wearing one name. Splitting gives each its own precise description, its own schema (a URL is not a file path is not a repo reference), and its own output contract - so selection becomes obvious and each implementation can be optimised for its domain. This is the mirror image of UD2-009: there, two tools needed disambiguation; here, one tool needs decomposition.

Why distractors fail:

- **A** and **B** - Both document an overloaded tool more thoroughly. The model still calls one tool that does three jobs adequately and none of them well; "poor results for each use case" is an *implementation* symptom, not only a selection one.
- **D** - Internal dispatch hides the three behaviours behind one interface, so the model still cannot see which capability it is getting or shape its input for it.

EXAM TIP

Exam Rule: - The tool is genuinely three tools wearing one name. Splitting gives each its own precise description, its own schema (a URL is not a file pa

182Using the Python MCP SDK to create a file-reading tool. Easiest way to define it?

- (A) Use the `@mcp.tool()` decorator on a Python function.
- (B) Create a separate configuration file.
- (C) Write JSON schemas manually.
- (D) Send HTTP requests to register the tool.

Answer: (A) Use the `@mcp.tool()` decorator on a Python function.

EXPLANATION

The decorator is the SDK's ergonomic path: it registers the function as a tool and derives the input schema from the signature's type hints, with the docstring serving as the description. One decorator, no separate schema to keep in sync.

Why distractors fail:

- **B** - Tools are defined in code, not in an external config file.
- **C** - Hand-writing JSON Schema is exactly the work the decorator eliminates, and it drifts from the implementation.
- **D** - Registration is in-process at server start-up, not over HTTP.

EXAM TIP

Exam Rule: - The decorator is the SDK's ergonomic path: it registers the function as a tool and derives the input schema from the signature's type hints,

183Tools each accept a `confidence_threshold` float (0-1), but semantics drift between teams (0.7 means "high" to some, "medium" to others). BEST architectural fix?

- (A) Add a description note to each tool documenting how that tool interprets the float.
- (B) Replace the float with a shared enum (low/medium/high) defined in a common schema module, imported by every tool so semantics are unified at the type level.
- (C) Standardize on float and publish a guideline that 0.5 means medium.
- (D) Have the orchestrator post-process tool calls to normalize the threshold before invoking.

Answer: (B) Replace the float with a shared enum (low/medium/high) defined in a common schema module, imported by every tool so semantics are unified at the type level.

EXPLANATION

The root problem is that a bare float carries **no shared meaning** - every team must invent a mapping. A shared enum makes the semantics part of the type: `high` means the same thing everywhere by construction, there is no scale to misinterpret, and a single imported definition prevents drift as tools are added.

Why distractors fail:

- **A** - Documents N different interpretations instead of unifying them; the model must now track per-tool conventions.
- **C** - A guideline is a convention with no enforcement; the same drift recurs with the next team.
- **D** - Central normalisation cannot recover intent: the orchestrator sees `0.7` with no way to know which team's scale produced it. It also patches the symptom downstream of the ambiguity.

EXAM TIP

Exam Rule: - The root problem is that a bare float carries no shared meaning - every team must invent a mapping. A shared enum makes the semantics part o

184 A `refund_payment` call appears to fail with a network timeout, Claude retries, and the user is double-refunded. Which tool design change PREVENTS this?

- (A) Server-side rate limiting rejecting duplicate calls within five seconds.
- (B) Increase the timeout from 30 seconds to 5 minutes.
- (C) Make the tool synchronous so Claude waits and cannot retry prematurely.
- (D) Require an `idempotency_key` parameter, with the server deduplicating retries by that key.

Answer: (D) Require an `idempotency_key` parameter, with the server deduplicating retries by that key.

EXPLANATION

A timeout is **ambiguous** - the refund may have succeeded with only the response lost. An idempotency key resolves it authoritatively: a retry carrying the same key returns the original outcome instead of performing a second refund. Correct under every interleaving, including retries minutes apart or from a different process.

Why distractors fail:

- **A** - A time window is a heuristic: it blocks legitimate distinct refunds inside 5 seconds and permits duplicates outside it.
- **B** - Reduces timeout frequency; the duplicate still occurs whenever one happens.
- **C** - The call already is synchronous; the retry happens **after** the timeout, which blocking does not prevent.

EXAM TIP

Exam Rule: - A timeout is **ambiguous** - the refund may have succeeded with only the response lost. An idempotency key resolves it authoritatively: a retr

185 An MCP server lists 12 tools with comprehensive `inputSchema` and `description`, yet clients report some are confusing to choose between. Which addition MOST directly helps the **host application surface clearer choices to its users**?

- (A) Add an `annotations` block (`readOnlyHint`, `destructiveHint`, `idempotentHint`) so the host can render visual cues about each tool's nature.
- (B) Add a `priority` integer so the host can sort tools in display order.
- (C) Include the full source code of the tool implementation in the `description`.
- (D) Provide a screenshot of the tool's output in each tool's metadata.

Answer: (A) Add an `annotations` block (`readOnlyHint`, `destructiveHint`, `idempotentHint`) so the host can render visual cues about each tool's nature.

EXPLANATION

Note the audience: **the host's users**, not the model. MCP annotations are machine-readable behavioural hints designed for exactly this - a host can badge a tool as read-only, warn on a destructive one, or indicate that retrying is safe. That is UX metadata no amount of prose description provides.

Why distractors fail:

- **B** - Ordering does not tell a user what a tool **does** or how risky it is.
- **C** - Source code in a description bloats the model's context and is unreadable as end-user UI.
- **D** - Not part of the MCP tool definition, and screenshots do not generalise across inputs.

EXAM TIP

Exam Rule: - Note the audience: the host's users, not the model. MCP annotations are machine-readable behavioural hints designed for exactly this - a hos

186 A tool-use schema has 8 fields; the model occasionally populates fewer. Which interpretation is MOST consistent with required-field semantics?

- (A) Mark all eight as required to fix the omissions.
- (B) Tool use ignores the `required` array, so marking fields required has no effect.
- (C) Lower temperature to fill the missing fields.
- (D) If the model omits fields, those fields aren't in the `required` array; review the required list and add the fields that must always be populated.

Answer: (D) If the model omits fields, those fields aren't in the `required` array; review the required list and add the fields that must always be populated.

EXPLANATION

It reasons from the mechanism: omission is **permitted** for optional fields, so an omitted field is evidence that it was not declared required. The remedy is a deliberate review of which fields genuinely must always be present, then adding those - a schema change grounded in intent.

Why distractors fail:

- **A** - Superficially similar but wrong in method: marking **all eight** required forces the model to fabricate values for fields that legitimately have no value (see GEN2-004). D adds the ones that must be populated; A adds all of them.
- **B** - False: `required` is honoured, and under `strict: true` it is grammar-enforced.
- **C** - Temperature does not change which fields the schema permits to be absent.

EXAM TIP

Exam Rule: - It reasons from the mechanism: omission is **permitted** for optional fields, so an omitted field is evidence that it was not declared require

187 Built-in web search plus a custom MCP academic-database search, both well-described. A system prompt asks the model to prefer the academic database for scholarly queries. Why might this still fail to enforce routing?

- (A) System-prompt rules are always ignored by Claude.
- (B) System-prompt routing rules are instructions the model usually follows but can be overridden by adversarial inputs or context drift.
- (C) System-prompt rules only work for custom tools, not for distinguishing built-in from MCP-exposed tools.
- (D) Such routing rules are forbidden by Anthropic's terms of service.

Answer: (B) System-prompt routing rules are instructions the model usually follows but can be overridden by adversarial inputs or context drift.

EXPLANATION

A system prompt is **strong guidance, not enforcement** - the same principle as Domain 1's hooks-vs-prompts theme. Compliance is high but not guaranteed, and it degrades under adversarial content in retrieved documents or under context drift in a long session. If routing must be guaranteed, it belongs in code (a gate, or simply not exposing the competing tool).

Why distractors fail:

- **A** - Overstated: system prompts are followed most of the time.
- **C** - Fabricates a distinction; the model selects from one tool list regardless of a tool's origin.
- **D** - Invented policy claim.

EXAM TIP

Exam Rule: - A system prompt is strong guidance, not enforcement - the same principle as Domain 1's hooks-vs-prompts theme. Compliance is high but not guaranteed

188 An MCP server exposes a `run_shell` tool flagged as too permissive. MOST important hardening before production?

- (A) Add a description noting the agent should only use safe commands.
- (B) Replace the open shell with an allowlist of permitted commands and validated argument shapes, enforced server-side before any execution.
- (C) Log every shell command for post-hoc review.
- (D) Require user confirmation before each shell invocation in the client UI.

Answer: (B) Replace the open shell with an allowlist of permitted commands and validated argument shapes, enforced server-side before any execution.

EXPLANATION

A server-side allowlist with validated argument shapes reduces the tool's **capability** to what is actually needed. Anything outside the allowlist is unexecutable regardless of prompt injection, model error, or a compromised client - enforcement at the boundary that performs the execution.

Why distractors fail:

- **A** - Model-facing guidance on a tool that grants arbitrary code execution.
- **C** - Detection after execution; the damage precedes the log entry.
- **D** - Valuable defence in depth, and the strongest runner-up - but it is **client-side**, so it can be bypassed by a different client or an auto-approve mode, and approval fatigue erodes it. Server-side capability reduction is the control that holds.

EXAM TIP

Exam Rule: - A server-side allowlist with validated argument shapes reduces the tool's capability to what is actually needed. Anything outside the allowlist

189 An orchestrator has 12 tools. Which observable signal BEST indicates it is approaching the partitioning threshold?

- (A) The raw number of tools exceeds ten.
- (B) Total tokens spent on tool definitions exceeds five thousand.
- (C) The orchestrator starts making wrong tool selections, redundant calls (same tool twice for the same intent), or routing drift across similar queries.
- (D) Average end-to-end response time increases.

Answer: (C) The orchestrator starts making wrong tool selections, redundant calls (same tool twice for the same intent), or routing drift across similar queries.

EXPLANATION

These are **direct measurements of the failure itself** - degraded routing quality. They also appear regardless of **why** the catalogue became hard to route (overlap, generic names, semantic proximity), which counts and token totals cannot capture: 30 crisply distinct tools may route perfectly while 8 overlapping ones do not.

Why distractors fail:

- **A** and **B** - Proxies with arbitrary thresholds. Neither measures whether routing is actually degrading, and both fire on well-designed large catalogues while missing small confusing ones.
- **D** - Latency has many causes (slow tools, retries, longer tasks) and is a lagging, non-specific indicator.

EXAM TIP

Exam Rule: - These are direct measurements of the failure itself - degraded routing quality. They also appear regardless of **why** the catalogue became ha

190 A priority enum [low, medium, high] gains a fourth level, `urgent`. Which migration is MOST aligned with schema-evolution discipline?

- (A) Add `urgent` to the enum, regenerate the schema, update few-shot examples to demonstrate when `urgent` applies, and run the test dataset to confirm prior cases still classify correctly.
- (B) Add the value silently and deploy, since one additive enum value is low-risk.
- (C) Keep three levels and use `high` as a proxy for `urgent`.
- (D) Lower temperature for accurate `urgent` classification.

Answer: (A) Add `urgent` to the enum, regenerate the schema, update few-shot examples to demonstrate when `urgent` applies, and run the test dataset to confirm prior cases still classify correctly.

EXPLANATION

A complete change: schema updated, **examples updated** so the model learns the new boundary (without them, `urgent` exists but is under-used or over-used), and **regression tested** so you can prove that tickets previously classified `high` have not silently migrated. That last step is the discipline - adding an enum value shifts the decision boundary of every neighbouring value.

Why distractors fail:

- **B** - Its premise is exactly wrong: introducing `urgent` changes what counts as `high`, so existing classifications **can** shift.
- **C** - Refuses the requirement; the business asked to distinguish the two levels.
- **D** - Temperature is unrelated to classification boundaries.

EXAM TIP

Exam Rule: - A complete change: schema updated, examples updated so the model learns the new boundary (without them, `urgent` exists but is under-used or

191 A `run_command` tool takes shell-like arguments; the team wants to constrain what Claude can pass without rewriting the tool per project. BEST schema design balancing flexibility and safety?

- (A) Take a base64-encoded command string, decode and execute.
- (B) Take a single freeform `command_string` parameter.
- (C) Take an array of arguments and join them with spaces.
- (D) Use a discriminated-union schema with a `command_type` enum and per-type argument shapes (`type=test -> test_runner` enum + `test_path`; `type=build -> build_target` enum), validated server-side against an allowlist for each type.

Answer: (D) Use a discriminated-union schema with a `command_type` enum and per-type argument shapes (`type=test -> test_runner` enum + `test_path`; `type=build -> build_target` enum), validated server-side against an allowlist for each type.

EXPLANATION

The discriminated union makes the **space of expressible commands** exactly the space of intended operations. The model chooses an operation type and fills typed fields for it; there is no free-text channel through which an arbitrary command could be constructed. Flexibility comes from adding variants, not from loosening the type - so new projects extend the enum rather than rewriting the tool.

Why distractors fail:

- **A** - Base64 is an encoding, not a control: it conceals the payload from inspection while granting the same arbitrary execution.
- **B** - Maximum flexibility, zero safety; the tool is arbitrary code execution with the model as the only gate.
- **C** - Joining arbitrary array elements with spaces reconstructs an arbitrary command line; array shape provides no constraint on content.

EXAM TIP

Exam Rule: - The discriminated union makes the space of expressible commands exactly the space of intended operations. The model chooses an operation typ

192 An MCP tool needs to return formatted Markdown plus a code block plus an image. BEST return mechanism?

- (A) Concatenate everything into one large text string with inline references to the image.
- (B) Return multiple content items in the tool result's content array.
- (C) Make three separate tool calls, one per content type.
- (D) Rich content is not supported in MCP tool responses; flatten to plain text and drop the image.

Answer: (B) Return multiple content items in the tool result's content array.

EXPLANATION

An MCP tool result carries a content **array**, and each item is typed (text, image, and other content types). One call returns all three parts with their types preserved, so the host renders each appropriately and the model receives the image as an image rather than as a description of one.

Why distractors fail:

- **A** - Flattening to text loses the image entirely; an inline reference is not the content.
- **C** - Three round trips for one logical result, and the pieces arrive decoupled from each other.
- **D** - Factually wrong: multi-item, multi-type content is part of the protocol.

EXAM TIP

Exam Rule: - An MCP tool result carries a content array, and each item is typed (text, image, and other content types). One call returns all three parts

193 An `embedding_model` parameter is free-text; Claude passes valid names ~80% of the time and occasionally outdated or non-existent ones. The team maintains a supported-models list. Which schema change BEST eliminates invalid calls?

- (A) Accept any string and have the server reject invalid names with a clear error.
- (B) Add a description note listing all supported models.
- (C) Type the parameter as an enum of current supported model names, regenerated from the supported-models list whenever it changes.
- (D) Pin to one default model and remove the choice entirely.

Answer: (C) Type the parameter as an enum of current supported model names, regenerated from the supported-models list whenever it changes.

EXPLANATION

An enum makes an invalid value **unrepresentable** - under strict tool use the schema constrains generation, so the model cannot emit a name outside the set. Regenerating from the maintained list keeps the schema and the truth in sync automatically, so the enum never goes stale.

Why distractors fail:

- **A** - Detects invalid calls after the fact and pays a wasted round trip plus a retry each time; "eliminates" is not achieved.
- **B** - A description is guidance; 80% compliance is the current state, reached with the model already able to read the tool definition.
- **D** - Eliminates invalid values by eliminating the feature.

EXAM TIP

Exam Rule: - An enum makes an invalid value unrepresentable - under strict tool use the schema constrains generation, so the model cannot emit a name out

2.2 Structured Error Handling & Resilience

Claude CCAF: Tool Design & MCP Integration - 5 questions

194 A developer needs to distinguish "the tool errored" from "the tool succeeded but found no results"; the agent currently says "I couldn't find that information" for both. What response distinction should be implemented?

- (A) Distinguish access failures (`isError: true`, `isRetryable: true` for timeouts) from valid empty results (`isError: false` with an empty result set).
- (B) Always return `isError: true` and include a severity field to differentiate.
- (C) Return the same response format but include a `log_level` field (warning vs error).
- (D) Use HTTP status codes (404 for not found, 500 for errors) in the tool response.

Answer: (A) Distinguish access failures (`isError: true`, `isRetryable: true` for timeouts) from valid empty results (`isError: false` with an empty result set).

EXPLANATION

`isError: false` with an empty array indicates a successful query that found zero records. `isError: true` signals an execution or connectivity failure. Conflating them confuses the agent.

EXAM TIP

Empty result set = `isError: false`. Tool/network failure = `isError: true`.

195How should a business rule violation (refund amount exceeds policy limit) be communicated back to the agent?

- (A) Return `isError: false` with the policy limit details embedded in the success response.
- (B) Throw an exception that terminates the agentic loop.
- (C) Return `isError: true` with structured metadata including `errorCategory='policy_violation'`, `isRetryable: false`, and a customer-friendly description.
- (D) Return `isError: true` with a generic "Operation failed" message.

Answer: (C) Return `isError: true` with structured metadata including `errorCategory='policy_violation'`, `isRetryable: false`, and a customer-friendly description.

EXPLANATION

Business rule violations must return `isError: true` with `errorCategory='policy_violation'` and `isRetryable: false` so the agent knows not to retry and can explain the policy to the customer.

EXAM TIP

Business rule violation = `isError: true`, `isRetryable: false`, `category: policy_violation`.

196A payment tool's error response is just `{"status": "error"}`. Which redesign best follows the four-field structured error pattern?

- (A) `{"error_code": "INSUFFICIENT_FUNDS", "message": "The account balance is too low to complete this payment.", "context": {"account_id": "acct_123", "attempted_amount": 500}, "suggested_action": "ask_user"}`
- (B) `{"status": "error", "details": "payment failed, see logs for stack trace"}`
- (C) `{"status": "error", "error_code": "ERROR"}`
- (D) `{"status": "success", "note": "payment may not have completed, please verify"}`

Answer: (A) `{"error_code": "INSUFFICIENT_FUNDS", "message": "The account balance is too low to complete this payment.", "context": {"account_id": "acct_123", "attempted_amount": 500}, "suggested_action": "ask_user"}`

EXPLANATION

The 4-field error pattern provides: (1) `error_code` (stable machine code), (2) `message` (explanatory text), (3) `context` (exact parameters involved), and (4) `suggested_action` (retry, ask_user, escalate).

EXAM TIP

4-field structured error schema = `error_code + message + context + suggested_action`.

197Which operation is idempotent by nature, and which requires deliberate design to become retry-safe?

- (A) A GET-style balance lookup is idempotent by nature (repeat calls re-read the same state); a payment-charge tool is NOT (each call is a new side effect) and needs a deliberate mechanism such as a client-generated idempotency key.
- (B) Both are idempotent by nature.
- (C) Neither is idempotent, since any tool call could theoretically be retried.
- (D) The payment-charge tool is idempotent by nature since gateways deduplicate internally by default.

Answer: (A) A GET-style balance lookup is idempotent by nature (repeat calls re-read the same state); a payment-charge tool is NOT (each call is a new side effect) and needs a deliberate mechanism such as a client-generated idempotency key.

EXPLANATION

Reads/lookups are inherently idempotent. Mutating actions (charges, refunds, emails) are non-idempotent and require client-generated idempotency keys for retry safety.

EXAM TIP

Reads = Naturally idempotent. Mutating actions = Require client-generated idempotency keys.

198A circuit breaker is Closed and failures start exceeding the threshold. What happens next, in order?

- (A) Closed -> threshold exceeded -> Open (calls blocked outright, fallback or error returns immediately) -> after a configured wait -> Half-Open (a single probe call tests recovery) -> probe succeeds -> Closed; probe fails -> Open.
- (B) Closed -> Half-Open (calls blocked) -> Open (probe tests recovery) -> back to Closed regardless of the probe's result.
- (C) Once a circuit breaker opens, it never recovers automatically.
- (D) Circuit breakers only have two states, Closed and Open.

Answer: (A) Closed -> threshold exceeded -> Open (calls blocked outright, fallback or error returns immediately) -> after a configured wait -> Half-Open (a single probe call tests recovery) -> probe succeeds -> Closed; probe fails -> Open.

EXPLANATION

Circuit breaker lifecycle: Closed (normal) -> threshold breached -> Open (fail fast without calling downstream) -> sleep wait -> Half-Open (single probe) -> Closed (if probe OK) or Open (if probe fails).

EXAM TIP

Circuit Breaker states: Closed (healthy) -> Open (failing/blocked) -> Half-Open (canary probe).

Structured Error Handling & Error Categorization

Claude CCAF: Tool Design & MCP Integration - 4 questions

199A research agent calls an external API via an MCP server. After 30 queries in a batch of 50, the API returns HTTP 429 rate limit errors. The server currently returns generic 'Request failed'. What MCP server change would most improve resilience?

- (A) Implement automatic retry with exponential backoff inside the MCP server, hiding rate limits from the agent
- (B) Return `errorCategory: 'transient', isRetryable: true`, with a `retryAfterMs` field telling the agent how long to wait
- (C) Return `errorCategory: 'business', isRetryable: false` to tell the agent to stop making requests
- (D) Queue all 50 requests at the MCP server level and process them sequentially

Answer: (B) Return `errorCategory: 'transient', isRetryable: true`, with a `retryAfterMs` field telling the agent how long to wait

EXPLANATION

Rate limiting is a transient error. Returning structured error metadata with `errorCategory: 'transient', isRetryable: true`, and `retryAfterMs` enables the agent to pause intelligently and resume rather than abandoning the batch.

EXAM TIP

HTTP 429 / Rate Limit = `errorCategory: 'transient', isRetryable: true + retryAfterMs`.

200A `search_papers` MCP tool has three failure patterns: (1) upstream HTTP 503 during peak hours, (2) user requests papers from an unlicensed restricted journal, and (3) agent submits a malformed DOI string. Which `errorCategory` and `isRetryable` combination is correct?

- (A) All three should be `errorCategory: 'transient', isRetryable: true`
- (B) HTTP 503: `transient/retryable`; restricted journal: `business/not retryable`; malformed DOI: `validation/retryable`
- (C) HTTP 503: `transient/retryable`; restricted journal: `transient/retryable`; malformed DOI: `transient/retryable`
- (D) HTTP 503: `validation/retryable`; restricted journal: `business/not retryable`; malformed DOI: `business/not retryable`

Answer: (B) HTTP 503: `transient/retryable`; restricted journal: `business/not retryable`; malformed DOI: `validation/retryable`

EXPLANATION

HTTP 503 = Transient (temporary outage, retry later). Restricted Journal = Business (licensing policy, retries will never work -> escalate). Malformed DOI = Validation (agent must fix format and retry).

EXAM TIP

The 4 MCP Error Categories: 1. Transient (503/429 - retryable), 2. Validation (bad schema/regex - repair input & retry), 3. Business (policy/license - non-retryable, escalate), 4. Permission (auth failed - non-retryable).

201Your MCP server implements `check_availability` for a calendar API. You encounter: (1) missing required `user_email` parameter, (2) calendar API returns 404 user not found, (3) calendar API returns 503 service unavailable. How should each be reported in MCP?

- (A) Report all three as tool results with `isError: true`
- (B) Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as tool result with `isError: true`
- (C) Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with `isError: true`
- (D) Report all three as JSON-RPC protocol errors

Answer: (C) Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with `isError: true`

EXPLANATION

Malformed JSON-RPC requests (missing required params at protocol level) are JSON-RPC Protocol Errors. Valid tool calls that execute but fail upstream (404 user not found, 503 outage) are Tool Results with `isError: true`.

EXAM TIP

Protocol Error = Malformed RPC message / unknown method. Tool Result with `isError: true` = Downstream business / execution / API failures.

202The data platform's `fetch_api` tool calls a pricing API. When the API key has expired, the tool returns `{ 'data': [], 'status': 'ok' }`. The agent tells the user 'No pricing data is available for that product'. What is the root cause?

- (A) The agent should be instructed via prompt to treat empty arrays as errors
- (B) The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data
- (C) The third-party API is poorly designed and should be replaced
- (D) The context window was too small to hold the API response

Answer: (B) The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data

EXPLANATION

Masking an access/auth failure as a successful empty array { data: [], status: 'ok' } prevents the agent from realizing an authentication error occurred. Tools must return `isError: true` with specific `errorCategory` for access failures.

EXAM TIP

Access Failure vs Empty Result: Never return empty data [] for permission/network errors. Use `isError: true` with `permission/transient errorCategory`.

2.3 Tool Distribution, Selection & tool_choice

Claude CCAF: Tool Design & MCP Integration - 4 questions

203 An agent has 16 tools across multiple MCP servers; selection reliability is degrading and not all tools are relevant to its role. How do you optimize tool distribution?

- (A) Routing classifier dynamically filtering tools based on request type.
- (B) Training period for the agent to learn optimal selection patterns.
- (C) Consolidate MCP servers to a single server, reducing management complexity.
- (D) Role-specific MCP configuration exposing only the relevant 4-5 tools per agent.

Answer: (D) Role-specific MCP configuration exposing only the relevant 4-5 tools per agent.

EXPLANATION

Role-specific configuration restricts each agent's catalog to 4-5 relevant tools, removing distractor candidates and reducing tool definition token overhead.

EXAM TIP

Catalog degradation with too many tools = Scope 4-5 role-specific tools per agent.

204 Match `tool_choice` settings: (1) an open-ended assistant that answers directly for simple questions but uses tools for complex ones; (2) a database dispatcher that must always act (lookup/create/update) but the action depends on the request; (3) a pipeline step that must always run one specific extraction tool; (4) a turn where tools are attached but this turn should be text-only.

- (A) (1) auto, (2) any, (3) tool, (4) none
- (B) (1) any, (2) auto, (3) none, (4) tool
- (C) All four should use auto.
- (D) (1) tool, (2) none, (3) auto, (4) any

Answer: (A) (1) auto, (2) any, (3) tool, (4) none

EXPLANATION

`tool_choice` modes: 'auto' (model decides text or tool), 'any' (forces at least one tool call), 'tool' (forces specific named tool), 'none' (forces text response while keeping tools cached).

EXAM TIP

`tool_choice`: auto (optional), any (some tool required), tool (specific tool required), none (text only).

205 200 tools aggregated across MCP servers; a tool search tool is added and `defer_loading: true` set on most. What happens to those definitions, and what must remain true for search to work?

- (A) The 200 deferred definitions are entirely omitted from the request; only the search tool's definition is sent.
- (B) Deferred tool definitions are still included in the request (so search can find and expand them) but kept out of the active context until the model searches for and matches one; at least one tool must remain non-deferred (and the search tool itself must never be deferred).
- (C) Setting `defer_loading: true` immediately loads all 200 definitions in a compressed format.
- (D) `defer_loading` only works if every tool, including the search tool, is deferred.

Answer: (B) Deferred tool definitions are still included in the request (so search can find and expand them) but kept out of the active context until the model searches for and matches one; at least one tool must remain non-deferred (and the search tool itself must never be deferred).

EXPLANATION

`defer_loading: true` defers tool schema expansion until searched. The tool search tool itself must NEVER be deferred, and at least one non-deferred tool must exist.

EXAM TIP

Tool search: `defer_loading: true` on catalog; tool search tool itself must NEVER be deferred.

206 extract_metadata must run before lookup_citations (which needs the DOI it produces), but on "extract the metadata and tell me how cited it is" Claude sometimes calls lookup_citations first and it fails. Most effective way to ensure extraction happens first?

- (A) Set tool_choice: "any" plus system prompt instructions prioritizing extract_metadata.
- (B) Set tool_choice: "auto" and reorder the tools array so extract_metadata appears first, since Claude prioritizes earlier-listed tools.
- (C) Set tool_choice: {"type": "tool", "name": "extract_metadata"} and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- (D) Set tool_choice: {"type": "tool", "name": "extract_metadata"} for every API call in the pipeline.

Answer: (C) Set tool_choice: {"type": "tool", "name": "extract_metadata"} and process the enrichment requests in subsequent turns after receiving the extracted metadata.

EXPLANATION

Force the named tool on the FIRST turn only. Once the DOI is extracted and returned in context, revert to auto on subsequent turns so enrichment tools can be called.

EXAM TIP

Enforce first step: Force tool_choice: {'type': 'tool', 'name': '...'} on Turn 1 -> auto on Turn 2.

2.4 MCP Server Integration & Protocols

Claude CCAF: Tool Design & MCP Integration - 5 questions

207 Users should be able to click a button to trigger a "summarize document" workflow. Which MCP primitive?

- (A) Resources - because you need to fetch document data.
- (B) Functions - because it involves processing.
- (C) Prompts - because users control when to start the workflow.
- (D) Tools - because the AI needs new capabilities.

Answer: (C) Prompts - because users control when to start the workflow.

EXPLANATION

Prompts are user-controlled templates surfaced in UI slash menus. Tools are model-controlled actions. Resources are application-controlled data context.

EXAM TIP

MCP primitive ownership: Prompts = User-controlled. Tools = Model-controlled. Resources = App-controlled.

208 Which transport requires both client and server to run on the same machine?

- (A) TCP transport
- (B) Stdio transport
- (C) WebSocket transport
- (D) HTTP transport

Answer: (B) Stdio transport

EXPLANATION

STDIO communicates via standard input/output streams of a spawned child process, requiring same-machine execution. StreamableHTTP is networked for remote connections.

EXAM TIP

Same-machine child process = STDIO. Remote/networked multi-client = StreamableHTTP.

209 What is sampling in MCP?

- (A) A method for collecting data from multiple sources.
- (B) A way for servers to access language models through connected MCP clients.
- (C) A process for validating client credentials.
- (D) A technique for optimizing server performance.

Answer: (B) A way for servers to access language models through connected MCP clients.

EXPLANATION

Sampling allows an MCP server to request LLM completions through the host client, leveraging the client's API keys, billing, and user approval gates.

EXAM TIP

MCP Sampling = Server requests LLM generation routed through the host client.

210 Correct sequence for MCP connection initialization?

- (A) Initialized Notification -> Initialize Request -> Initialize Result
- (B) Initialize Request -> Initialized Notification -> Initialize Result
- (C) Initialize Result -> Initialize Request -> Initialized Notification
- (D) Initialize Request -> Initialize Result -> Initialized Notification

Answer: (D) Initialize Request -> Initialize Result -> Initialized Notification

EXPLANATION

Handshake sequence: Client sends Initialize Request -> Server responds with Initialize Result -> Client sends Initialized Notification to begin normal operation.

EXAM TIP

MCP Handshake: Initialize Request -> Initialize Result -> Initialized Notification.

211 All engineers should have three approved internal MCP servers available from Day 1 without configuring them manually. Correct architecture?

- (A) Add the servers to allowedMcpServers in managed-settings.json.
- (B) Add the configs to the shared .mcp.json in the team's main repository.
- (C) Bundle the MCP server configs into a plugin, assign the engineering SCIM group to it via RBAC, and deploy org-wide.
- (D) Send each developer the configuration file and include setup steps in the onboarding doc.

Answer: (C) Bundle the MCP server configs into a plugin, assign the engineering SCIM group to it via RBAC, and deploy org-wide.

EXPLANATION

Enterprise distribution uses Plugins assigned to SCIM user groups via RBAC for zero-touch configuration. allowedMcpServers only permits servers but does not distribute them.

EXAM TIP

Zero-touch MCP enterprise distribution = Plugins + SCIM Group RBAC.

2.5 Built-in Tools vs Custom Client Tools

Claude CCAF: Tool Design & MCP Integration - 4 questions

212 Which built-in tools run on Anthropic's infrastructure with no handler code, and which execute in the developer's environment requiring a handler?

- (A) web_search and code_execution are server tools (Anthropic infra, no handler); computer_use, bash, and text_editor are client tools (developer's environment, still need handler code even though schema authoring is skipped).
- (B) All five run entirely on Anthropic's infrastructure with zero developer-side code.
- (C) computer_use and code_execution are server tools; web_search, bash, and text_editor are client tools.
- (D) The server/client distinction doesn't apply to built-in tools, only to custom MCP tools.

Answer: (A) web_search and code_execution are server tools (Anthropic infra, no handler); computer_use, bash, and text_editor are client tools (developer's environment, still need handler code even though schema authoring is skipped).

EXPLANATION

Server tools (web_search, code_execution) execute on Anthropic servers with zero client handler code. Client tools (bash, text_editor, computer_use) execute inside your local environment.

EXAM TIP

Server tools (Anthropic infra) = web_search, code_execution. Client tools (local infra) = bash, text_editor, computer_use.

213 Find all files matching **/*.test.tsx across a large codebase. Which built-in tool?

- (A) Read
- (B) Glob
- (C) Bash
- (D) Grep

Answer: (B) Glob

EXPLANATION

Glob is purpose-built for filename pattern matching. Grep searches file contents. Read opens a specific file.

EXAM TIP

Search file names / directory paths = Glob. Search file contents = Grep.

214A security review asks which built-in tools could theoretically be tricked into reaching the company's private internal network. Which can, and why?

- (A) Client tools like `bash`, `text_editor`, and `computer_use` can reach private resources, because they run in the developer's own environment, wherever that environment is connected; server tools like `web_search` and `code_execution` stay inside Anthropic's boundary and cannot reach a company's private network at all.
- (B) All built-in tools can equally reach a private network, since they're all controlled by the same model.
- (C) Only `web_search` can, since it makes outbound requests.
- (D) None of the built-in tools can ever reach a private network.

Answer: (A) Client tools like `bash`, `text_editor`, and `computer_use` can reach private resources, because they run in the developer's own environment, wherever that environment is connected; server tools like `web_search` and `code_execution` stay inside Anthropic's boundary and cannot reach a company's private network at all.

EXPLANATION

Network reach follows execution location. Client tools execute inside your private subnet/VPC; server tools execute on Anthropic infrastructure with no routing into your private network.

EXAM TIP

Private network access: Only Client Tools (`bash`, `text_editor`) run inside your perimeter.

215Inserting a helper function between two existing functions in a 150-line module; Edit fails because `old_string` cannot find unique text (repetitive docstrings, names, structure). Most reliable way?

- (A) Use Edit with a 30+ line `old_string` to guarantee uniqueness.
- (B) Use Edit's `replace_all` to target a common pattern and embed the new function in the replacement.
- (C) Use Bash to append the function to the end of the file with a heredoc.
- (D) Use Read to load the file, add the function at the appropriate location, then Write the updated file.

Answer: (D) Use Read to load the file, add the function at the appropriate location, then Write the updated file.

EXPLANATION

When `old_string` uniqueness fails in repetitive files, Read-modify-Write bypasses anchor matching: load the file, place the function in context, and Write the full updated file.

EXAM TIP

Edit unique anchor match fails -> Fall back to Read-modify-Write pattern.

4.1 Explicit Criteria & False-Positive Reduction

Claude CCAF: Prompt Engineering & Structured Output - 26 questions

216A prompt extracts invoice data (amount \$1,247.83, date 2024-03-15, vendor Acme Corp) but the agent summarizes as "approximately \$1,200 mid-March from vendor". What principle prevents lossy summarization?

- (A) Validation checking extracted data matches source document exactly.
- (B) Schema constraints enforcing specific formats for amounts and dates.
- (C) Never summarize transactional facts: preserve exact amounts, dates, IDs, names.
- (D) More explicit instructions emphasizing precise extraction without summarization.

Answer: (C) Never summarize transactional facts: preserve exact amounts, dates, IDs, names.

EXPLANATION

The question asks for the **principle**, and this is it: extraction is a **copying** operation, not a **describing** one. Transactional facts have no acceptable paraphrase - "approximately \$1,200" is not a lossy version of \$1,247.83, it is a different and unusable value, and the same holds for dates and identifiers.

Why distractors fail:

- **A** - Validation is a **detective control** that catches violations of the principle; it presupposes the principle rather than stating it.
- **B** - Schema constraints enforce the **shape** of a value (a decimal, an ISO date) but cannot detect that a correctly-shaped number is the wrong one.
- **D** - Emphasis on an instruction that is already being disobeyed; it names no principle.

EXAM TIP

Exam Rule: - The question asks for the principle, and this is it: extraction is a **copying** operation, not a **describing** one. Transactional facts have n

217 Automated code review in CI/CD has a high false-positive rate; **detailed criteria in the prompt doesn't reduce them**. Which technique reduces false positives?

- (A) More explicit criteria defining exactly what constitutes a legitimate issue.
- (B) Multiple review passes with a voting mechanism.
- (C) Increase confidence threshold, requiring high-confidence findings only.
- (D) Few-shot examples showing borderline cases with reasoning about accept vs reject.

Answer: (D) Few-shot examples showing borderline cases with reasoning about accept vs reject.

EXPLANATION

The stem's key clause is that **detailed criteria have already failed**. False positives cluster at the *boundary* between "real issue" and "acceptable code", and that boundary resists description - but it is demonstrable. Borderline examples *with the reasoning attached* teach the discrimination directly: this looks like a problem and is not, and here is why.

Why distractors fail:

- **A** - Explicitly ruled out by the stem; more of what is not working.
- **B** - Voting over N passes that share the same faulty boundary reproduces the same false positives at Nx cost.
- **C** - Raising the threshold suppresses low-confidence findings indiscriminately, discarding true positives along with false ones. It trades recall for precision rather than improving the judgement.

EXAM TIP

Exam Rule: - The stem's key clause is that detailed criteria have already failed. False positives cluster at the *boundary* between "real issue" and "acc

218 Invoice extraction should preserve amount \$3,127.45, date 2024-01-22, vendor DataCorp LLC, order #PO-2024-1847, but the agent extracts "about \$3,100 January 2024 from vendor". What principle prevents lossy extraction?

- (A) Schema constraints enforcing precise formats for all critical fields.
- (B) Never summarize transactional facts: preserve exact amounts, dates, IDs, names.
- (C) Validation ensuring extracted data exactly matches source content.
- (D) Explicit prompt instructions emphasising precise extraction over summarisation.

Answer: (B) Never summarize transactional facts: preserve exact amounts, dates, IDs, names.

EXPLANATION

Same principle as UD4-003, and the dropped **order number** makes the point sharply: #PO-2024-1847 has no approximate form at all. Identifiers, amounts, dates, and names are copied, never characterised.

Why distractors fail:

- **A** - Formats cannot detect a wrong value in the right shape.
- **C** - Detection after the fact, and it presupposes the principle.
- **D** - Emphasis, not principle.

EXAM TIP

Exam Rule: - Same principle as UD4-003, and the dropped order number makes the point sharply: #PO-2024-1847 has no approximate form at all. Identifiers

219 Automated code review has a 40% false-positive rate; **detailed review criteria in the prompt doesn't reduce it**. Which technique reduces the rate?

- (A) Few-shot examples showing borderline cases with reasoning about accept versus reject.
- (B) Increase confidence threshold, requiring only high-confidence findings.
- (C) More explicit review criteria defining exactly what constitutes legitimate issues.
- (D) Multiple review instances with a voting mechanism for consensus.

Answer: (A) Few-shot examples showing borderline cases with reasoning about accept versus reject.

EXPLANATION

Identical to UD4-006: criteria have been tried and failed, so the remedy is demonstrating the boundary rather than describing it. At a 40% false-positive rate the boundary is badly miscalibrated, and worked borderline cases with reasoning are what recalibrate it.

Why distractors fail:

- **C** - Ruled out by the stem.
- **B** - Suppresses findings by confidence rather than correcting the judgement, losing true positives too.
- **D** - Consensus among instances sharing the same miscalibration is agreement, not accuracy.

EXAM TIP

Exam Rule: - Identical to UD4-006: criteria have been tried and failed, so the remedy is demonstrating the boundary rather than describing it. At a 40% f

220 During a long debugging session the agent starts giving vague, generic responses ("this follows typical patterns") instead of referencing specific classes and methods it examined earlier. What is happening and what should be done?

- (A) The model's temperature is increasing over time.
- (B) The model is rate-limited.
- (C) The agent needs more explicit instructions.
- (D) Context degradation from an extended session.

Answer: (D) Context degradation from an extended session.

EXPLANATION

The symptom is diagnostic: responses become **generic precisely where they were previously specific**, which is what happens as earlier detail is crowded out or diluted in a long, accumulating context. The remedy is context management - compact with a summary that preserves the key findings, or start a fresh session seeded with them (Domain 1, 1.7).

Why distractors fail:

- **A** - Temperature is a fixed request parameter; it does not drift during a session.
- **B** - Rate limiting produces 429 errors, not degraded content.
- **C** - Instructions cannot recover specifics no longer effectively available in context; adding them consumes more of the resource that is already scarce.

EXAM TIP

Exam Rule: - The symptom is diagnostic: responses become generic precisely where they were previously specific, which is what happens as earlier detail is

221 Inconsistent severity ratings, growing developer distrust ("half are wrong"), and high-false-positive categories eroding trust in accurate ones. Which approach best restores developer trust while improving the system?

- (A) Keep all categories enabled and add few-shot examples to improve accuracy over the next few weeks.
- (B) Temporarily disable high-false-positive categories (style, naming, documentation) and keep only high-precision categories while improving prompts.
- (C) Keep all categories enabled but display confidence scores so developers decide what to investigate.
- (D) Apply a uniform strictness reduction across all categories.

Answer: (B) Temporarily disable high-false-positive categories (style, naming, documentation) and keep only high-precision categories while improving prompts.

EXPLANATION

Trust is the binding constraint, and it is **contaminated across categories** - developers dismissing findings unread means even accurate security findings go unactioned. Disabling the noisy categories immediately raises observed precision, so the remaining findings are worth reading; "temporarily ... while improving prompts" preserves the path back. It fixes trust now and quality in parallel.

Why distractors fail:

- **A** - Correct long-term work, wrong sequencing: trust keeps eroding for the weeks it takes, and by then developers have stopped reading.
- **C** - Confidence scores shift the filtering burden onto the developers who have already opted out of reading; a low-confidence finding is still noise in the queue.
- **D** - Uniform reduction lowers sensitivity on the 8% -false-positive security category too, suppressing exactly the findings that are trustworthy.

EXAM TIP

Exam Rule: - Trust is the binding constraint, and it is contaminated across categories - developers dismissing findings unread means even accurate securi

222 Measured false-positive rates by category: security/correctness 8%, performance 18%, style/naming 52%, documentation 48%. Developers dismiss findings unread. Which approach best restores trust?

- (A) Apply a uniform strictness reduction across all categories.
- (B) Keep all categories enabled and add few-shot examples over the next few weeks.
- (C) Keep all enabled but display confidence scores.
- (D) Temporarily disable high-false-positive categories (style, naming, documentation) and keep only high-precision categories while improving prompts.

Answer: (D) Temporarily disable high-false-positive categories (style, naming, documentation) and keep only high-precision categories while improving prompts.

EXPLANATION

Same as UD4-029 with the numbers supplied, and they make the case arithmetic: style and documentation are near coin-flips, while security sits at 92% precision. Removing the two noisy categories transforms the observed hit rate without touching the findings that matter - and the highest-value findings become visible again.

Why distractors fail:

- **A** - Would degrade the 8% category to protect the 52% one.
- **B** - Right work, too slow; trust is eroding now.

- **C** - Delegates triage to developers who have already disengaged.

EXAM TIP

Exam Rule: - Same as UD4-029 with the numbers supplied, and they make the case arithmetic: style and documentation are near coin-flips, while security si

223 Which three elements make up an effective prompt?

- (A) Introduction, body, and conclusion.
- (B) Setting the stage, defining the task, and specifying rules.
- (C) Question, answer, and follow-up.
- (D) Subject, verb, and object.

Answer: (B) Setting the stage, defining the task, and specifying rules.

EXPLANATION

The three elements map to the three things a model needs: **context** (role, audience, background - setting the stage), **the task** (what specifically to produce), and **constraints** (format, tone, length, prohibitions). Omitting any one is where most prompt failures in this section originate.

Why distractors fail:

- **A** - Essay structure, describing a document's shape rather than a prompt's components.
- **C** - Conversation structure; the answer is what you are asking *for*, not part of the prompt.
- **D** - Sentence grammar, unrelated.

EXAM TIP

Exam Rule: - The three elements map to the three things a model needs: context (role, audience, background - setting the stage), the task (what specifica

224 If Claude's response is too generic, what should you try?

- (A) Switch to a different AI model.
- (B) Ask Claude to search the web for better answers.
- (C) Add more details about your audience, role, or constraints.
- (D) Start a new conversation and ask the same question.

Answer: (C) Add more details about your audience, role, or constraints.

EXPLANATION

Generic output is the expected response to a generic prompt: absent an audience, a role, or constraints, the model produces something broadly acceptable to everyone. Supplying those specifics narrows the target, and specificity in the prompt is what produces specificity in the answer.

Why distractors fail:

- **A** - A more capable model still cannot know an audience you have not named.
- **B** - Web results address *missing information*, not missing specificity.
- **D** - The same question in a new conversation yields the same generic answer.

EXAM TIP

Exam Rule: - Generic output is the expected response to a generic prompt: absent an audience, a role, or constraints, the model produces something broadl

225 A good Cowork prompt typically names which three things?

- (A) The deliverable you want, the inputs to use, and any nuance like audience or format.
- (B) Your name, your team, and your deadline.
- (C) The model, the temperature, and the token limit.
- (D) A greeting, the question, and a thank-you.

Answer: (A) The deliverable you want, the inputs to use, and any nuance like audience or format.

EXPLANATION

In a file-based workflow the three things Claude cannot infer are **what to produce** (the deliverable), **what to work from** (which files), and **how it should come out** (audience, format, length). Naming all three is what turns "summarise these" into an actionable task.

Why distractors fail:

- **B** - Metadata about you, not about the work.
- **C** - API parameters, not exposed or relevant in this surface.
- **D** - Politeness, which changes nothing about the output.

EXAM TIP

Exam Rule: - In a file-based workflow the three things Claude cannot infer are what to produce (the deliverable), what to work from (which files), and ho

226:"Generate idiomatic SQL queries" yields inconsistent style - keyword casing and column quoting vary. Which change MOST directly stabilizes style?

- (A) Name the style guide explicitly.
- (B) Raise temperature so style varies less from query to query.
- (C) Provide one example query and instruct the model to match its keyword casing and quoting.
- (D) Use prefilling to start each response with SELECT so the style is anchored early.

Answer: (A) Name the style guide explicitly.

EXPLANATION

"Idiomatic" is undefined - several mutually incompatible SQL conventions are all idiomatic, so the model picks one each time. Naming the specific guide (or spelling out its rules: uppercase keywords, double-quoted identifiers) replaces an underspecified word with a **complete, unambiguous standard** that covers cases no single example would.

Why distractors fail:

- **C** - The reasonable runner-up, and it would help - but one example demonstrates only the constructs it happens to contain, leaving JOINs, CTEs, and window functions unspecified. A named guide covers all of them.
- **B** - Backwards: higher temperature *increases* variance. (And sampling variance is not the cause; underspecification is.)
- **D** - Prefilling anchors one token and says nothing about identifier quoting or the rest of the query - and assistant prefill is rejected on current models.

EXAM TIP

Exam Rule: - "Idiomatic" is undefined - several mutually incompatible SQL conventions are all idiomatic, so the model picks one each time. Naming the spe

227:"Fetch the most relevant 5 sources" yields sources that sometimes overlap heavily. Which change BEST improves source diversity?

- (A) Increase the source count to 10 so diversity emerges naturally.
- (B) Require sources to cover distinct viewpoints.
- (C) Raise temperature so source selection is more varied.
- (D) Sort sources alphabetically and take the first five for reproducibility.

Answer: (B) Require sources to cover distinct viewpoints.

EXPLANATION

The prompt optimises for **relevance alone**, and the five most relevant sources are frequently near-duplicates of each other - the overlap is the criterion working as specified. Adding diversity as an explicit selection criterion changes what "best five" means, so the model optimises for coverage as well as relevance.

Why distractors fail:

- **A** - Ten sources selected by the same criterion gives ten overlapping sources; volume does not create coverage.
- **C** - Randomness is not diversity: it produces varied *runs*, not varied *viewpoints within a run*, and degrades relevance.
- **D** - Alphabetical order is unrelated to either relevance or diversity; reproducibly bad.

EXAM TIP

Exam Rule: - The prompt optimises for relevance alone, and the five most relevant sources are frequently near-duplicates of each other - the overlap is t

228:The agent should reply in Spanish when the user writes in Spanish, but sometimes replies in English or code-switches. Which change MOST directly enforces language matching?

- (A) Add "always be linguistically appropriate" so the model self-corrects.
- (B) Run a language-detection step in a separate API call first.
- (C) Lower temperature to 0 so the language choice becomes deterministic.
- (D) Rule: detect the dominant language of the user's most recent message and respond entirely in that language unless the user explicitly requests another.

Answer: (D) Rule: detect the dominant language of the user's most recent message and respond entirely in that language unless the user explicitly requests another.

EXPLANATION

It converts a vague intent into a **complete decision procedure**: which signal to read (the most recent message), how to resolve mixed input (the *dominant* language - which is what fixes code-switching), what to output (entirely that language), and the one exception. Nothing is left to interpretation.

Why distractors fail:

- **A** - "Linguistically appropriate" is vaguer than the original instruction.
- **B** - An extra API call, extra latency, and its own error rate, to determine something the model can determine inline.
- **C** - Temperature 0 makes the *same wrong choice* consistently; determinism is not correctness.

EXAM TIP

Exam Rule: - It converts a vague intent into a complete decision procedure: which signal to read (the most recent message), how to resolve mixed input (t

229."Extract dates in ISO format" yields 2026-06-15, 2026/06/15, and June 15 2026. Which change BEST stabilizes the format?

- (A) Use JSON mode so the date is emitted as a clean string value.
- (B) Provide three examples of ISO dates and trust the model to generalize.
- (C) Lower temperature so the date-format choice stabilizes.
- (D) Specify "YYYY-MM-DD with hyphens, zero-padded; reject any other format" and provide one canonical example.

Answer: (D) Specify "YYYY-MM-DD with hyphens, zero-padded; reject any other format" and provide one canonical example.

EXPLANATION

It removes every degree of freedom the failures exploited: the **separator** (hyphens, which kills the / variant), **zero-padding**, the exact field order, and an explicit rejection of alternatives - plus one example to anchor it. Rule *and* demonstration, which is stronger than either alone.

Why distractors fail:

- **B** - Three examples without a stated rule leave the model to infer which properties are essential; and the item's own phrasing ("trust the model to generalize") is the weakness.
- **A** - JSON mode guarantees the value is a valid *string*; "June 15 2026" is a perfectly valid JSON string.
- **C** - Determinism without a specification just fixes which wrong format recurs.

EXAM TIP

Exam Rule: - It removes every degree of freedom the failures exploited: the separator (hyphens, which kills the / variant), zero-padding, the exact fie

230.A system prompt has 8 imperative instructions, with new rules appended at the bottom over time. Which change BEST improves long-term maintainability?

- (A) Convert the imperatives to declaratives so they read more naturally.
- (B) Number the rules so each can be referenced by a stable index.
- (C) Group related rules under XML-tagged sections.
- (D) Lower temperature so the rules are applied more consistently.

Answer: (C) Group related rules under XML-tagged sections.

EXPLANATION

The maintainability problem is **structural**: an append-only flat list means related rules drift apart, duplicates go unnoticed, and there is no obvious place to add the next one. XML-tagged sections give each rule a home (<style>, <testing>, <security>), which both guides future edits and improves the model's ability to attend to the relevant group - XML structuring is a documented Claude prompting technique, not merely cosmetic.

Why distractors fail:

- **B** - Stable indices help *reference* rules but do nothing to organise them; and inserting a rule rennumbers everything after it.
- **A** - Imperatives are the clearer form for instructions; declaratives read as description rather than requirement.
- **D** - Temperature is unrelated to maintainability, and this is the recurring non-answer of this domain.

EXAM TIP

Exam Rule: - The maintainability problem is structural: an append-only flat list means related rules drift apart, duplicates go unnoticed, and there is n

231."Refactor for readability while keeping behavior the same" - some refactors silently change function signatures. Which change MOST directly prevents signature changes?

- (A) Lower temperature to 0 so refactors become more conservative.
- (B) Explicitly forbid signature changes.
- (C) Run the refactor through a linter after Claude finishes.
- (D) Use prefilling to start with def and the original function name.

Answer: (B) Explicitly forbid signature changes.

EXPLANATION

"Behavior the same" is genuinely ambiguous - a model can read it as *observable behaviour for a given call*, under which renaming a parameter seems harmless. Naming the prohibition explicitly closes that reading, and it is the most direct fix because it addresses the exact misinterpretation causing the failures. (BU4-013 restates this item with the full wording spelled out.)

Why distractors fail:

- **C** - A linter checks style; it has no notion of the *original* signature to compare against, so it cannot detect the change.
- **A** - Temperature does not make a model "conservative"; it narrows sampling around the same interpretation.
- **D** - Prefill is rejected on current models, and anchoring the first line would not prevent a changed parameter list further along it.

EXAM TIP

Exam Rule: - "Behavior the same" is genuinely ambiguous - a model can read it as *observable behaviour for a given call*, under which renaming a paramete

232:"Write a good README" - some include installation but skip usage, others include badges but no quickstart. Which change MOST efficiently fixes the gaps?

- (A) Require a fixed section order.
- (B) Show Claude five high-quality READMEs from popular projects.
- (C) Lower temperature so the README structure stabilizes.
- (D) Add "be thorough and complete" so no section is skipped.

Answer: (A) Require a fixed section order.

EXPLANATION

The failure is **missing sections**, and a required section list makes omission impossible to overlook - the model has an explicit checklist rather than a judgement about what a good README contains. Note "MOST **efficiently**": this is a few lines of prompt, not five documents.

Why distractors fail:

- **B** - Would work, but costs thousands of tokens on every call and still leaves the required set implicit; the model must infer which sections were essential versus incidental.
- **D** - "Thorough" is exactly the vagueness that produced inconsistent coverage.
- **C** - Determinism does not add a missing section.

EXAM TIP

Exam Rule: - The failure is missing sections, and a required section list makes omission impossible to overlook - the model has an explicit checklist

233:"Verify the customer's identity before sharing account details" - the model sometimes shares partial details without verification. Which change BEST hardens the rule?

- (A) Lower temperature so the model is less likely to share details prematurely.
- (B) Add "verify thoroughly and completely" to the existing prompt.
- (C) Specify the verification steps.
- (D) Wrap account details in an <account> XML tag in the response.

Answer: (C) Specify the verification steps.

EXPLANATION

"Verify identity" names an outcome without a procedure, so the model has to decide what counts as verified - and "partial details" suggests it is treating partial confirmation as sufficient. Enumerating the steps ("confirm account number **and** registered email; both must match before any detail is disclosed") makes the precondition checkable rather than judged.

> **Worth knowing beyond this item:** for a genuine guarantee on a security-sensitive action, the prompt layer is the wrong layer entirely - a programmatic prerequisite gate is (Domain 1, 1.4, UD1-008). Among the options offered here, C is the strongest, but a real system should gate `share_account_details` on a completed verification call.

Why distractors fail:

- **B** - Adverbs on an unspecified procedure.
- **A** - Temperature does not encode a policy.
- **D** - Tags the output after the disclosure decision was already made.

EXAM TIP

Exam Rule: - "Verify identity" names an outcome without a procedure, so the model has to decide what counts as verified - and "partial details" suggests

234Merge recommendations vary between "merge" and "consider merging"; reviewers want a binary decision. Which change BEST enforces a binary verdict?

- (A) Add "be decisive and commit to a single verdict".
- (B) Lower temperature to 0 so the recommendation is more confident.
- (C) Require the response to end with exactly one of: VERDICT: MERGE or VERDICT: HOLD.
- (D) Use prefilling to start the response with "I recommend".

Answer: (C) Require the response to end with exactly one of: VERDICT: MERGE or VERDICT: HOLD.

EXPLANATION

It replaces a request for decisiveness with a **format constraint that has no hedging form**: there is no way to emit VERDICT: MAYBE. The fixed token also makes the verdict machine-parseable, which is what a CI gate needs - the response can still explain its reasoning above the line.

Why distractors fail:

- **A** - "Be decisive" is an adverb on the same open-ended output; "consider merging" can be delivered decisively.
- **B** - Determinism produces a *consistent* hedge, not a binary one.
- **D** - Anchors an opening phrase that leads naturally into hedged prose, and prefill is rejected on current models.

EXAM TIP

Exam Rule: - It replaces a request for decisiveness with a format constraint that has no hedging form: there is no way to emit VERDICT: MAYBE. The fix

23511 explicit field-handling criteria but no rule for missing fields; outputs use `null`, `"`, and `"unknown"` inconsistently. Which change MOST directly stabilizes missing-field handling?

- (A) Lower temperature so the choice becomes deterministic.
- (B) Drop missing fields from the output entirely.
- (C) Add the 12th criterion: "Missing fields must use `null`; never empty string, never sentinel strings."
- (D) Use JSON mode so the output is always syntactically valid.

Answer: (C) Add the 12th criterion: "Missing fields must use `null`; never empty string, never sentinel strings."

EXPLANATION

The inconsistency exists because **the case was never specified** - with 11 criteria covering present fields and none covering absent ones, the model improvises. Adding the twelfth criterion, with the alternatives explicitly forbidden, closes the gap. It also picks the semantically right marker: `null` means absent, whereas `"` and `"unknown"` are values that downstream code may treat as data.

Why distractors fail:

- **A** - Determinism would fix one arbitrary marker, possibly `"unknown"`.
- **B** - Omitting keys makes "absent from the document" indistinguishable from "field not in this schema version", and breaks consumers expecting a fixed shape.
- **D** - All three current outputs are already valid JSON; syntax was never the problem.

EXAM TIP

Exam Rule: - The inconsistency exists because the case was never specified - with 11 criteria covering present fields and none covering absent ones, the

236."Apologize when the customer is upset" - some responses apologize for routine inquiries, others miss apologies after genuine complaints. Which change BEST calibrates the behavior?

- (A) Lower temperature to make the apology choice deterministic.
- (B) Always apologize first to be safe.
- (C) Use few-shot with two upset and two non-upset tickets so the model learns the boundary.
- (D) Specify trigger conditions for apology.

Answer: (D) Specify trigger conditions for apology.

EXPLANATION

"Upset" is undefined, so the model is applying its own threshold - which is why it errs in both directions. Naming the triggers (explicit dissatisfaction, a service failure on our side, a repeat contact about an unresolved issue) gives a checkable condition instead of a mood judgement.

> **Discriminator against UD4-006 / UD4-015**, where few-shot beat explicit criteria: in those items the stem states that **detailed criteria were already tried and failed**, so the remaining move was demonstration. Here the current prompt is a single vague clause - explicit criteria have not been tried yet, and are the cheaper, more direct first step. Option **C** is the correct *escalation* if D proves insufficient.

Why distractors fail:

- **B** - Apologising on routine inquiries is one of the two failure modes; making it universal entrenches it and reads as insincere.
- **A** - Determinism fixes the threshold in place rather than correcting it.

EXAM TIP

Exam Rule: - "Upset" is undefined, so the model is applying its own threshold - which is why it errs in both directions. Naming the triggers (explicit di

237.Two pipelines share a model with near-identical system prompts differing only in schema names (invoices vs. purchase orders). A developer suggests consolidating into one prompt with a runtime flag. MOST accurate assessment?

- (A) Routing logic must always live in the system prompt for consolidation to work.
- (B) Consolidation is always better because it reduces maintenance overhead.
- (C) Two separate user messages on one consolidated prompt achieve the same isolation as two prompts.
- (D) Consolidation increases prompt complexity and risks cross-document interference; keep specialized system prompts unless the deduplication win is large.

Answer: (D) Consolidation increases prompt complexity and risks cross-document interference; keep specialized system prompts unless the deduplication win is large.

EXPLANATION

A consolidated prompt must describe **both** schemas, so every invoice extraction is performed with purchase-order fields in context - an invitation to field bleed, particularly where names are similar. It names the real trade-off and the right default: the duplication here is small (schema names), while the interference risk is on output correctness.

Why distractors fail:

- **B** - "Always better" ignores the interference cost; maintenance savings on two near-identical prompts are minor.
- **C** - Sharing a system prompt means sharing the context; separate user messages do not isolate what the system prompt describes.
- **A** - Fabricates a rule; routing can live in application code, and it is not what makes consolidation succeed or fail.

EXAM TIP

Exam Rule: - A consolidated prompt must describe both schemas, so every invoice extraction is performed with purchase-order fields in context - an invita

238*(restatement of BU4-006 with the fix spelled out)* Refactors silently change function signatures. Which change MOST directly prevents this?

- (A) Lower temperature to 0 so refactors become more conservative.
- (B) Run the refactor through a linter after Claude finishes.
- (C) Explicitly forbid signature changes - "Function name, parameter names, parameter order, and return type must remain identical."
- (D) Use prefilling to start with def and the original function name.

Answer: (C) Explicitly forbid signature changes - "Function name, parameter names, parameter order, and return type must remain identical."

EXPLANATION

The spelled-out version shows why it works: it enumerates **all four** components of a signature, so no part of it is left to interpretation. "Keep behavior the same" permitted a reading under which parameter renaming was harmless; this does not.

Why distractors fail:

- **B** - A linter has no reference to the original signature.
- **A** - Temperature is not a conservatism control.
- **D** - Prefill is rejected on current models and would constrain only the first tokens.

EXAM TIP

Exam Rule: - The spelled-out version shows why it works: it enumerates all four components of a signature, so no part of it is left to interpretation. "K

239Generated SQL migrations occasionally drop columns that should remain. Which prompt refinement BEST protects against unintended destructive schema changes?

- (A) Explicitly state that existing tables and columns must not be dropped unless the prompt explicitly requests deletion.
- (B) Lower temperature to make schema generation more deterministic.
- (C) Ask Claude to generate shorter SQL scripts so accidental drops are less likely.
- (D) Place the migration inside a Markdown code block so Claude treats it more carefully.

Answer: (A) Explicitly state that existing tables and columns must not be dropped unless the prompt explicitly requests deletion.

EXPLANATION

It states a **default-deny rule for the destructive operation** and names the single condition that lifts it. A model refactoring a schema may reasonably infer that a superseded column should go; this removes that inference and makes deletion something that must be asked for.

Why distractors fail:

- **B** - Deterministic destructive migrations are still destructive.
- **C** - Length is unrelated to whether a DROP COLUMN is included.
- **D** - Formatting does not change the model's care; code blocks affect presentation.

> As with BU4-008, for production migrations the durable protection is a **review gate or a hook** that blocks destructive DDL - prompt rules reduce frequency but do not guarantee.

EXAM TIP

Exam Rule: - It states a default-deny rule for the destructive operation and names the single condition that lifts it. A model refactoring a schema may r

240A field the schema expects is simply not present in the source document. The extractor should:

- (A) Fill the field with a plausible value inferred from the rest of the document.
- (B) Return null for that field and mark it as not found, leaving the rest of the extraction intact.
- (C) Fail the entire extraction because one field is missing.
- (D) Repeat the previous record value for that field.

Answer: (B) Return null for that field and mark it as not found, leaving the rest of the extraction intact.

EXPLANATION

Absence is **information**, and it should be reported faithfully: null plus a not-found marker distinguishes "the document does not contain this" from "we did not look" or "the value is empty". Keeping the rest of the extraction preserves everything that *was* found, so one missing field does not discard a document's worth of correct data.

Why distractors fail:

- **A** - Fabrication, and the most dangerous option: an inferred value is indistinguishable downstream from an extracted one.
- **D** - Cross-record contamination; strictly worse than A because it looks authoritative and correlates across records.
- **C** - Discards valid data over one absent field; missing fields are routine, not exceptional.

EXAM TIP

Exam Rule: - Absence is information, and it should be reported faithfully: `null` plus a not-found marker distinguishes "the document does not contain th

241 An invoice extractor reads dates like 03/04/2025 that could be March 4 or April 3. The design that avoids silent errors is to:

- (A) Assume the United States month-first format everywhere.
- (B) Require an ISO date in the output schema, and when the input is ambiguous, flag the field for review instead of guessing.
- (C) Store the date as the raw string and sort it out later.
- (D) Drop any date that is ambiguous.

Answer: (B) Require an ISO date in the output schema, and when the input is ambiguous, flag the field for review instead of guessing.

EXPLANATION

Two mechanisms addressing the two halves. The **ISO schema requirement** makes the output unambiguous, so nothing downstream has to re-interpret it. **Flagging genuine ambiguity** rather than guessing is what makes the errors non-silent: a 50/50 guess is right half the time and wrong invisibly the other half, whereas a flagged field gets a human decision.

Why distractors fail:

- **A** - Silently wrong on every non-US invoice, which is precisely the "silent error" the question asks to avoid.
- **C** - Defers the same ambiguity to a consumer with *less* context than the extractor had.
- **D** - Discards data that is usually resolvable from other document evidence, and loses the invoice's date entirely.

EXAM TIP

Exam Rule: - Two mechanisms addressing the two halves. The ISO schema requirement makes the output unambiguous, so nothing downstream has to re-interpret

4.2 Few-Shot Prompting & Exemplar Engineering

Claude CCAF: Prompt Engineering & Structured Output - 3 questions

242: "Extract key information from document" produces inconsistent extraction; different documents yield different field selection, and detailed instructions don't improve consistency. Which technique provides consistent extraction?

- (A) More detailed explicit criteria defining exactly which fields constitute "key information".
- (B) Few-shot examples showing concrete input documents and expected extracted output.
- (C) Multiple extraction passes voting on field selection.
- (D) A validation loop rejecting inconsistent extractions until consistency is achieved.

Answer: (B) Few-shot examples showing concrete input documents and expected extracted output.

EXPLANATION

Few-shot input-output examples ground abstract terms like 'key information' in concrete demonstrations, establishing consistent field selection.

EXAM TIP

Subjective/ambiguous task consistency = Few-shot input-output examples.

243 Commit messages in a strict team format - zero-shot with a precise spec, or few-shot with several examples? The task is common and well-defined, and it runs on every commit in a token-billed CI pipeline. BEST choice and why?

- (A) Few-shot, because it always outperforms zero-shot for structured output formatting.
- (B) Zero-shot with a precise format specification, since well-defined common tasks can match few-shot performance while avoiding per-commit token bloat.
- (C) One-shot as a middle ground.
- (D) Few-shot but cache the example block to amortize the token cost.

Answer: (B) Zero-shot with a precise format specification, since well-defined common tasks can match few-shot performance while avoiding per-commit token bloat.

EXPLANATION

For well-defined, common standardized tasks (conventional commits), a precise zero-shot specification matches few-shot accuracy without incurring recurring token overhead per commit.

EXAM TIP

Common well-defined tasks in high-frequency CI = Zero-shot with precise specification.

244 A skills: string[] field shows three issues: compound phrases like "Python and SQL" sometimes kept as one entry; implied-but-unstated skills appearing; array lengths ranging 5-10 vs 40+ on similar documents. The prompt says "Extract all skills mentioned." Most effective improvement?

- (A) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- (B) Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- (C) Post-extraction normalization mapping skills to a canonical taxonomy and deduplicating.
- (D) Enrich the schema to {skill, confidence, source_quote}[] to capture extraction metadata.

Answer: (A) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

EXPLANATION

Granularity, compound phrase splitting, and explicit mention standards are best demonstrated through curated few-shot exemplars rather than arbitrary numeric caps.

EXAM TIP

List granularity / compound phrase splitting = Curated few-shot exemplars showing correct splitting.

4.3 Structured Output, JSON Schemas & Strict Mode

Claude CCAF: Prompt Engineering & Structured Output - 4 questions

245 Output must strictly conform to a complex nested JSON schema with required and optional fields. Which approach guarantees schema-compliant output?

- (A) System prompt with explicit instructions to follow the schema exactly.
- (B) Detailed schema description in the prompt with examples of valid JSON.
- (C) Validation loop checking output against the schema, retrying until valid.
- (D) Use tool_use function calling with a schema parameter enforcing valid structure.

Answer: (D) Use tool_use function calling with a schema parameter enforcing valid structure.

EXPLANATION

tool_use with JSON schema (or output_config.format with strict: true) enforces schema constraints at token generation time, guaranteeing valid structural output.

EXAM TIP

Guaranteed JSON schema output = tool_use schema or output_config.format (strict: true).

246 payment_status enum is ['paid', 'pending', 'overdue'], but documents sometimes use "processing" or "on_hold" and extraction fails. How do you handle enumeration edge cases?

- (A) Expand the enum to include all possible payment status values comprehensively.
- (B) Add an "other" enum value capturing statuses not in the predefined set.
- (C) Remove the enum constraint, using a free-text string field.
- (D) A validation loop retrying extraction until the status matches the enum.

Answer: (B) Add an "other" enum value capturing statuses not in the predefined set.

EXPLANATION

Open-ended real-world domains need an 'other' escape hatch in the enum (often paired with an other_detail string field) to prevent validation failures on unlisted values.

EXAM TIP

Open-ended enum domains = Add 'other' value + details string to avoid validation failures.

247 A required date field causes fabricated dates when the invoice date is unclear. How do you prevent data fabrication?

- (A) Use an optional or nullable date field in the schema, with an "unclear" enum for ambiguous cases.
- (B) Add validation checking extracted dates against invoice content before acceptance.
- (C) Stronger instructions emphasizing "never fabricate information, only extract what exists".
- (D) Few-shot examples showing correct handling of missing dates.

Answer: (A) Use an optional or nullable date field in the schema, with an "unclear" enum for ambiguous cases.

EXPLANATION

Forcing required fields on data that may be missing creates structural pressure to fabricate. Making the field nullable or optional eliminates the fabrication incentive.

EXAM TIP

Prevent fabrication on missing data = Make field nullable/optional in schema.

248A response must always be {"status": "success"|"error", "data": object, "metadata": object}, guaranteed on every response with no exceptions. Which approach provides the absolute strongest guarantee?

- (A) output_config with json_schema type, strict: true, and the specified schema.
- (B) Few-shot examples showing 5 different correct JSON responses.
- (C) System prompt with an explicit JSON template and instructions to follow it exactly.
- (D) Tool definition with the schema and tool_choice forced to that tool.

Answer: (A) output_config with json_schema type, strict: true, and the specified schema.

EXPLANATION

output_config.format with strict: true constrains the assistant's primary text response directly into the specified JSON grammar, providing the absolute strongest guarantee.

EXAM TIP

Absolute strongest response format guarantee = output_config with strict: true JSON schema.

4.4 Validation & Feedback Retry Loops

Claude CCAF: Prompt Engineering & Structured Output - 2 questions

249Claude generates a response, a separate validation step checks it against business rules, failures are fed back for correction, and this repeats until validation passes or a maximum attempt count is reached. What is this pattern called?

- (A) The convergence pattern
- (B) The feedback pattern
- (C) The validation-retry loop
- (D) The retry pattern

Answer: (C) The validation-retry loop

EXPLANATION

The validation-retry loop combines deterministic verification against business rules with structured error feedback fed back to the generator over a bounded attempt budget.

EXAM TIP

Generate -> Validate -> Feed error back -> Retry = Validation-Retry Loop.

250A validation-retry loop still fails on the same documents after three attempts each. In which situations will further retries not help?

- (A) The model placed a correct value in the wrong field on the last attempt.
- (B) The required information is simply absent from the source document (or exists only in an external unprovided document).
- (C) The validation service timed out on the last attempt.
- (D) The prompt format had minor whitespace variations.

Answer: (B) The required information is simply absent from the source document (or exists only in an external unprovided document).

EXPLANATION

Retries cannot recover information that does not exist in context. Retrying missing data causes loops to hit max attempts or hallucinate values.

EXAM TIP

Retries fail when required data is absent from input context.

4.5 Batch Processing Strategy & Multi-Pass Pipelines

Claude CCAF: Prompt Engineering & Structured Output - 2 questions

251Documents arrive continuously through business hours; you want the Batch API (50% discount, up-to-24-hour window). The SLA requires results within 30 hours of document arrival at 99.9% reliability. Which batching strategy is most appropriate?

- (A) Submit batches every 6 hours containing documents from that window.
- (B) Submit a single batch at end of day containing all documents from that day.
- (C) Submit batches every 4 hours containing documents from that window.
- (D) Use the real-time API for all documents instead of batch processing.

Answer: (C) Submit batches every 4 hours containing documents from that window.

EXPLANATION

Worst case latency = Max Queue Wait + 24h processing window. 4h wait + 24h processing = 28h worst-case (providing 2h safety margin against the 30h SLA). A 6h wait leaves zero margin (30h exact).

EXAM TIP

Batch SLA math: Worst-case latency = Batch interval + 24h processing window.

252. Two document types share a schema: standard monthly reports (archived after processing) and urgent exception reports (must trigger alerts within 30 minutes). Minimise API cost while meeting latency. How should the pipeline be architected?

(A) Submit all documents to the real-time Messages API for consistent latency.

(B) Submit all to the Batch API with custom_ids; when results arrive, immediately process urgent documents and trigger delayed alerts.

(C) Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when results return.

(D) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

Answer: (D) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

EXPLANATION

Split routing by latency SLA: standard reports capture the 50% Batch API discount over 24h, while urgent sub-30-minute alerts use the real-time Messages API.

EXAM TIP

Hybrid batch routing: Real-time API for low-latency alerts (<1h); Batch API for bulk tolerant work (24h).

